

计算机视觉——基元检测

2026年春季

桑 农

第5章 基元检测

- 概念
- 基元：泛指图像中有特点的基本单元，比如边缘、角点、直线段、圆、孔（及它们结合体）等。也称为特征，因此基元检测也称为特征检测

第5章 基元检测

5.1 特征点的概念及作用

5.2 SUSAN算子

5.3 Harris算子

5.4 SIFT算子

5.5 SURF算子

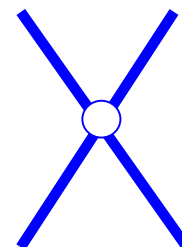
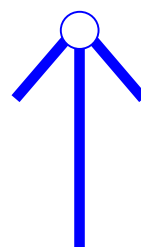
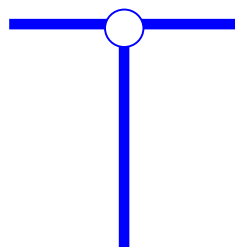
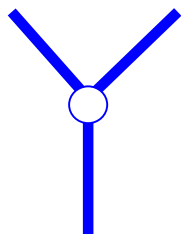
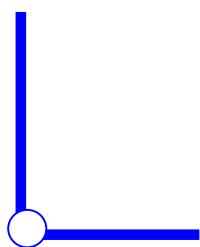
5.6 FAST算子

5.7 SuperPoint深度网络

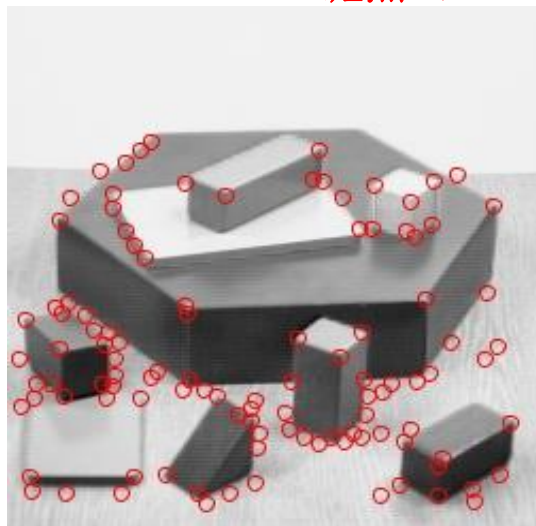
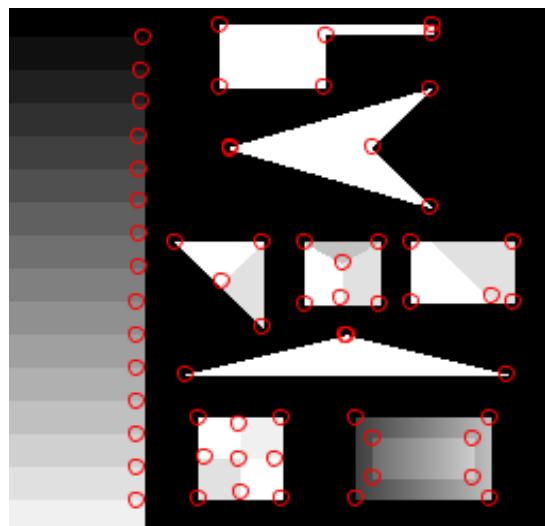
5.8 其他典型深度网络

5.1 特征点的概念

- 与周围像素点具有不同特性的像素，比如角点（corner point）

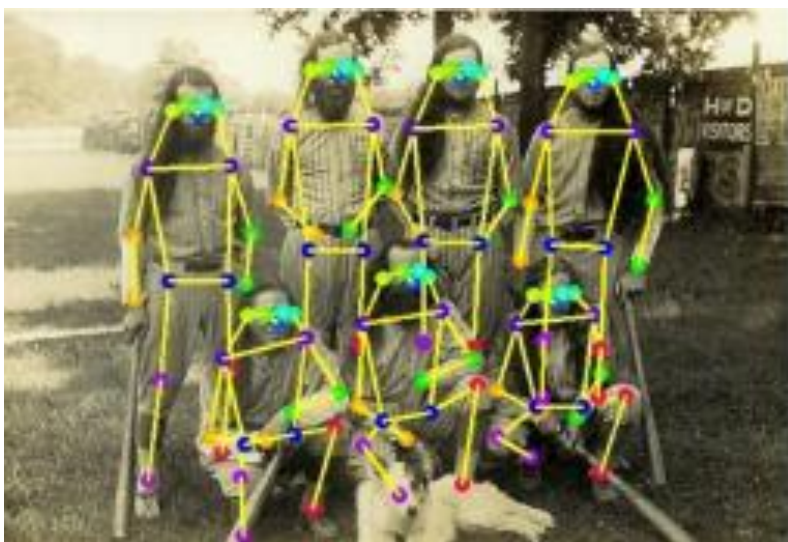


斑点 (blob) 中心



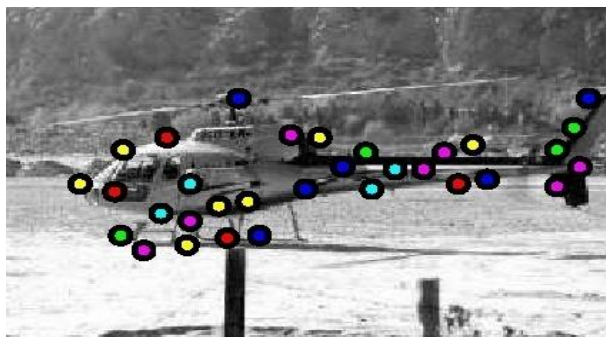
5.1 特征点的概念

- 三种相似的术语
 - 特征点：feature point，统称
 - 兴趣点：interest point，传统
 - 关键点：keypoint，有时包含了一定的语义

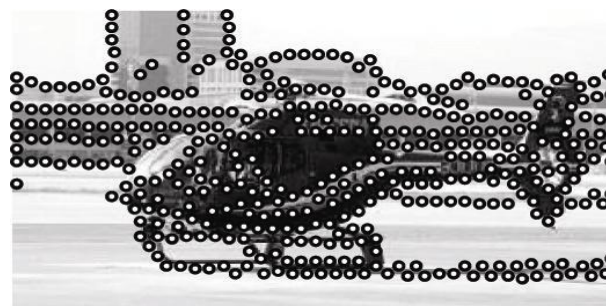


5.1 特征点的作用

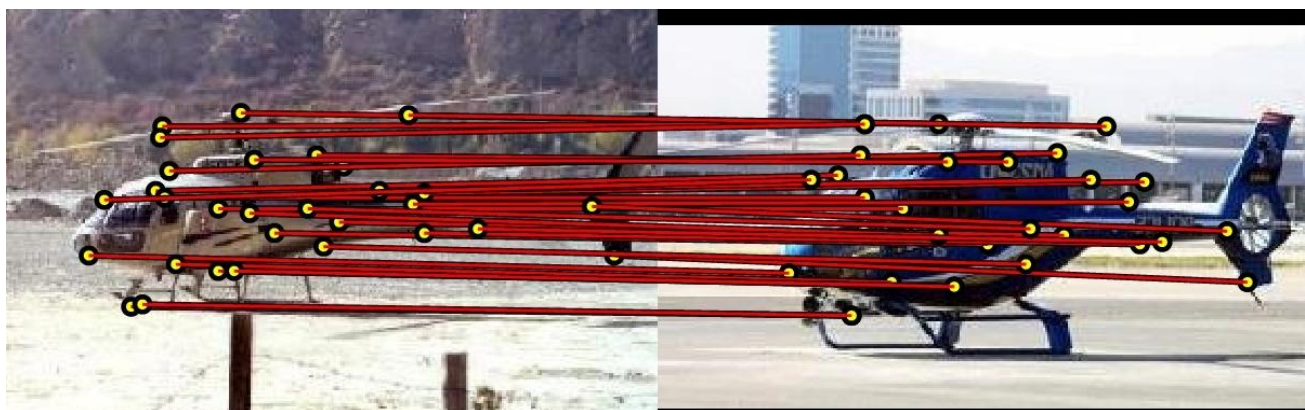
- 反映目标形状：通过特征点匹配可实现目标识别



模板图



待匹配图



5.1 特征点的作用

- 反映目标姿态：通过特征点匹配可分析目标姿态，进而判断其行为



5.1 特征点的作用

- 进行人体姿态转换：通过特征点匹配实现人体的姿态转换

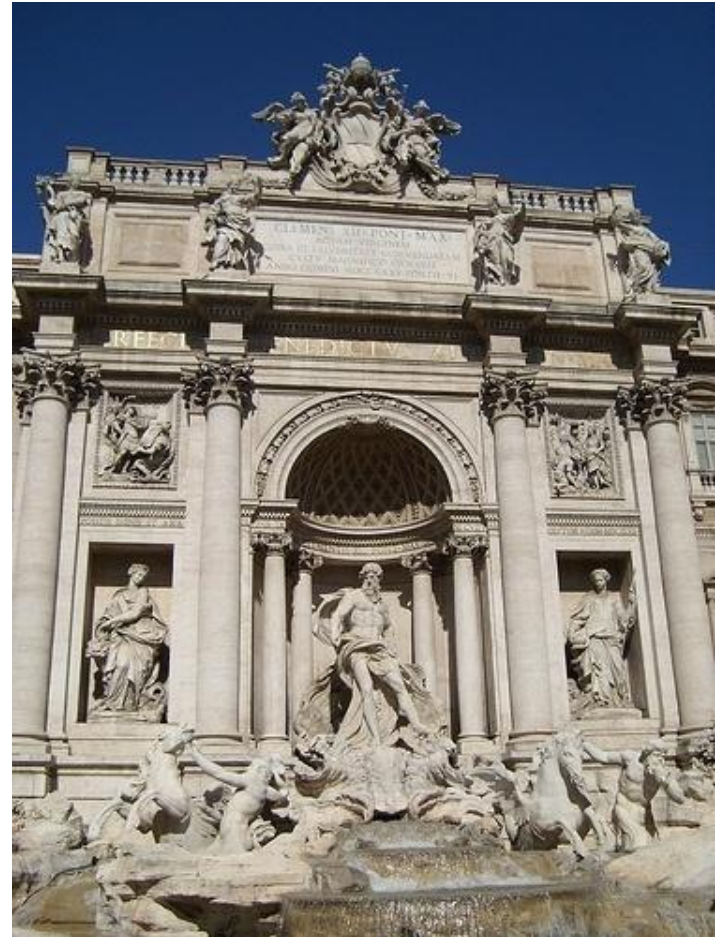


5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接



by [Diva Sian](#)



by [swashford](#)

5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接



by [Diva Sian](#)



by [scgbt](#)

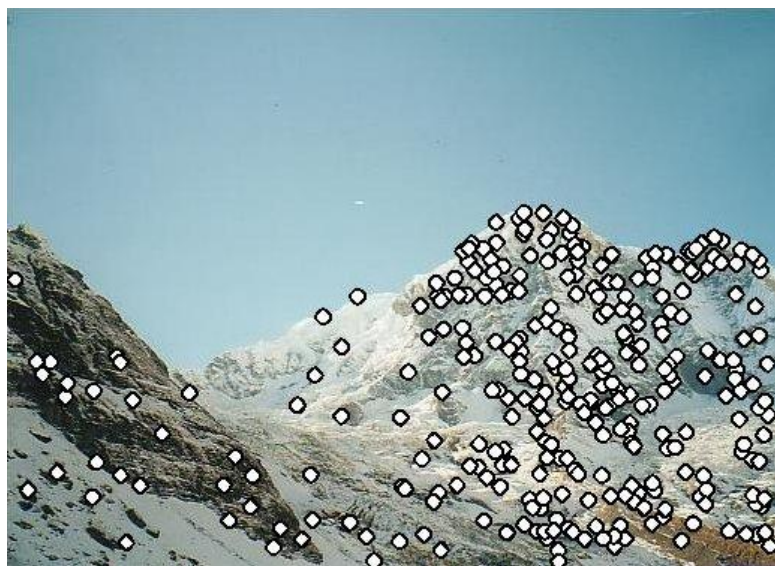
5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接



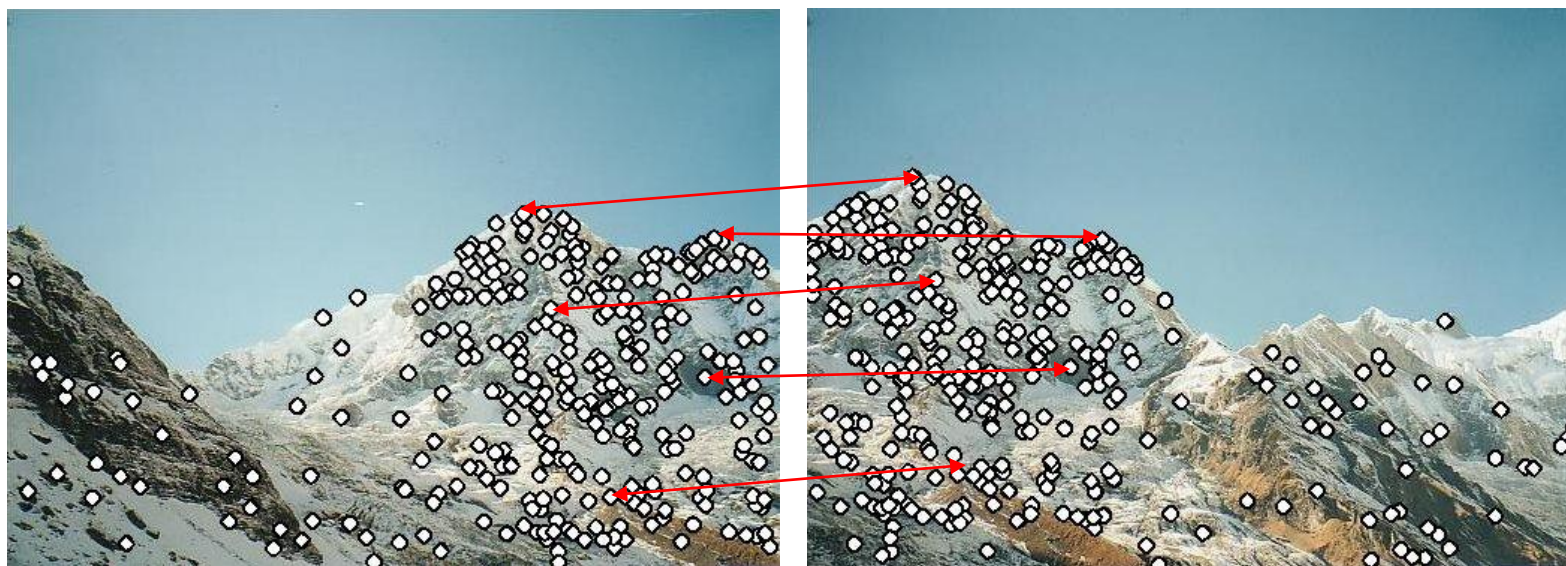
5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接
 - 在两幅图像中检测特征点



5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接
 - 在两幅图像中检测特征点
 - 寻找对应点



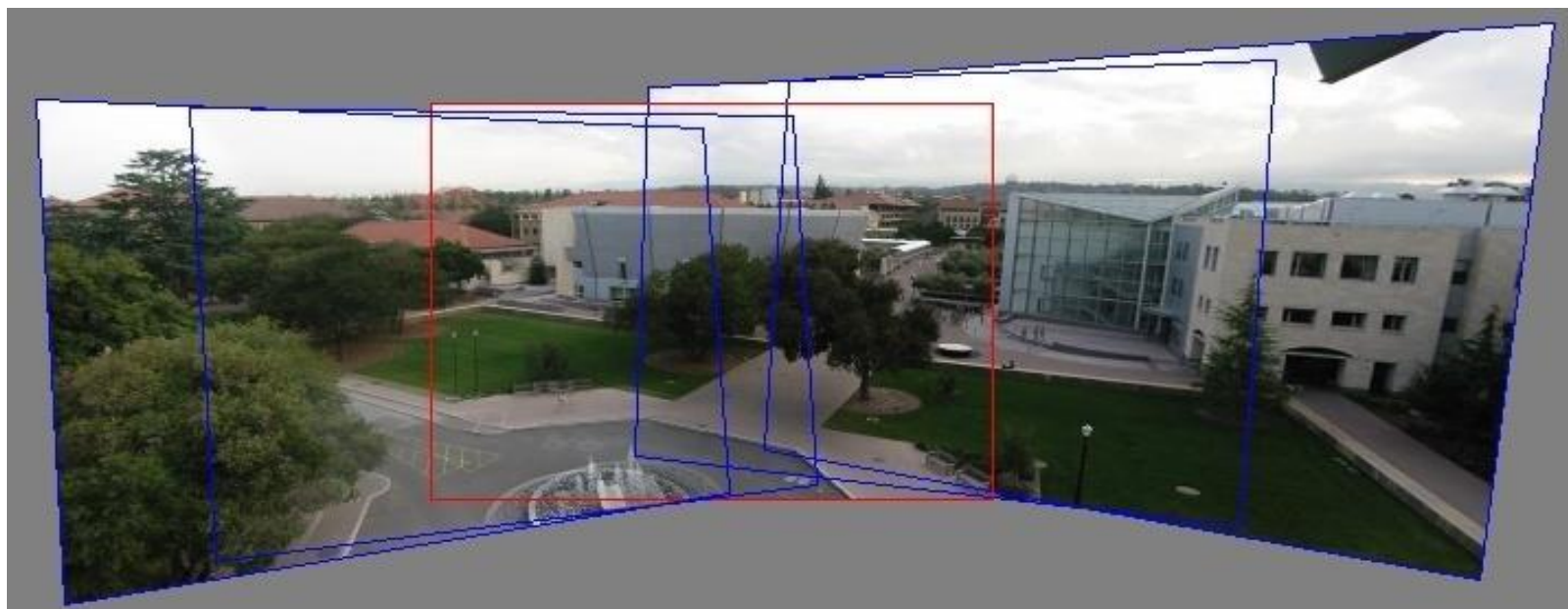
5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接
 - 在两幅图像中检测特征点
 - 寻找对应点
 - 利用点对计算两图间的变换关系并拼接图像



5.1 特征点的作用

- 反映场景特征：通过特征点匹配实现图像配准与拼接
 - 手机的全景相机



5.1 特征点的作用

- 立体匹配

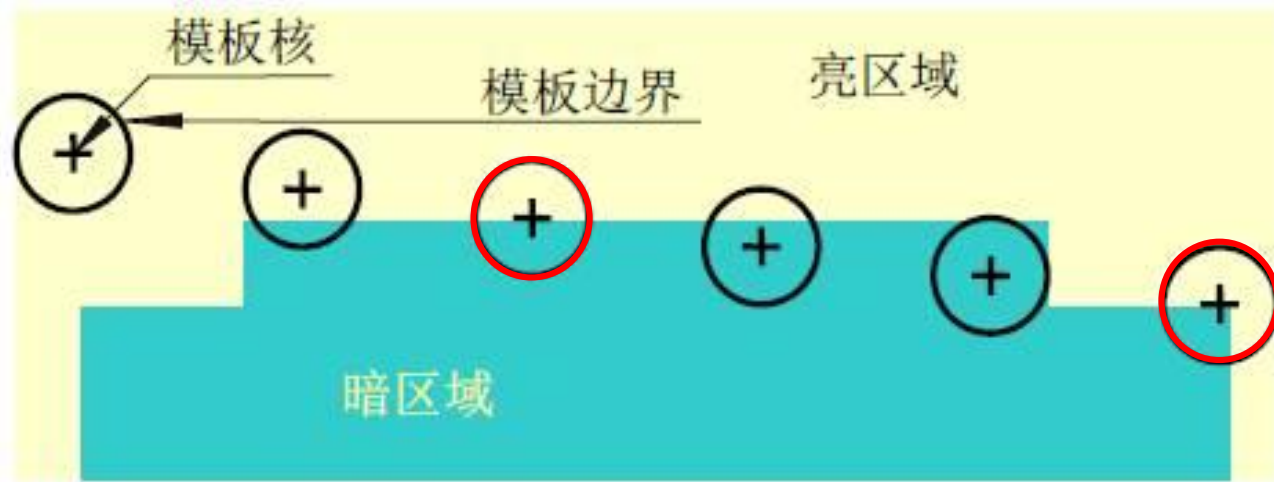
— 演示

5.2 SUSAN算子

- SUSAN: **S**mallest **U**nivalued **S**egment **A**ssimilating **N**ucleus (最小同值核)
- Stephen M. Smith, J. Michael Brady, **SUSAN—A New Approach to Low Level Image Processing**, International Journal of Computer Vision, 1997, 23(1): 45–78

5.2 SUSAN算子——USAN

- 核同值区域（univalue segment assimilating nucleus, USAN），即与核有相同值的区域

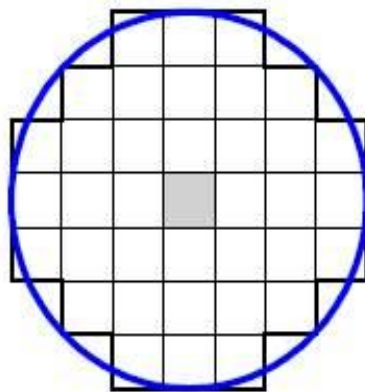


- 特点：
 - 可用于检测角点、边缘
 - 不需要计算微分，对噪声不敏感

理论上小于 $S_{max}/2$ 为角点，等于 $S_{max}/2$ 为边缘：对于离散模板而言，不满足¹⁸

5.2 SUSAN算子——原理

- 设定模板
 - 通常取矩形模板



- 将模板内每个像素的灰度值与核的灰度值进行比较

灰度差阈值

$$C(x_0, y_0; x, y) = \begin{cases} 1 & \text{当 } |f(x_0, y_0) - f(x, y)| \leq T \\ 0 & \text{当 } |f(x_0, y_0) - f(x, y)| > T \end{cases}$$

5.2 SUSAN算子——原理

- 输出游程（run-length）和

$$S(x_0, y_0) = \sum_{(x,y) \in M(x,y)} C(x_0, y_0; x, y)$$

- 计算响应 $R(x_0, y_0)$

$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

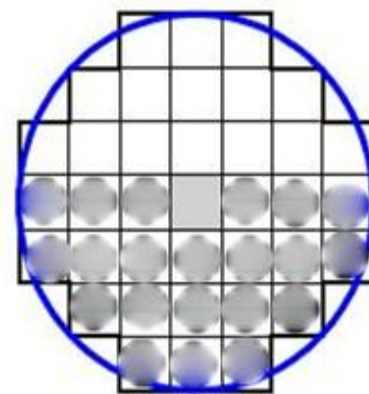
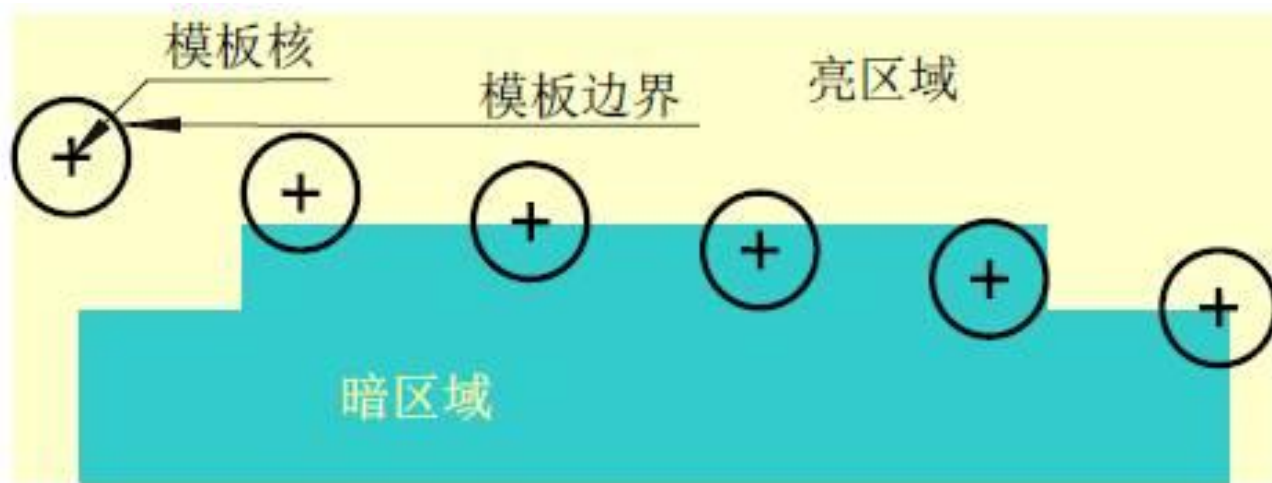
几何阈值

5.2 SUSAN算子——原理

- 对于阶跃边缘， s 的值总会在某一边小于 $s_{\max}/2$?

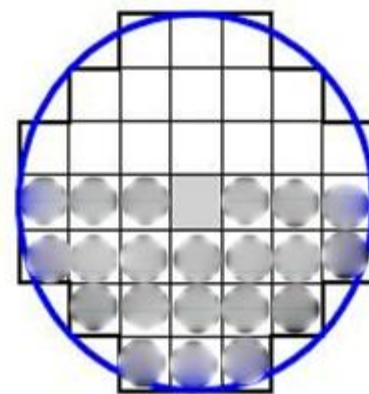
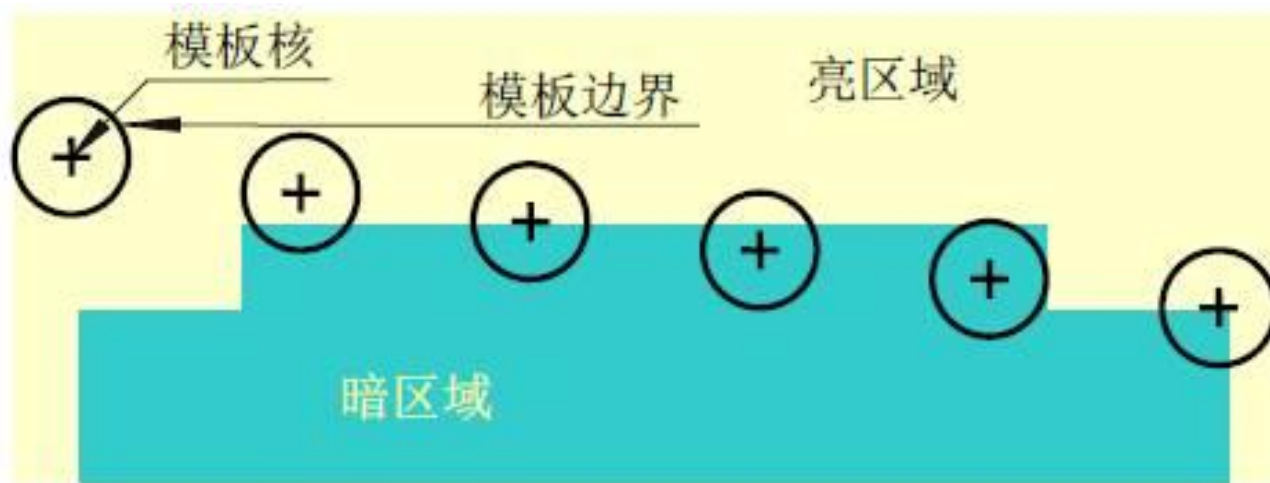
5.2 SUSAN算子——原理

- 对于阶跃边缘， s 的值总会在某一边小于 $s_{\max}/2$?



5.2 SUSAN算子——原理

- 对于阶跃边缘， s 的值总会在某一边小于 $s_{\max}/2$? $\times$

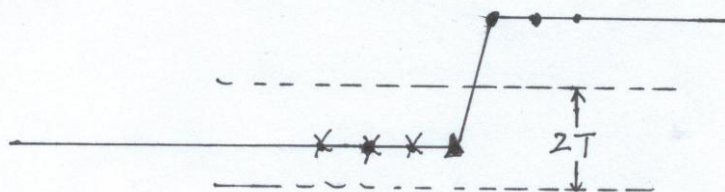


5.2 SUSAN算子——原理

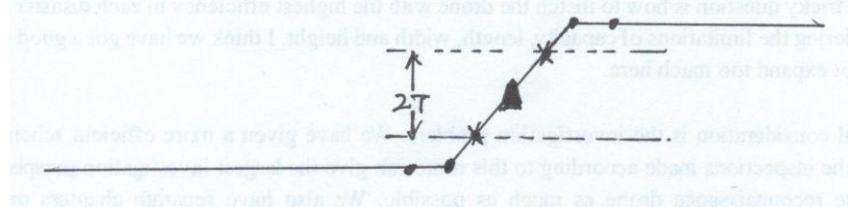
- 对于阶跃边缘， s 的值总会在某一边小于 $s_{\max}/2$? $\times$
- 对于非阶跃边缘， s 的值会更小，边缘检测不到的可能性更小？

5.2 SUSAN算子——原理

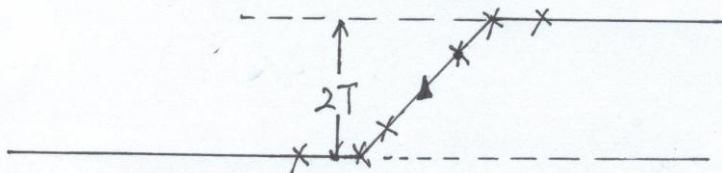
- 对于阶跃边缘， S 的值总会在某一边小于 $S_{\max}/2$? $\times$
- 对于非阶跃边缘， S 的值会更小，边缘检测不到的可能性更小?



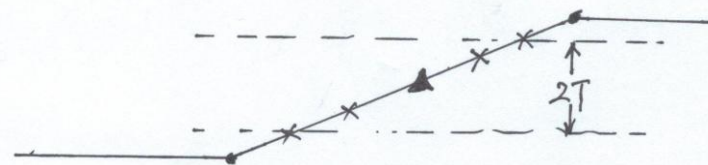
阶跃边缘, $S=3$



非阶跃边缘, $S=2$



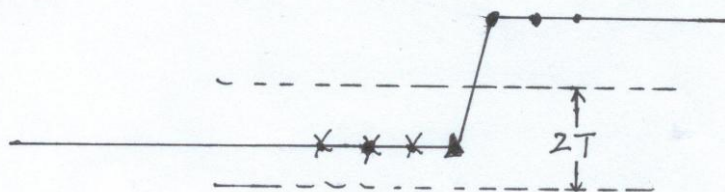
非阶跃边缘, 增大 T , $S=6$



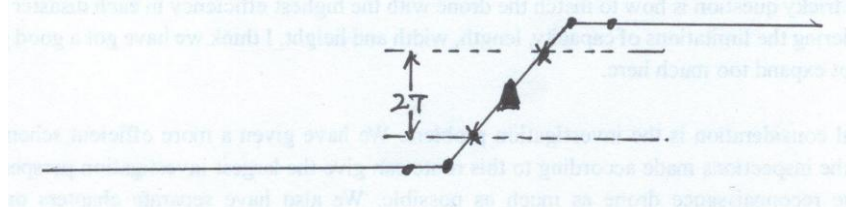
非阶跃边缘, 边缘变平坦, $S=4$

5.2 SUSAN算子——原理

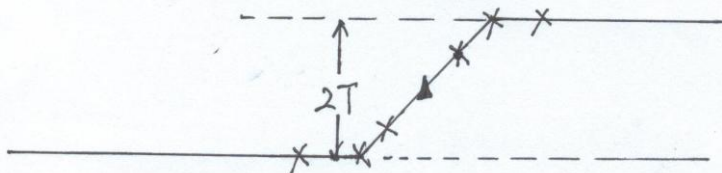
- 对于阶跃边缘， S 的值总会在某一边小于 $S_{\max}/2$? $\times$
- 对于非阶跃边缘， S 的值会更小，边缘检测不到的可能性更小? 取决于 T 和边缘的平滑度



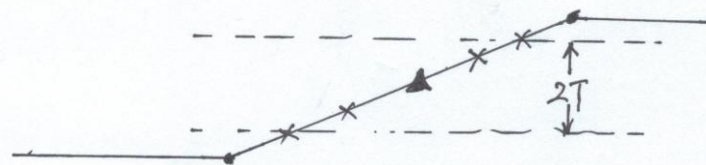
阶跃边缘, $S=3$



非阶跃边缘, $S=2$



非阶跃边缘, 增大 T , $S=6$



非阶跃边缘, 边缘变平坦, $S=4$

5.2 SUSAN算子——原理

- 更稳定的计算 $C(.;.)$ 的公式

$$C(x_0, y_0; x, y) = \exp \left\{ - \left[\frac{f(x_0, y_0) - f(x, y)}{T} \right]^2 \right\}$$

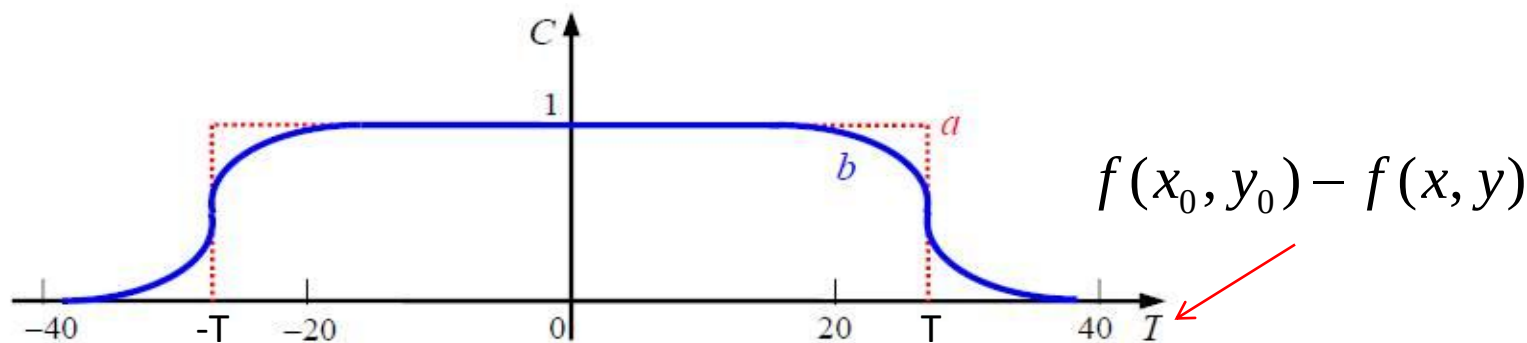
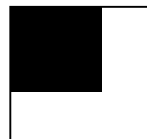


图 5.2.4 不同的函数 $C(.;.)$ 示例

5.2 SUSAN算子——实验



SUSAN模板



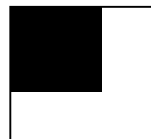
	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

5.2 SUSAN算子——实验



SUSAN模板



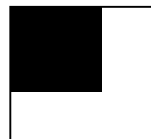
	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

5.2 SUSAN算子——实验



SUSAN模板

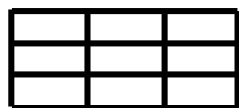


	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

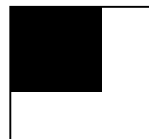
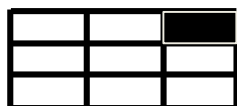
原图



5.2 SUSAN算子——实验



SUSAN模板



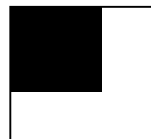
	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

5.2 SUSAN算子——实验

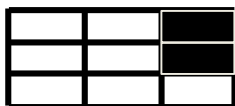


SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

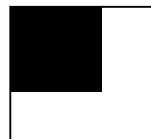
原图



5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

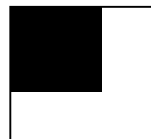
原图



5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

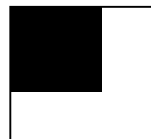
原图



5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

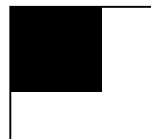
原图



5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

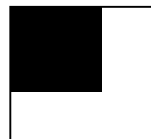


$S=3 (3/8)$

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

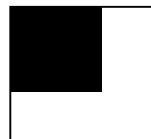


$S=3 (3/8)$

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

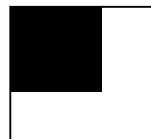


$S=3 (3/8)$

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图



$S=3 (3/8)$

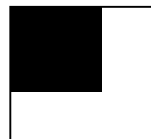


$S=7 (7/8)$

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图



$S=3$ (3/8)

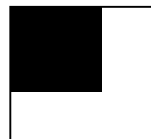


$S=7$ (7/8)

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

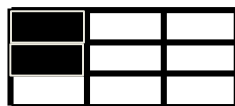
原图



$S=3$ (3/8)



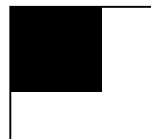
$S=7$ (7/8)



5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

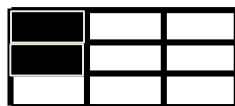
原图



$S=3$ (3/8)



$S=7$ (7/8)

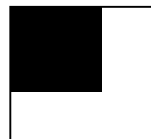


$S=6$ (6/8)

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	0	255	255	255
6	0	0	0	0	0	0	0	0	255	255	255
7	0	0	0	0	0	0	0	0	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

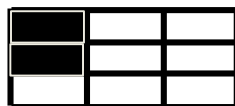
原图



$S=3$ (3/8)



$S=7$ (7/8)

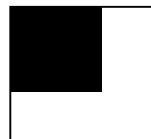


$S=6$ (6/8)

5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	0	255	255	255
6	0	0	0	0	0	0	0	0	255	255	255
7	0	0	0	0	0	0	0	0	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

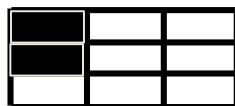
原图



S=3 (3/8)



S=7 (7/8)



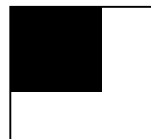
S=6 (6/8)



5.2 SUSAN算子——实验



SUSAN模板



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

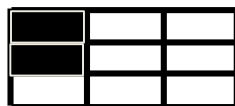
原图



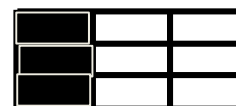
S=3 (3/8)



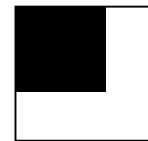
S=7 (7/8)



S=6 (6/8)



S=5 (5/8)

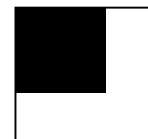


SUSAN模板

	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	255	255	255	255
2	0	0	0	0	0	0	0	255	255	255	255
3	0	0	0	0	0	0	0	255	255	255	255
4	0	0	0	0	0	0	0	255	255	255	255
5	0	0	0	0	0	0	0	255	255	255	255
6	0	0	0	0	0	0	0	255	255	255	255
7	0	0	0	0	0	0	0	255	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	5	5	5	5	5	3	6	8	8	
	5	5	5	5	5	6	7	8	8	
	8	8	8	8	8	8	8	8	8	
	8	8	8	8	8	8	8	8	8	

原图



SUSAN模板

$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

$$3S_{\max}/4 = 6$$

	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	0	255	255	255
2	0	0	0	0	0	0	0	0	255	255	255
3	0	0	0	0	0	0	0	0	255	255	255
4	0	0	0	0	0	0	0	0	255	255	255
5	0	0	0	0	0	0	0	0	255	255	255
6	0	0	0	0	0	0	0	0	255	255	255
7	0	0	0	0	0	0	0	0	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	5	5	5	5	5	3	6	8	8	
	5	5	5	5	5	6	7	8	8	
	8	8	8	8	8	8	8	8	8	
	8	8	8	8	8	8	8	8	8	

原图

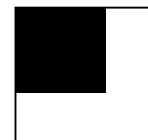


SUSAN模板

$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

$$3S_{\max}/4 = 6$$

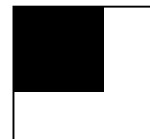
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	8	8	8	8	8	5	5	8	8	
	5	5	5	5	5	3	6	5	5	
	5	5	5	5	5	6	7	8	8	
	8	8	8	8	8	8	8	8	8	
	8	8	8	8	8	8	8	8	8	



	1	2	3	4	5	6	7	8	9	10	11
1	0	0	0	0	0	0	0	0	255	255	255
2	0	0	0	0	0	0	0	0	255	255	255
3	0	0	0	0	0	0	0	0	255	255	255
4	0	0	0	0	0	0	0	0	255	255	255
5	0	0	0	0	0	0	0	0	255	255	255
6	0	0	0	0	0	0	0	0	255	255	255
7	0	0	0	0	0	0	0	0	255	255	255
8	255	255	255	255	255	255	255	255	255	255	255
9	255	255	255	255	255	255	255	255	255	255	255
10	255	255	255	255	255	255	255	255	255	255	255
11	255	255	255	255	255	255	255	255	255	255	255

原图

理论上小于
 $S_{\max}/2$ 为角点，
等于 $S_{\max}/2$ 为边
缘


$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

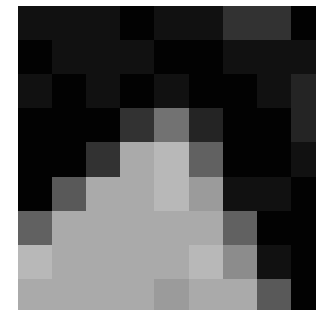
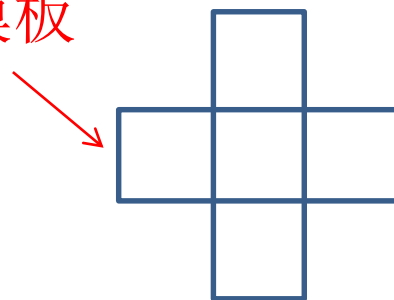
$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

$$3S_{\max}/4=6$$

[illegible][illegible][illegible]

SUSAN算子——实例

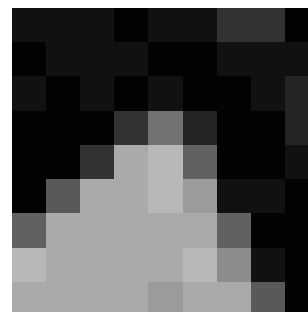
模板



13	26	25	13	14	29	50	45	11
11	17	17	9	6	9	11	24	20
6	10	8	7	18	8	5	16	36
5	8	2	55	114	27	6	7	38
74	13	57	155	184	94	5	6	24
70	76	162	177	183	150	26	7	6
97	171	177	167	172	170	89	4	5
176	173	168	167	167	178	140	24	6
178	166	165	165	163	171	166	79	2

- 模板； 阈值设为 $G=3S_{\max}/4$ ； 灰度相似性阈值 $T=67$

SUSAN算子——实例



0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	2	0	0	0	0	0
0	0	1	1	1	1	0	0	0	0
0	2	1	0	0	0	0	0	0	0
0	1	0	0	0	0	1	0	0	0
0	0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0	0

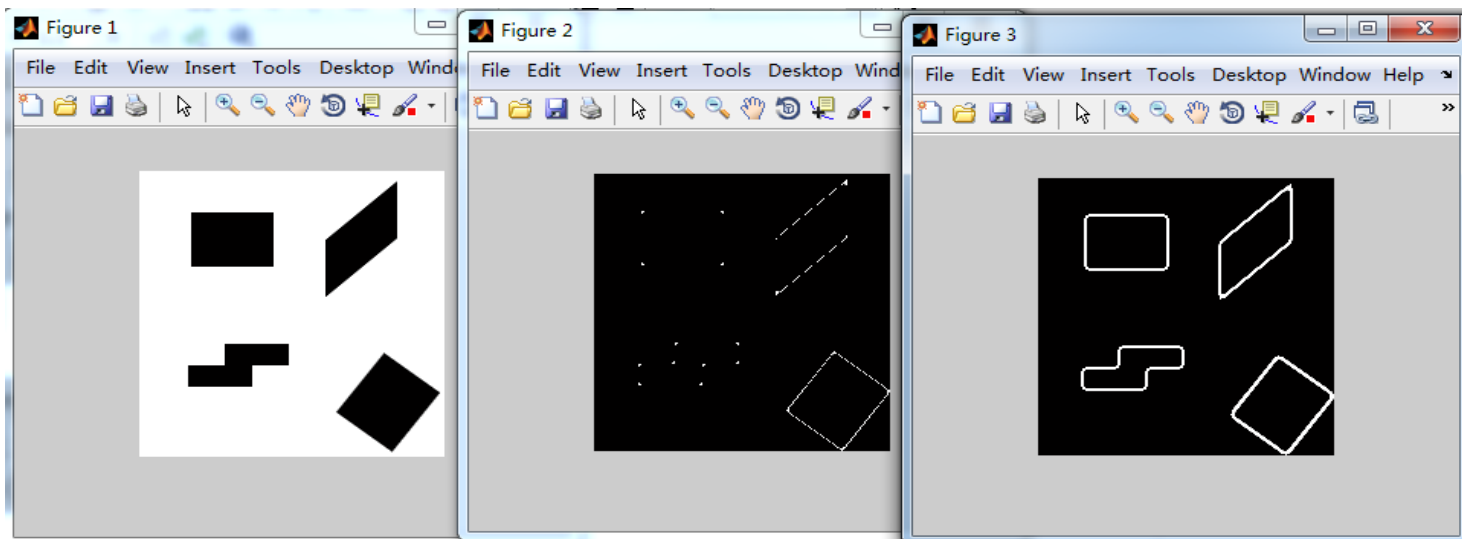
- 模板； 阈值设为 $G=3S_{\max}/4$ ； 灰度相似性阈值 $T=67$

5.2 SUSAN算子——实验



图 5.2.5 用 SUSAN 算子检测到的角点

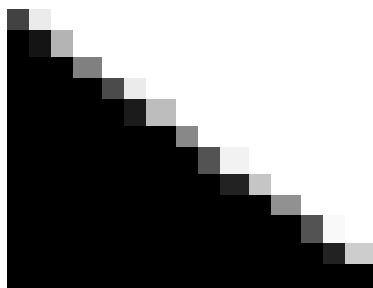
5.2 SUSAN算子——实验



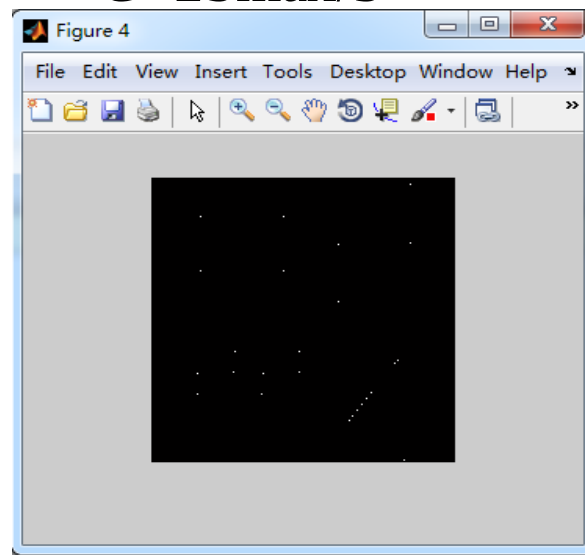
原图

$G=1S_{max}/2$

$G=2S_{max}/3$



非最大抑制
(non maximum
suppression)



5.2 SUSAN算子——特点

- 无微分计算，抗噪性强
- 对边缘的响应随着边缘的平滑或模糊而增强
- 能提供不依赖于模板尺寸的边缘精度
- 控制参数（**T**和**G**）的选择很简单，且任意性较小

T越小 → 核同值区越小 → 检测到的边缘点和角点越多

G越小 → 对检测得到的边缘点和角点要求越高 → 角点越尖

作业

- 5.6

5.6 如果用排成 5 行的 13 个像素来近似表示圆形模板(各行分别有 1, 3, 5, 3, 1 个像素), 这相当于一个半径约为多少个像素的圆? 此时灰度差的阈值应选多少?

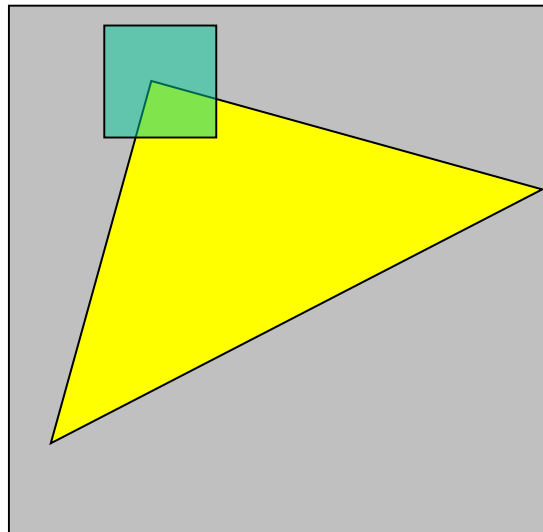
几何

5.3 Harris算子

- Chris Harris, Mike Stephens, **A Combined Corner and Edge Detector**, 4th Alvey Vision Conference, 1988, pp.147-151

5.3 Harris算子——原理

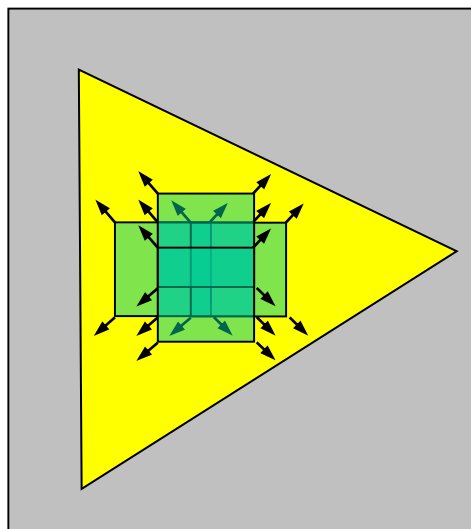
- 基本思想
 - 通过一个小的滑动窗口检测角点
 - 若窗口以角点为中心，那么在任意方向上平移该窗口，窗口内的灰度值都将会有剧烈的变化



5.3 Harris算子——原理

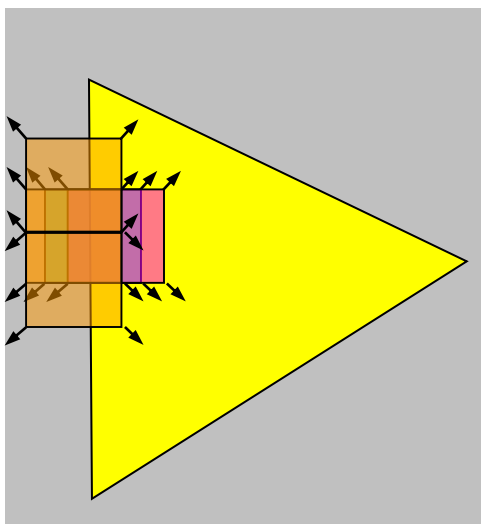
- 基本思想

平滑区域



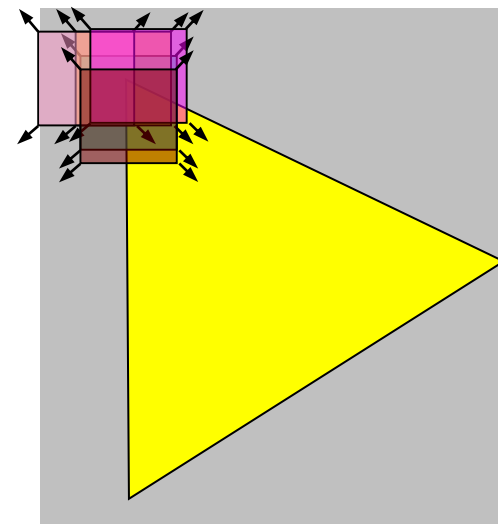
各个方向
基本不变

边缘



边缘方向
基本不变

角点



各个方向均
有剧烈变化

5.3 Harris算子——原理

- 设平移量为 (u, v) ，用图像的自相关函数表示窗口内灰度值的变化

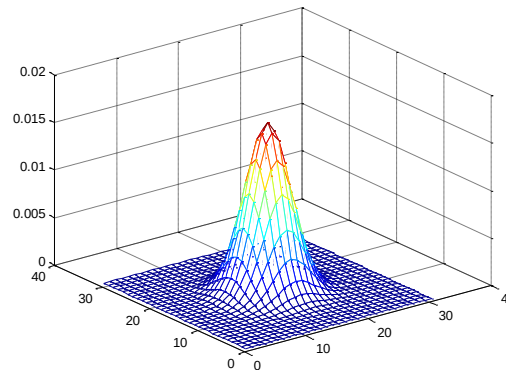
$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

窗口函数

平移后的
灰度值

灰度值

窗口函数 $w(x, y) =$



Gaussian

5.3 Harris算子——原理

- 用Taylor展开近似任意方向

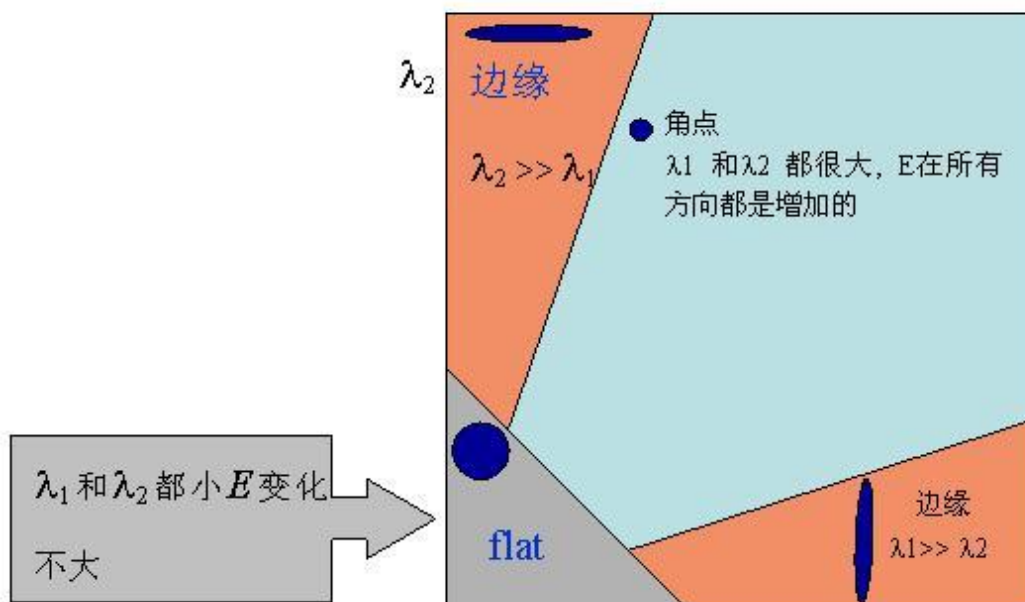
$$\begin{aligned} E(u, v) &= \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2 \\ &= \sum_{x, y} w(x, y) [I_x u + I_y v + o(u^2, v^2)]^2 \end{aligned}$$

- 表达为矩阵形式

$$\begin{aligned} E(u, v) &\cong [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix} \\ M &= \sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix} \end{aligned}$$

5.3 Harris算子——原理

- 计算矩阵 M 的两个特征值
- 若两个都很大就是角点，一大一小就是边缘，两个都小就是在变化缓慢的图像区域



5.3 Harris算子——原理

- 利用矩阵 M 两个特征值的和等于 M 的迹，两个特征值的积等于 M 的行列式，计算Harris角点响应值

$$R = \det M - k(\text{Trace}M)^2 \quad k = 0.04 \sim 0.06$$

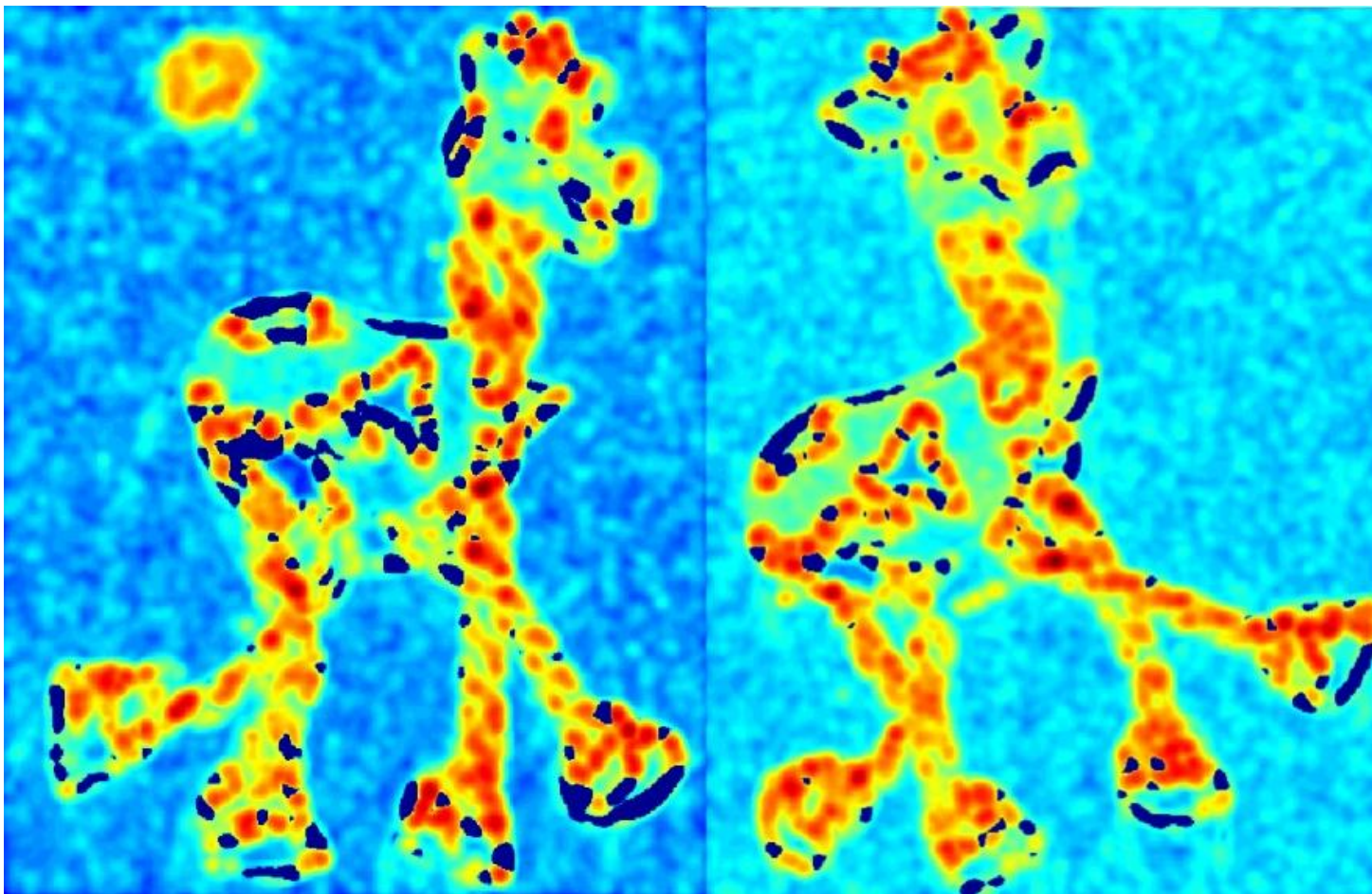
- 响应值大于阈值的点为候选点
- 非最大抑制去除干扰

5.3 Harris算子——实验



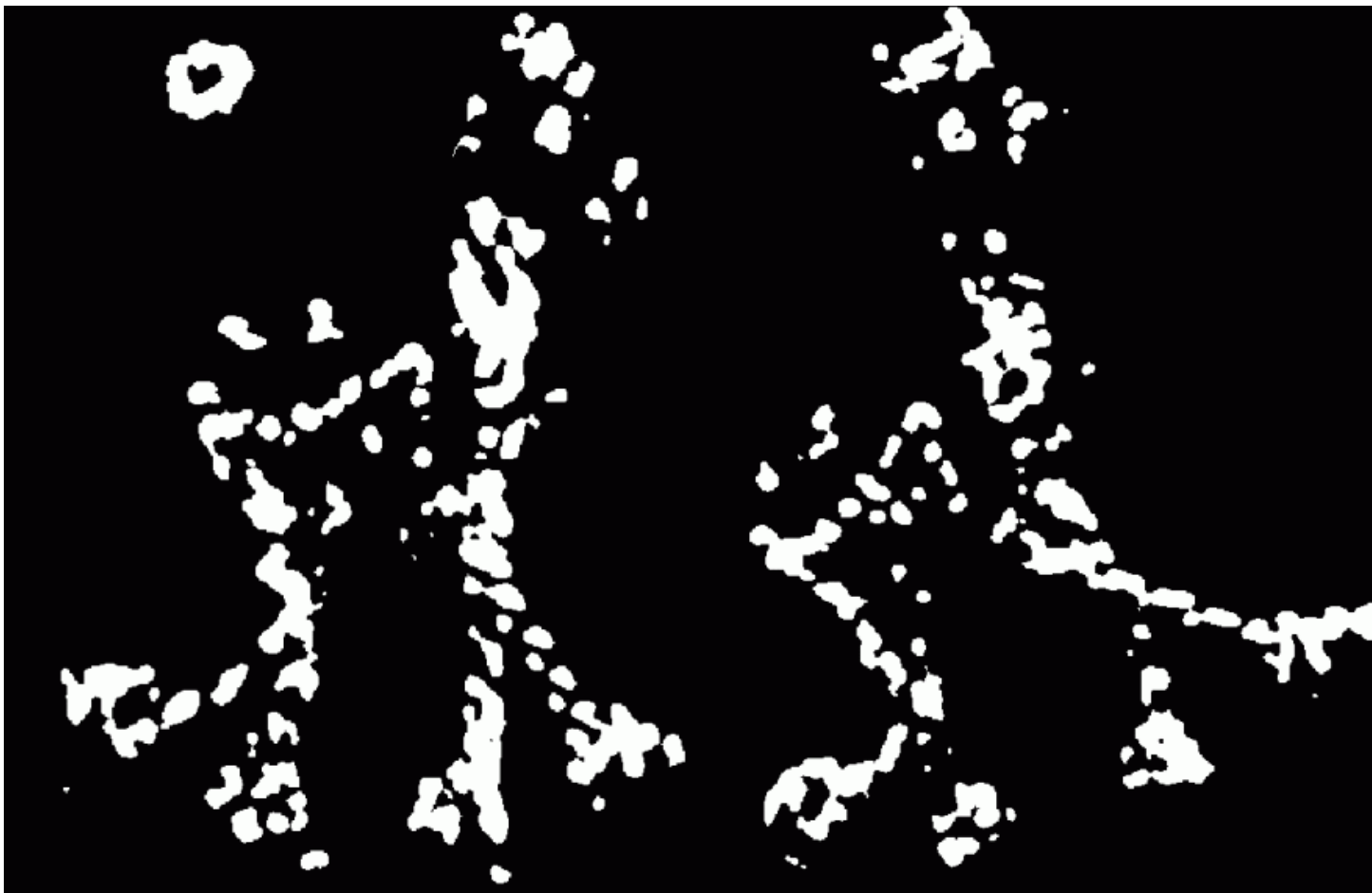
原始图像

5.3 Harris算子——实验



角点响应图

5.3 Harris算子——实验



响应图阈值化

5.3 Harris算子——实验



非最大抑制

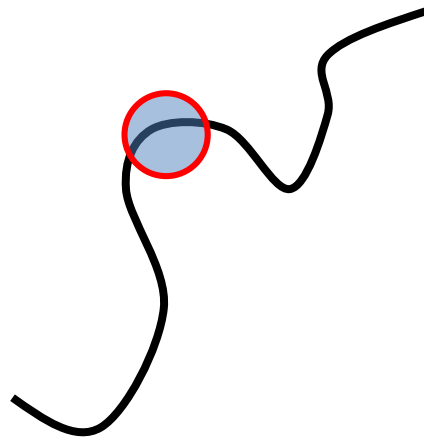
5.3 Harris算子——实验



角点检测结果

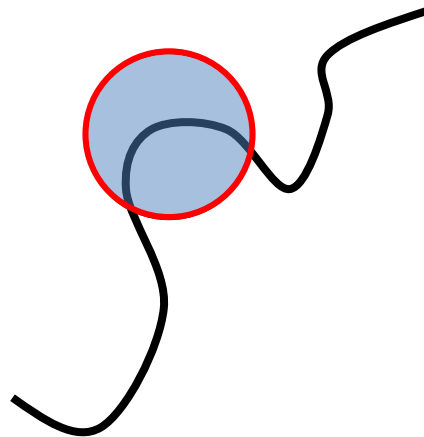
角点自动尺度选择

- 尺度变化检测
 - 角点应该在尺度和位置上都是变化最大



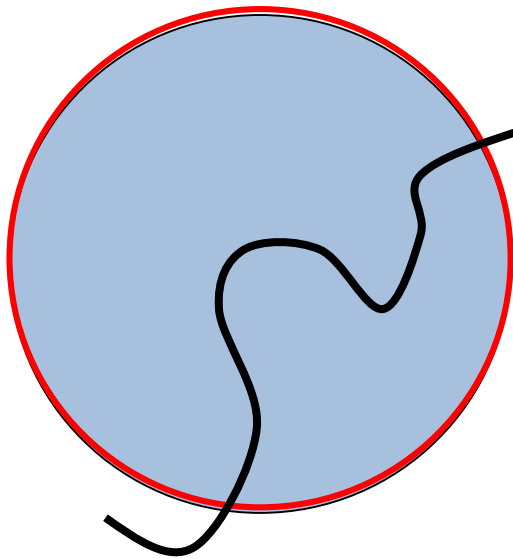
角点自动尺度选择

- 尺度变化检测
 - 角点应该在尺度和位置上都是变化最大



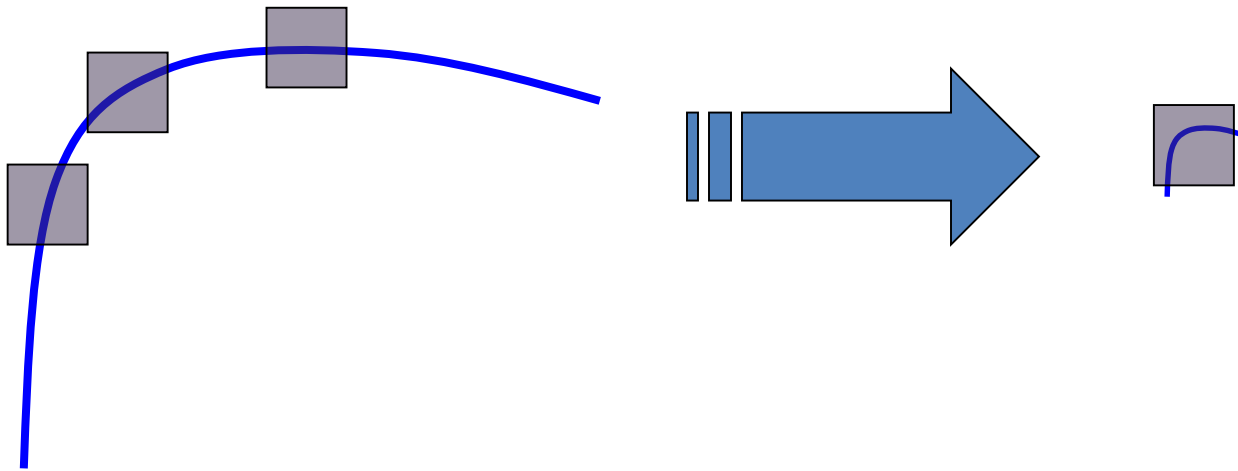
角点自动尺度选择

- 尺度变化检测
 - 角点应该在尺度和位置上都是变化最大



角点自动尺度选择

- 基于固定窗口的角点检测方法对尺度敏感

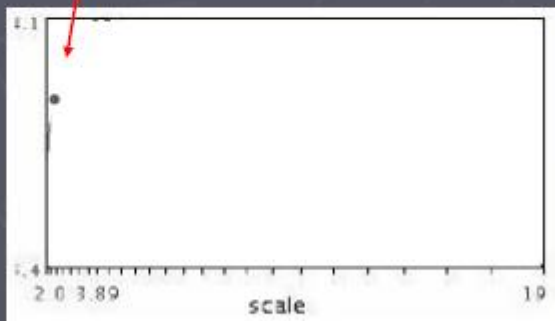


大尺度上可能检测为边缘

小尺度上为角点!

角点自动尺度选择

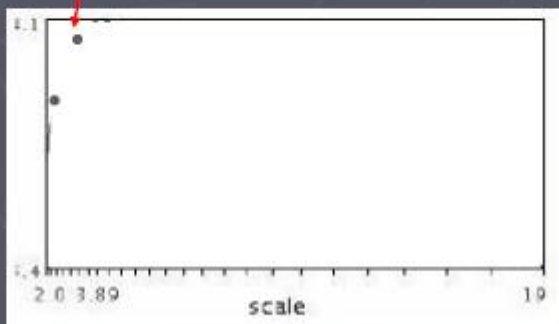
Lindeberg et al., 1996



$$f(I_{l_1...l_m}(x, \sigma))$$

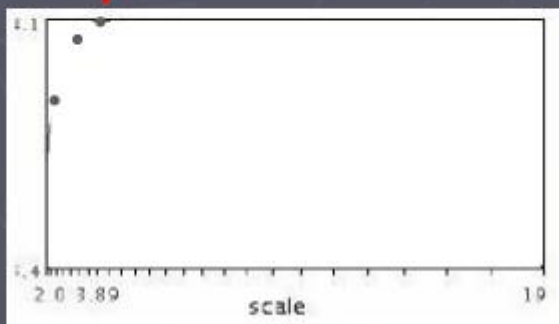
from Tinne Tuytelaars

角点自动尺度选择



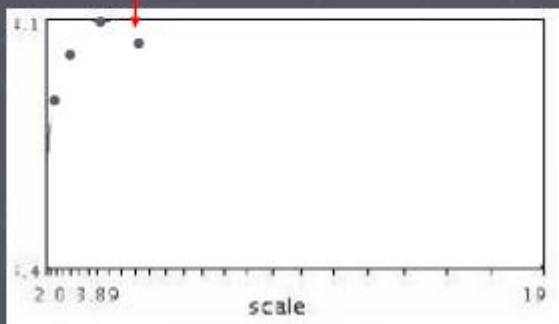
$$f(I_{l_1...l_m}(x, \sigma))$$

角点自动尺度选择



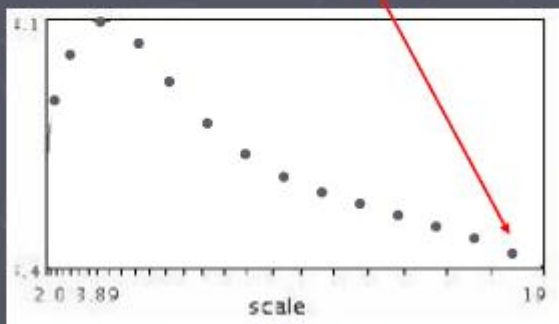
$$f(I_{l_1...l_m}(x, \sigma))$$

角点自动尺度选择



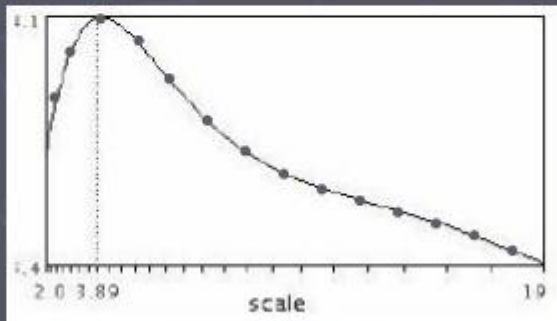
$$f(I_{l...l_m}(x, \sigma))$$

角点自动尺度选择



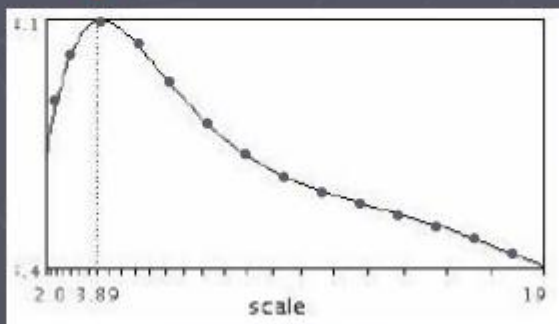
$$f(I_{l \dots l_m}(x, \sigma))$$

角点自动尺度选择

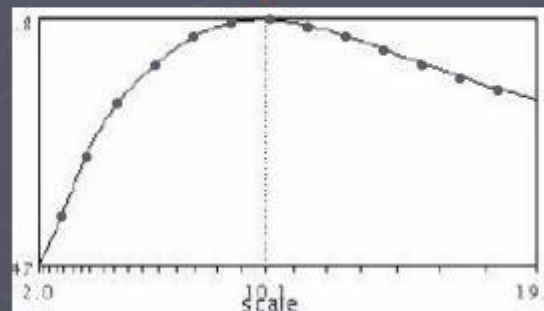


$$f(I_{l_1...l_m}(x, \sigma))$$

角点自动尺度选择



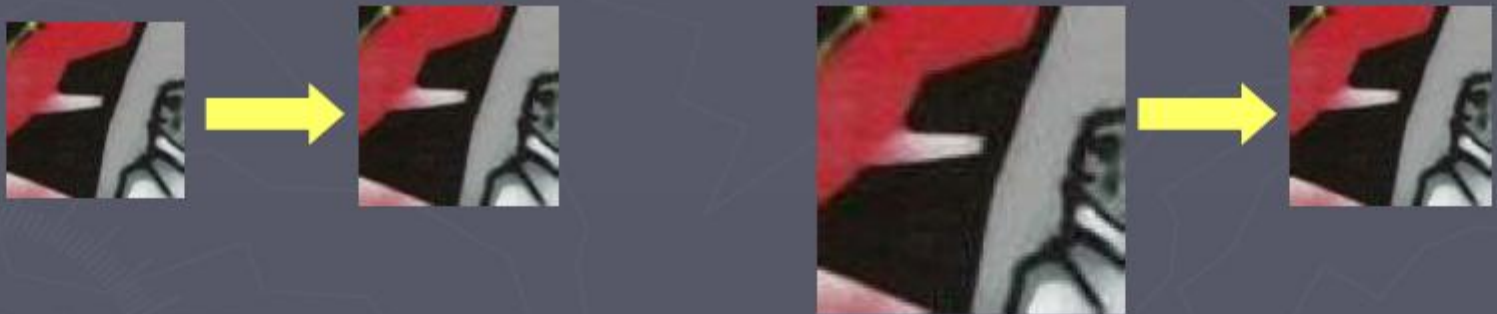
$$f(I_{i_1...i_m}(x, \sigma))$$



$$f(I_{i_1...i_m}(x', \sigma'))$$

Automatic scale selection

Normalize: rescale to fixed size



几种典型特征点检测算法回顾

- 典型特征点检测算法的特点
 - 仅给出特征点位置
 - 缺乏关于特征点性质的描述：尺度，方向，响应... ..?

5.4 SIFT算子

- SIFT: **S**cale **I**nvariant **F**eature **T**ransform
- David G. Lowe, **Object recognition from local scale-invariant features**, University of British Columbia, International Conference on Computer Vision(ICCV), 1999
- David G. Lowe, **Distinctive image features from scale-invariant keypoints**, University of British Columbia, International Journal of Computer Vision, 2004, 60(2): 91-110

5.4 SIFT算子——原理

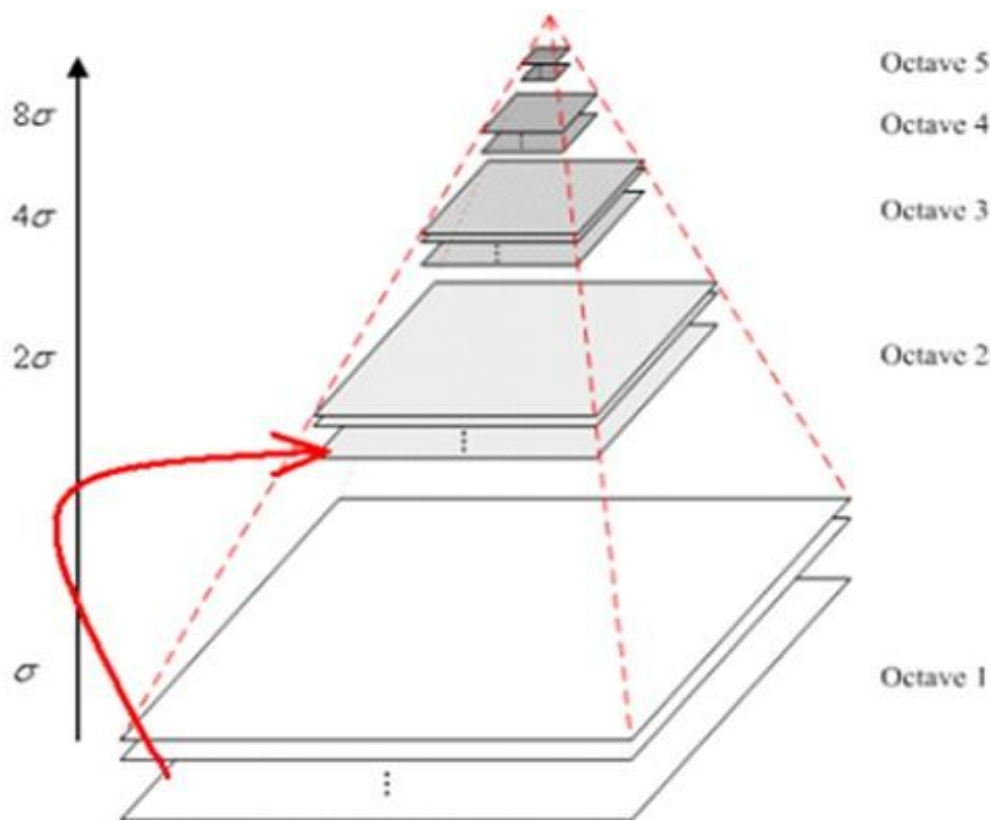
- 融特征点检测与描述于一体
- 基本步骤：
 - 1、尺度空间极值检测
 - 2、关键点定位
 - 3、获取关键点主方向
 - 4、关键点描述

5.4 SIFT算子——原理

- 1、尺度空间极值检测
 - 高斯金字塔构建
 - 高斯差分金字塔（ Difference of Gaussian , DoG）计算
 - 空间极值点检测
- 2、关键点定位
- 3、获取关键点主方向
- 4、关键点描述

5.4 SIFT算子——原理

- 尺度空间极值检测
 - 高斯金字塔构建



$$L(x, y, \sigma) = G(x, y, \sigma) \otimes I(x, y)$$

滤波后图像

Octave 5

Octave 4

Octave 3

Octave 2

Octave 1

原始图像

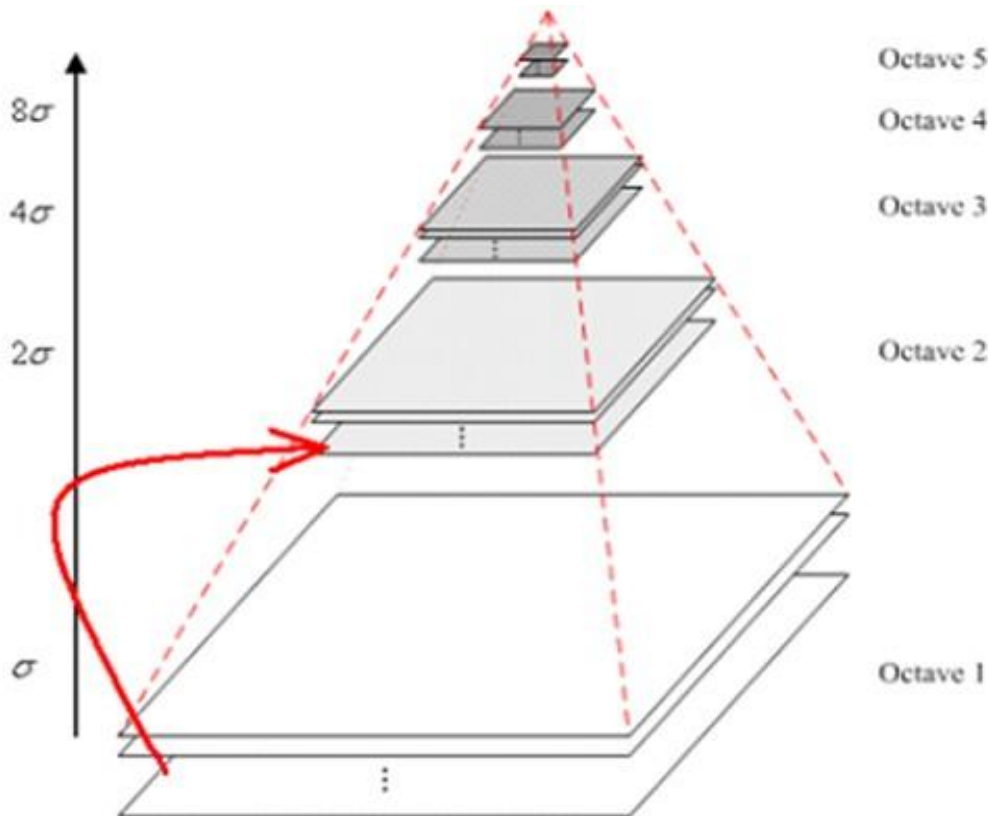
高斯滤波器

$$G(x, y, \sigma) = \frac{1}{2\pi\sigma^2} e^{-\frac{x^2+y^2}{2\sigma^2}}$$

$$F(\omega_x, \omega_y, \sigma) = e^{-\frac{(\omega_x^2 + \omega_y^2)\sigma^2}{2}}$$

5.4 SIFT算子——原理

- 尺度空间极值检测
 - 高斯金字塔构建



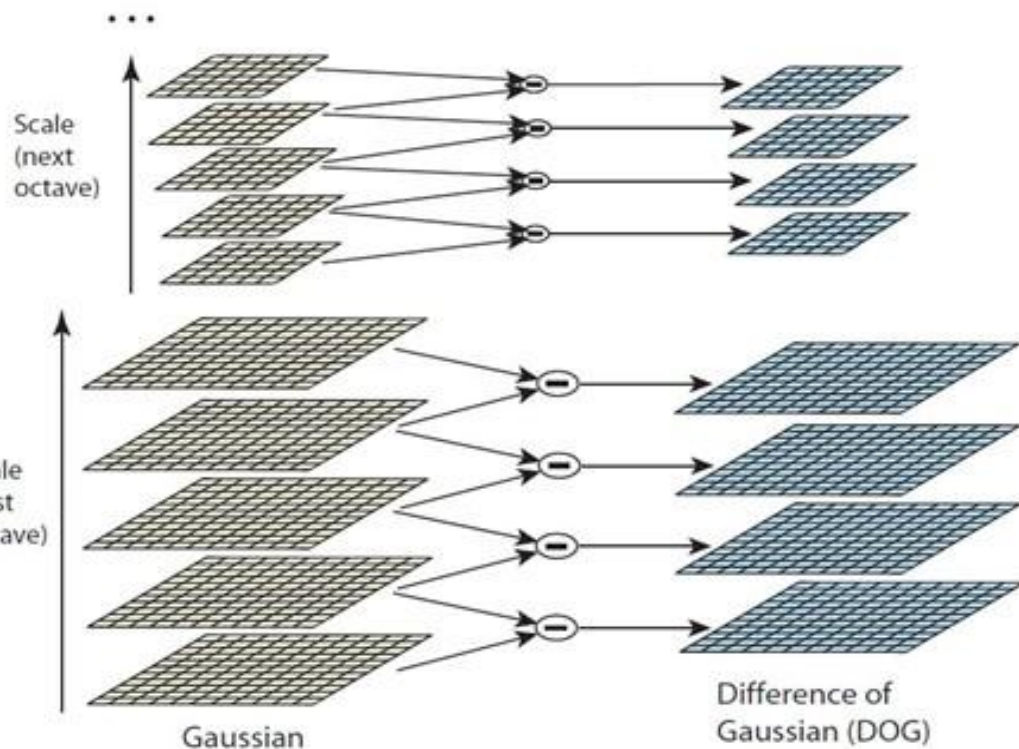
每一级octave分为 S 个尺度间隔，相邻两个尺度的 σ 之比为：

$$k = 2^{1/S}$$

则每一级包含 $1, 2^{1/S}, 2^{2/S}, \dots, 2$ ，共 $(S+1)$ 幅图像

5.4 SIFT算子——原理

- 尺度空间极值检测
 - DoG金字塔计算

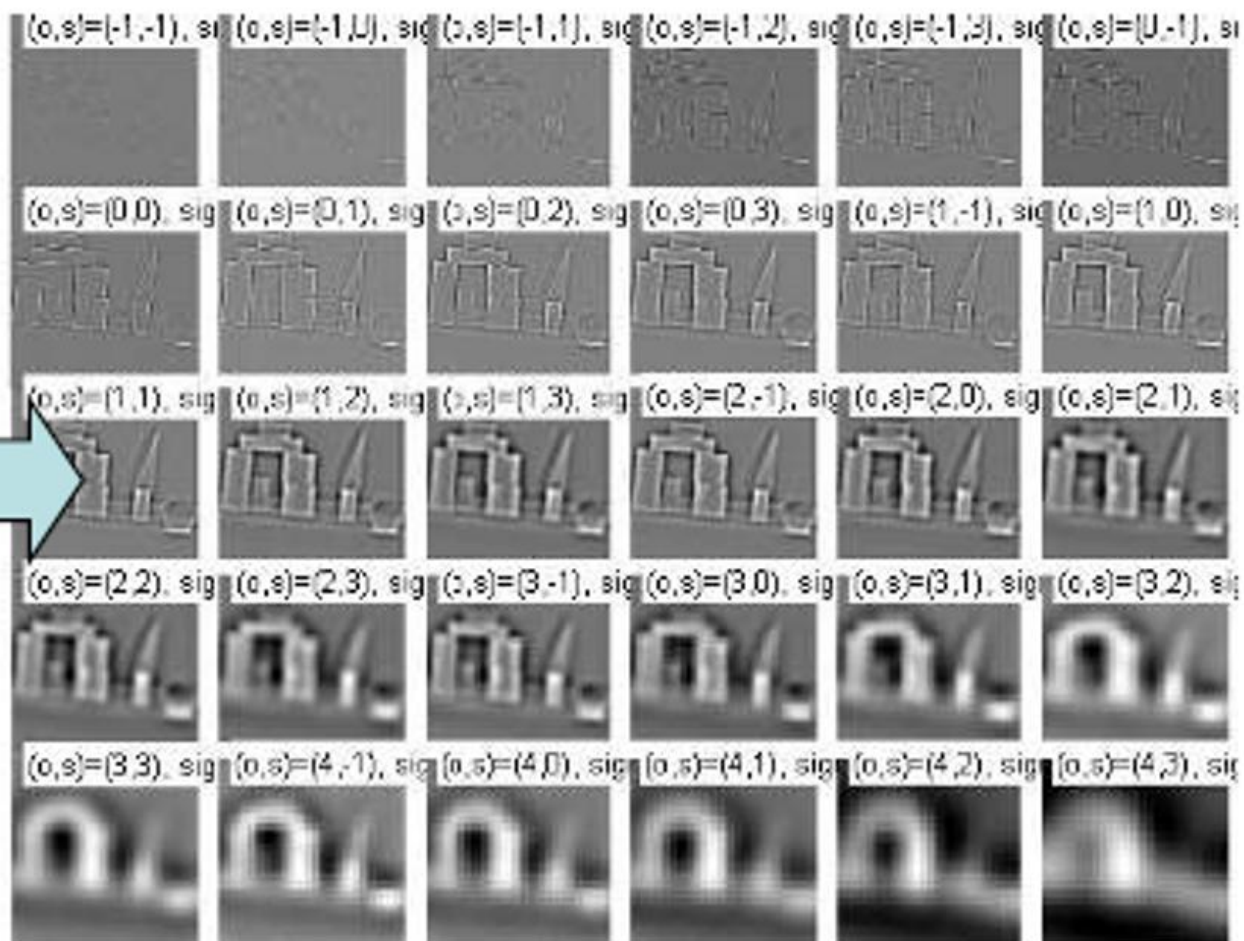
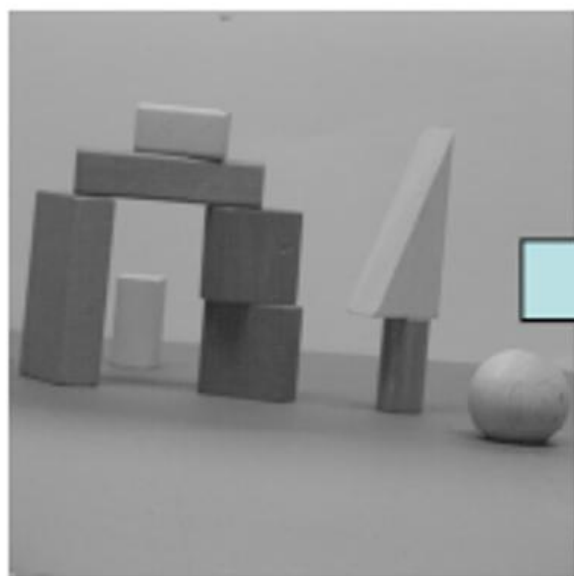


$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

选择DoG金字塔的原因：

- 1、计算简单
- 2、提供了尺度归一化Laplacian of Gaussian (LoG) 算子的良好近似（参考Lindeberg (1994)）。而LoG算子的极值，相对于梯度、Hessian、Harris等算子，能提供更稳定的图像特征（参考Mikolajczyk (2002)）

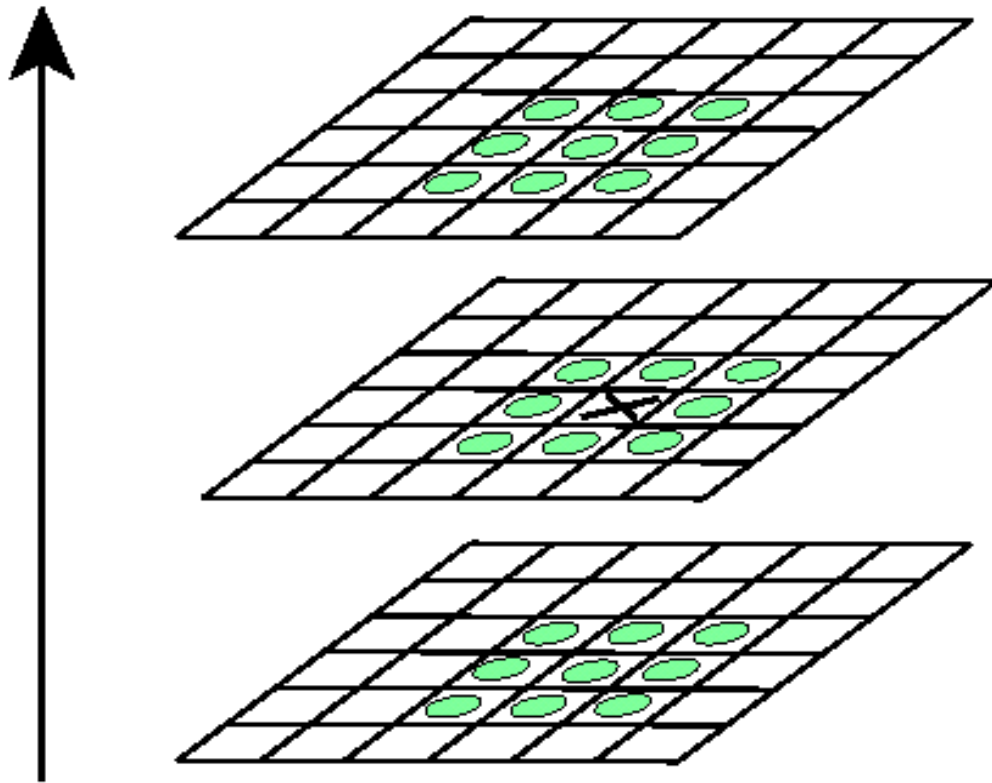
5.4 SIFT算子——原理



DoG算子结果

5.4 SIFT算子——原理

- 尺度空间极值检测
 - 空间极值点检测



对DoG金字塔中的每一像素，在三维邻域（两维图像、一维尺度）内判断是否极值点

可以确定极值点的位置及尺度

极值的大小反映了极值处的对比度（反差）

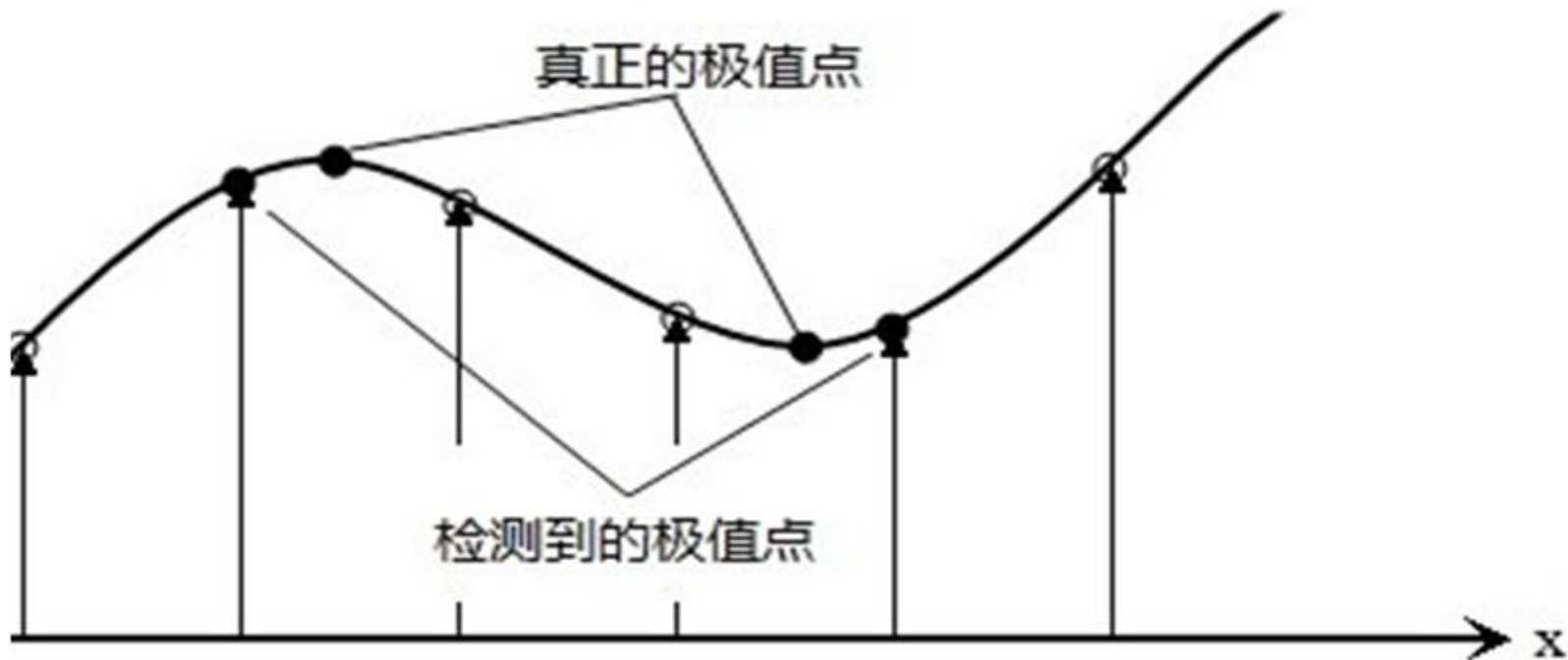
为得到每一级的 s 个极值结果，高斯金字塔的每一级需有 $(s + 3)$ 幅图像

5.4 SIFT算子——原理

- 1、尺度空间极值检测
- 2、关键点定位
 - 亚像素（子像素）精确定位
 - 去除低对比度极值点
 - 去除边缘点
- 3、获取关键点主方向
- 4、关键点描述

5.4 SIFT算子——原理

- 关键点定位
 - 亚像素（子像素）精确定位



5.4 SIFT算子——原理

- 关键点定位

- 亚像素（子像素）精确定位

将原点移到极值点处，并将高斯差分在该极值点处展开，保留一阶项及二阶项，得到：

$$D(\mathbf{x}) = D + \frac{\partial D}{\partial \mathbf{x}} \mathbf{x} + \frac{1}{2} \mathbf{x}^T \frac{\partial^2 D}{\partial \mathbf{x}^2} \mathbf{x}$$

其中 $\mathbf{x} = (x, y, \sigma)^T$

令 $D'(\mathbf{x}) = 0$ ，得：

$$\hat{\mathbf{x}} = -\frac{\partial^2 D}{\partial \mathbf{x}^2}^{-1} \frac{\partial D}{\partial \mathbf{x}}$$

尺度

5.4 SIFT算子——原理

- 关键点定位
 - 去除低对比度极值点

利用 $\hat{\mathbf{x}}$ 处的高斯差分值得 $D(\hat{\mathbf{x}})$ ，去除对比度较低的不稳定极值点

$$D(\hat{\mathbf{x}}) = D + \frac{1}{2} \frac{\partial D^T}{\partial \mathbf{x}} \hat{\mathbf{x}}$$

设对比度阈值为 T ，则当 $|D(\hat{\mathbf{x}})| < T$ 时，去除极值点 $\hat{\mathbf{x}}$

5.4 SIFT算子——原理

- 关键点定位
 - 去除边缘点

由于DoG算子对边缘点也很敏感，在边缘处也有很大的响应值。利用Hessian矩阵来处理边缘点。

在极值点处，利用 $D(\mathbf{x})$ 计算Hessian矩阵

$$H = \begin{bmatrix} D_{xx} & D_{xy} \\ D_{yx} & D_{yy} \end{bmatrix}$$

$D(\mathbf{x})$ 的主曲率，在边缘梯度的方向上比较大，而在沿着边缘的方向上则较小，且与 H 的特征值成正比。

5.4 SIFT算子——原理

- 关键点定位
 - 去除边缘点

利用矩阵 H 两个特征值的和等于 H 的迹，两个特征值的积等于 H 的行列式，有

$$\frac{\text{Tr}^2(H)}{\det(H)} = \frac{(r+1)^2}{r}$$

其中 r 为 H 的大特征值与小特征值之比

设关于 r 的阈值为 T_r ，则当 $\frac{\text{Tr}^2(H)}{\det(H)} > \frac{(T_r + 1)^2}{T_r}$ 时，判断该极值点为边缘点，并去除

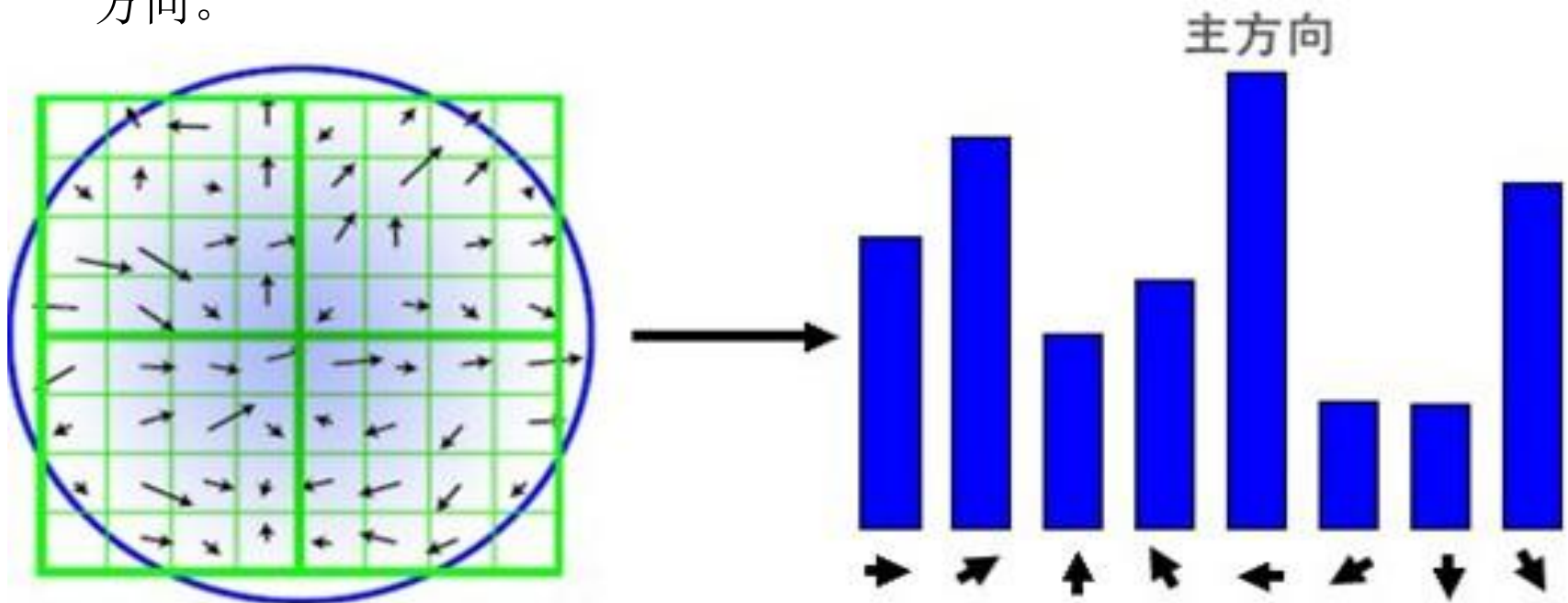
5.4 SIFT算子——原理

- 1、尺度空间极值检测
- 2、关键点定位
- 3、获取关键点主方向
- 4、关键点描述

5.4 SIFT算子——原理

- 获取关键点主方向

根据关键点的尺度，在高斯金字塔对应图像的关键点位置处的局部区域计算梯度方向直方图。直方图峰值对应得方向即为关键点的主方向。



基于梯度方向的线性插值：避免梯度方向直方图在梯度方向量化的边界处的突然变化

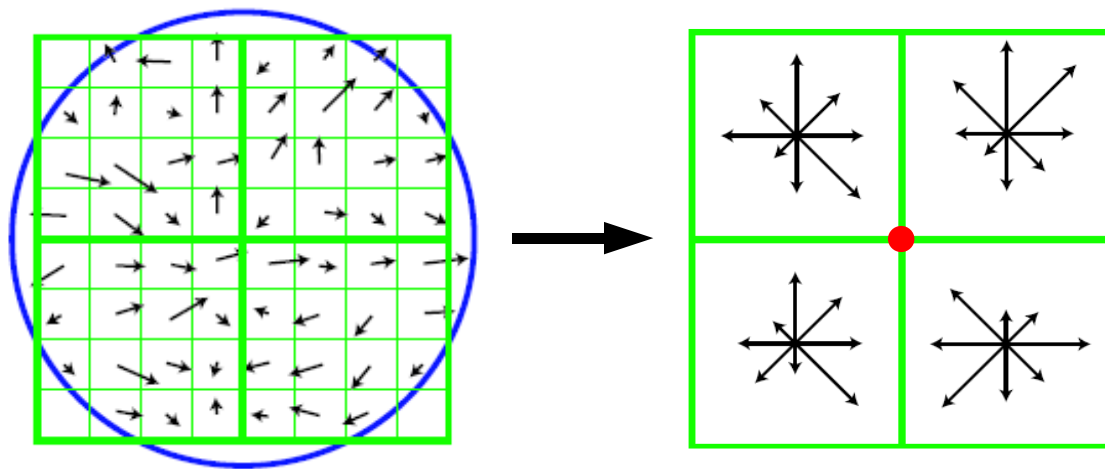
5.4 SIFT算子——原理

- 1、尺度空间极值检测
- 2、关键点定位
- 3、获取关键点主方向
- 4、关键点描述
 - 确定用于计算关键点描述的局部区域
 - 根据关键点主方向变换坐标系
 - 计算关键点描述
 - 归一化关键点描述

5.4 SIFT算子——原理

- 关键点描述

- 目的：利用关键点及其周围像素，计算一个特征向量，用于描述每个关键点
- 思路：通过对关键点周围图像区域分块，计算块内梯度直方图，生成特征向量
- 示例：



关键点周围区域图像梯度

关键点描述子

5.4 SIFT算子——原理

- 关键点描述

- 确定用于计算关键点描述的局部区域

描述子梯度方向直方图由关键点所在尺度的模糊图像计算产生，局部区域的半径通过下式计算：

$$\text{radius} = \frac{3\sigma_{\text{oct}} \times \sqrt{2} \times (d+1) + 1}{2} \quad ?$$

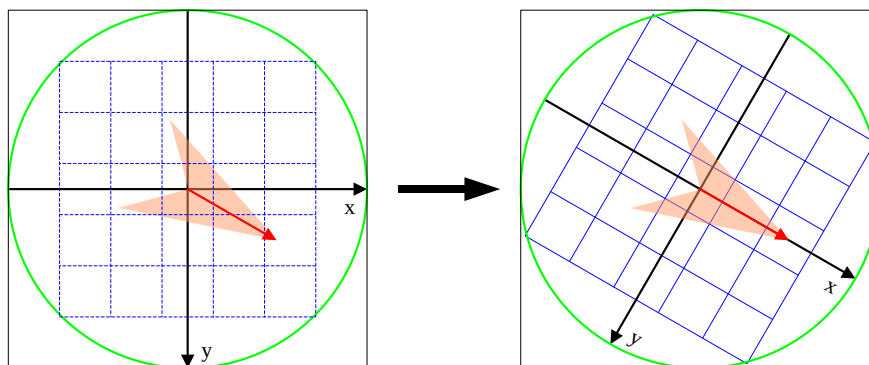
其中 σ_{oct} 为关键点所在组（octave）的组内尺度, $d = 4$

$$\text{diameter} = 6\sigma + 1 \quad \checkmark$$

其中 σ 为关键点所在尺度

5.4 SIFT算子——原理

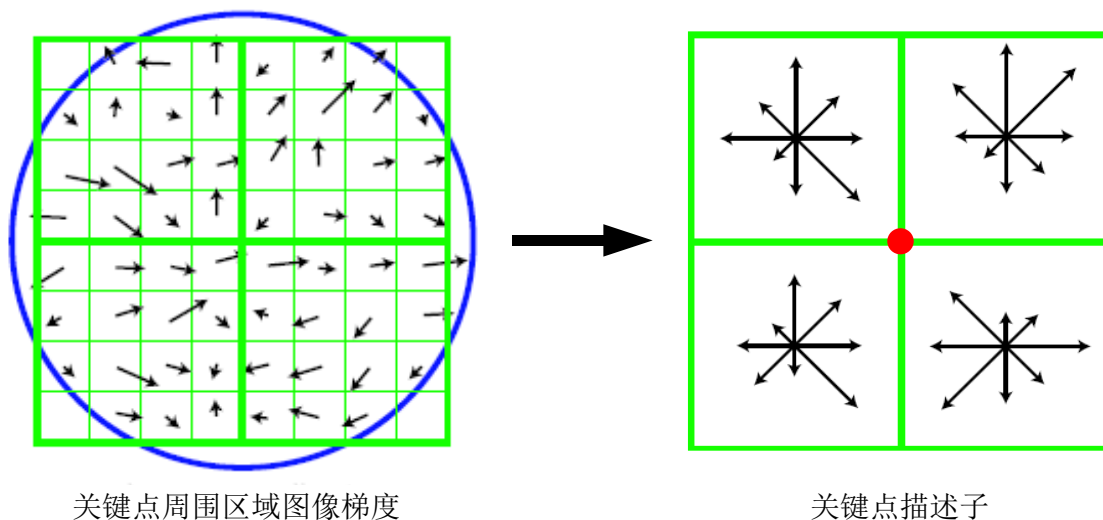
- 关键点描述
 - 根据关键点主方向变换坐标系



5.4 SIFT算子——原理

- 关键点描述
 - 计算关键点描述

在局部区域内对每个像素点求其梯度幅值和方向，生成方向直方图



梯度幅值高斯（ σ 为局部区域大小的一半）加权：离关键点越近/越远，其对关键点描述的作用越大/越小

基于像素位置和梯度方向的三线性插值：避免梯度方向直方图在分块边界和梯度方向量化的边界处的突然变化

5.4 SIFT算子——原理

- 关键点描述

- 归一化关键点描述

描述子向量归一化：克服光照影响，主要是增益的影响

方向直方图每个方向上梯度幅值限制在一定门限值以下（门限一般取0.2，像素灰度值在0~1范围内）：克服相机饱和、光照对3D表面不同方位影响不同等非线性光照情形

5.5 SURF算子

- SURF: Speeded Up Robust Features
- Herbert Bay¹, Tinne Tuytelaars², and Luc Van Gool^{1,2}, SURF: Speeded Up Robust Features, ¹ ETH Zurich, ²Katholieke Universiteit Leuven, European Conference on Computer Vision (ECCV), 2006
- Herbert Bay¹, Andreas Ess¹, Tinne Tuytelaars ², Luc Van Gool^{1,2}, Speeded-Up Robust Features (SURF), ¹ETH Zurich, ²K.U. Leuven, Computer Vision and Image Understanding, 2008, 110: 346–359
- SIFT算子的改进

5.5 SURF算子—— SIFT算子回顾

- 融特征点检测与描述于一体

- 基本步骤:

- 1、尺度空间极值检测:

高斯金字塔

高斯差分金字塔

- 2、关键点定位:

空间极值点

亚像素

低对比度极值点

边缘点

- 3、获取关键点主方向

- 4、关键点描述

5.5 SURF算子——原理

- 融特征点检测与描述于一体
- 基本步骤：
 - 1、特征点检测
 - 2、特征点描述

5.5 SURF算子——原理

- 特征点检测

- 基于Hessian矩阵的特征点检测
- 基于盒子（Box）滤波器的Hessian矩阵近似计算
- 基于积分图的盒子滤波器快速计算
- 尺度金字塔构建
- 特征点定位

- 特征点描述

- 快速索引匹配

5.5 SURF算子——原理

- 特点：
 - 具有尺度和旋转不变性特点的特征点提取和描述方法
 - 可重复性更好
 - 鲁棒性更强
 - 计算更快

5.5 SURF算子——原理

- 基于Hessian矩阵的特征点检测
 - 给定输入图像 I 中一点 $\mathbf{x}=(x, y)$ ，尺度为 σ 的

Hessian矩阵在该点的值 $H(\mathbf{x}, \sigma)$ 表示如下：

$$H(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$

其中， $L_{xx}(\mathbf{x}, \sigma)$ 为高斯二阶偏导数 $\frac{\partial^2}{\partial x^2} g(\sigma)$ 与

输入图像 I 在点 $\mathbf{x}$ 处的卷积，其余类似

- 行列式的值即为点 $\mathbf{x}$ 处的响应

5.5 SURF算子——原理

- 基于盒子（box）滤波器的Hessian矩阵近似计算

- LoG算子回顾

- 拉普拉斯算子对噪声比较敏感 → 平滑

- 高斯加权平滑函数 $h(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$

$$g(x, y) = h(x, y) \otimes f(x, y)$$

$$\nabla^2 g = \nabla^2 [h(x, y) \otimes f(x, y)] = \nabla^2 h(x, y) \otimes f(x, y)$$

$$= \left(\frac{r^2 - \sigma^2}{\sigma^4}\right) \exp\left(-\frac{r^2}{2\sigma^2}\right) \otimes f(x, y)$$

$$r^2 = x^2 + y^2$$

5.5 SURF算子——原理

- 基于盒子（box）滤波器的Hessian矩阵近似计算

- LoG算子回顾

$$h(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

$$\nabla^2 h(r) = \left(\frac{r^2 - \sigma^2}{\sigma^4}\right) \exp\left(-\frac{r^2}{2\sigma^2}\right)$$

$$r^2 = x^2 + y^2$$

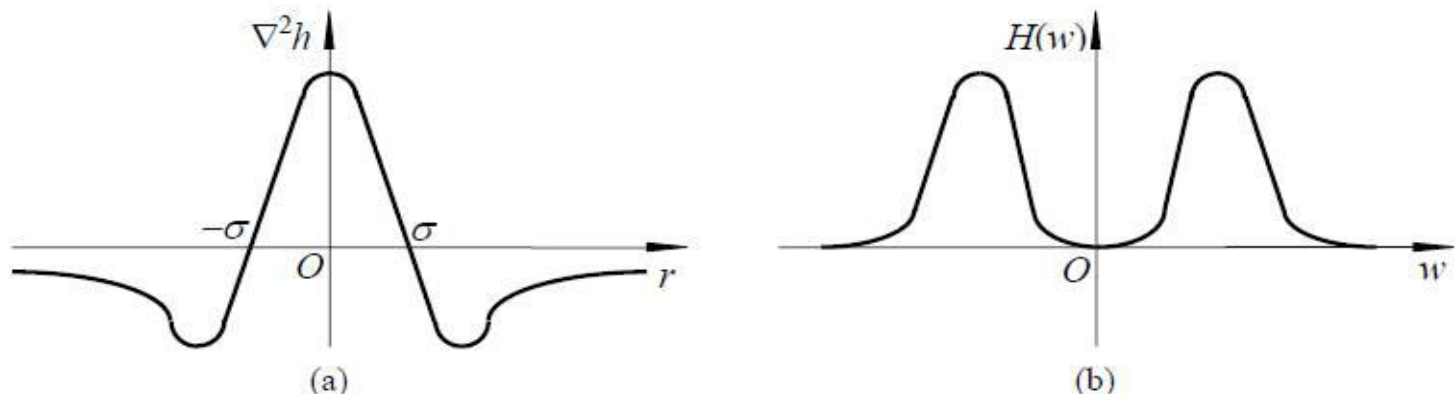


图 5.1.6 $\nabla^2 h$ 的剖面 and 对应的转移函数

5.5 SURF算子——原理

- 基于盒子（box）滤波器的Hessian矩阵近似计算

- LoG算子回顾

$$h(x, y) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

$$\nabla^2 h(r) = \left(\frac{r^2 - \sigma^2}{\sigma^4}\right) \exp\left(-\frac{r^2}{2\sigma^2}\right)$$

$$r^2 = x^2 + y^2$$

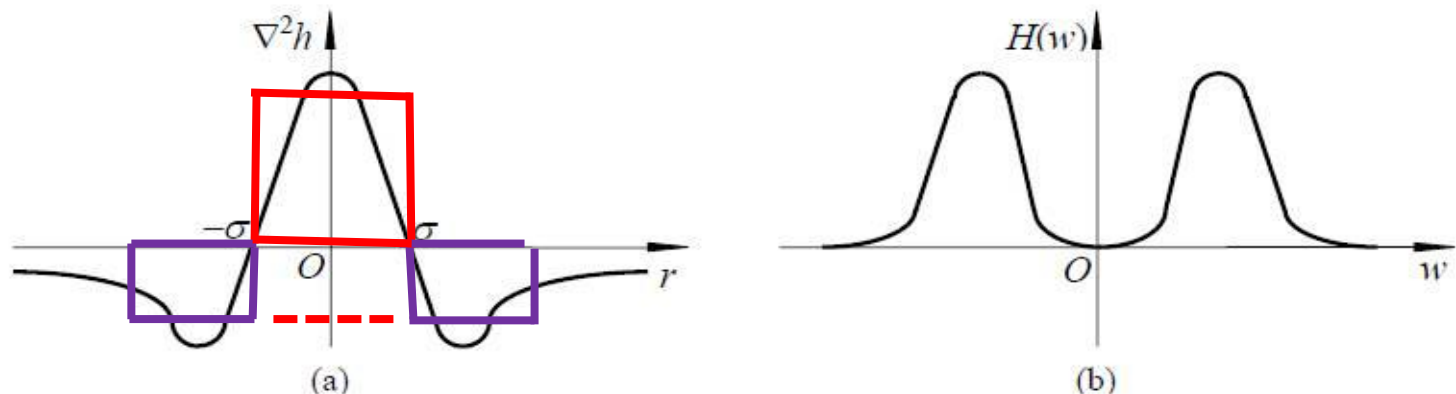


图 5.1.6 $\nabla^2 h$ 的剖面 and 对应的转移函数

5.5 SURF算子——原理

- 基于盒子滤波器的Hessian矩阵近似计算

$$g(\sigma) = \exp\left(-\frac{x^2 + y^2}{2\sigma^2}\right)$$

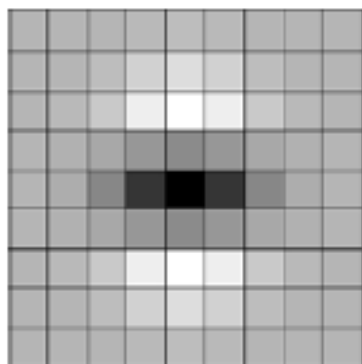
$$g_{xx}(\sigma) = \exp\left(-\frac{y^2}{2\sigma^2}\right) \bullet \left(\frac{x^2 - \sigma^2}{\sigma^4}\right) \exp\left(-\frac{x^2}{2\sigma^2}\right)$$

$$g_{xy}(\sigma) = \frac{x}{\sigma^2} \exp\left(-\frac{x^2}{2\sigma^2}\right) \bullet \frac{y}{\sigma^2} \exp\left(-\frac{y^2}{2\sigma^2}\right)$$

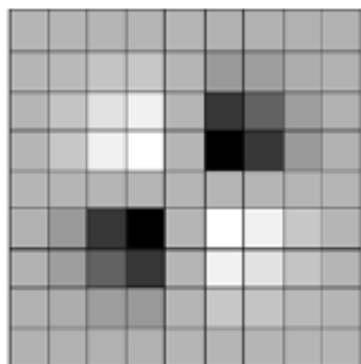
$$g_{yy}(\sigma) = \exp\left(-\frac{x^2}{2\sigma^2}\right) \bullet \left(\frac{y^2 - \sigma^2}{\sigma^4}\right) \exp\left(-\frac{y^2}{2\sigma^2}\right)$$

5.5 SURF算子——原理

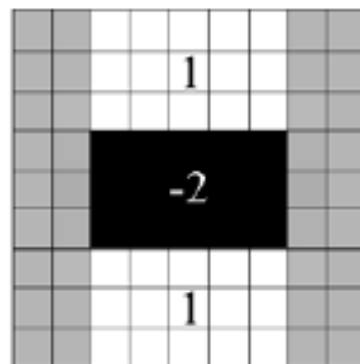
- 基于盒子滤波器的Hessian矩阵近似计算
 - 由于实际中高斯二阶导函数的离散化和截断造成的误差是不可避免的，近似为盒子（box）滤波以降低计算量



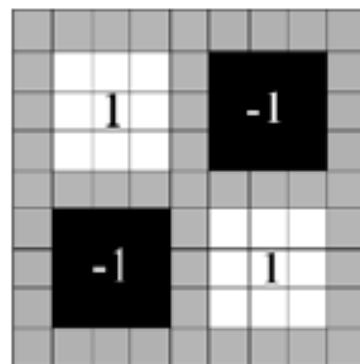
高斯二阶偏导数
 L_{yy}



高斯二阶偏导数
 L_{xy}



盒子滤波近似函数
 D_{yy}



盒子滤波近似函数
 D_{xy}

5.5 SURF算子——原理

- 基于盒子滤波器的Hessian矩阵近似计算

$$H(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$

➡ $H(\mathbf{x}, N) = \begin{bmatrix} D_{xx}(\mathbf{x}, N) & D_{xy}(\mathbf{x}, N) \\ D_{xy}(\mathbf{x}, N) & D_{yy}(\mathbf{x}, N) \end{bmatrix}$?

— 行列式的值即为点 $\mathbf{x}$ 处的响应

盒子滤波器的大小

5.5 SURF算子——原理

- 基于盒子滤波器的Hessian矩阵近似计算
 - 近似Hessian矩阵行列式可表示为：

$$\det(H_{approx}) = D_{xx}D_{yy} - (wD_{xy})^2$$

- 其中权重 w 为平衡因子

Frobenius
范数

$$w = \frac{|L_{xy}(1.2)|_F |D_{yy}(9)|_F}{|L_{yy}(1.2)|_F |D_{xy}(9)|_F} = 0.912... \approx 0.9$$

- 行列式的值即为点 $\mathbf{x}$ 处的响应

5.5 SURF算子——原理

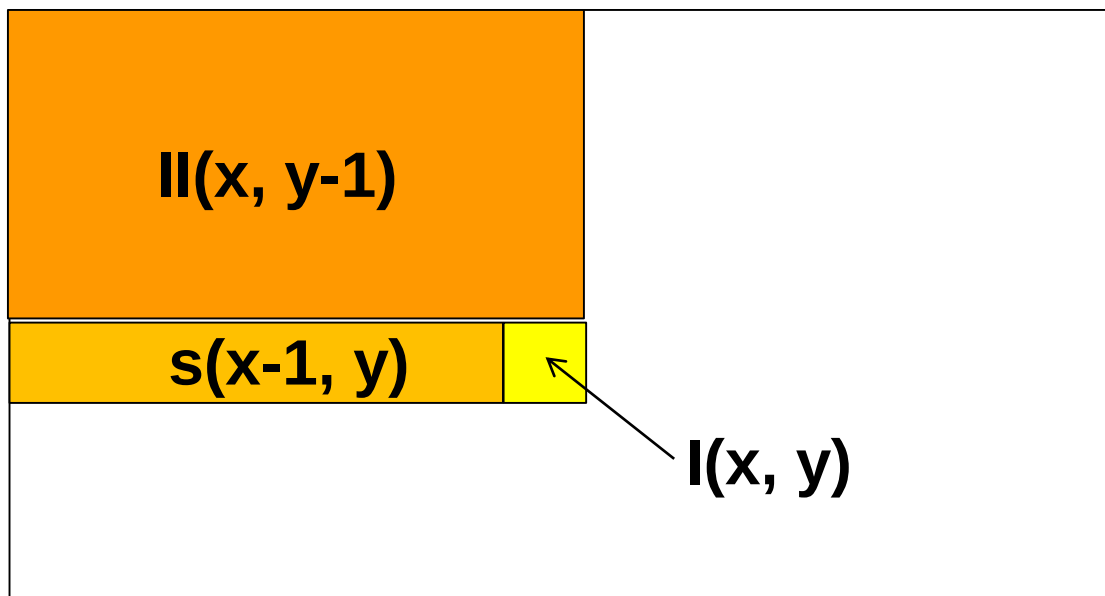
- 基于积分图的盒子滤波器快速计算
 - 积分图是输入图像经过像素间的累加运算得到的新的图像。
 - 积分图中的任意一点 (x, y) 的值是从输入图像的左上角到这个点所构成的矩形区域内所有的点的灰度值之和
 - 可以快速计算盒子卷积滤波

5.5 SURF算子——原理

- 基于积分图的盒子滤波器快速计算

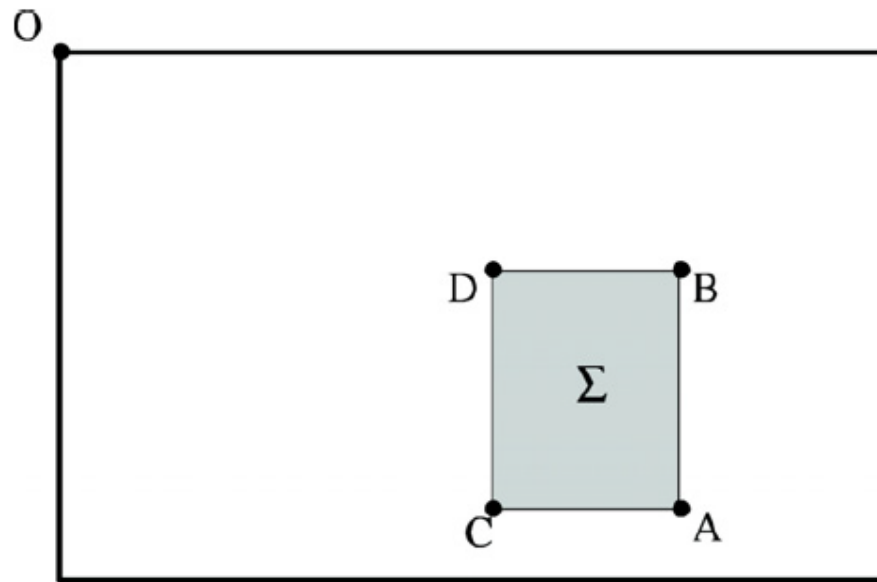
- 对积分图 $I_{\Sigma}(x)$ 中任一点 $\mathbf{x}=(x, y)^T$

$$\begin{aligned} II(x, y) &= \sum_{i=0}^{i \leq x} \sum_{j=0}^{j \leq y} I(i, j) = \sum_{i=0}^{i \leq x} \sum_{j=0}^{j \leq y-1} I(i, j) + \sum_{i=0}^{i \leq x-1} I(i, y) + I(x, y) \\ &= II(x, y-1) + S(x-1, y) + I(x, y) \end{aligned}$$



5.5 SURF算子——原理

- 基于积分图的盒子滤波器快速计算
 - 计算矩阵ABCD中像素和只需要积分图中A、B、C、D四点的值和3次加法运算



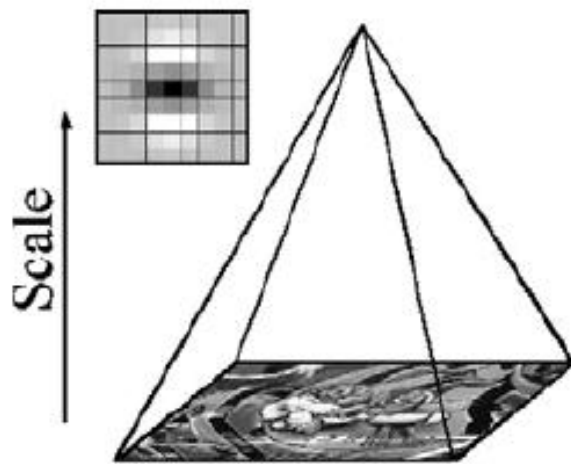
$$\Sigma = A - B - C + D$$

5.5 SURF算子——原理

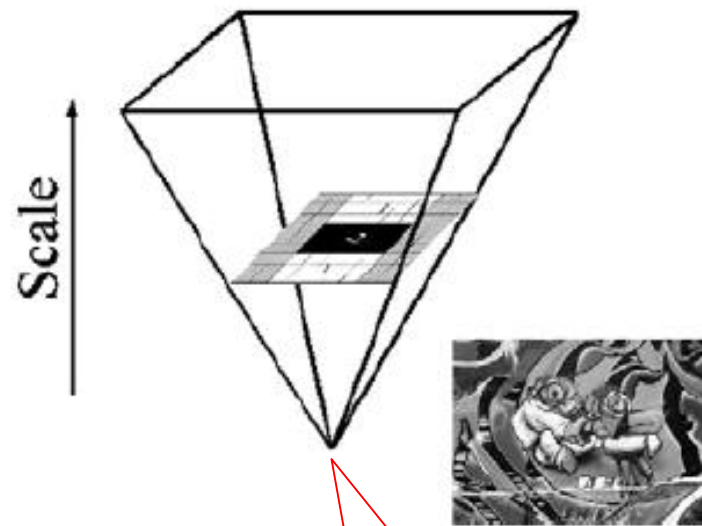
- 尺度金字塔构建
 - 一般尺度金字塔为对图像进行迭代下采样，用同一个滤波器滤波
 - 由于积分图和盒子滤波的计算便利性，对滤波尺度进行迭代上采样，在同一个图像上滤波

5.5 SURF算子——原理

- 尺度金字塔构建



固定滤波器尺度
下采样图像



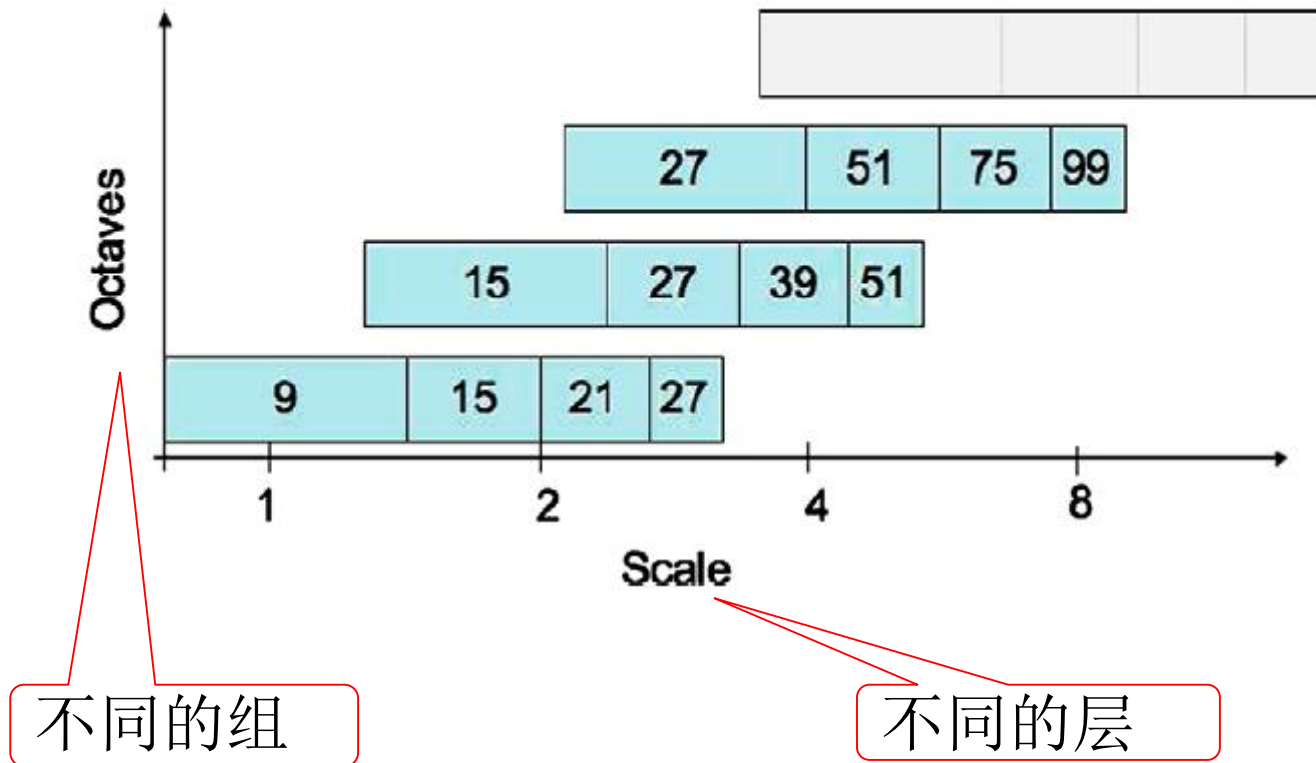
固定图像尺度
上采样滤波器

5.5 SURF算子——原理

- 尺度金字塔构建
 - 根据不同尺度，构建多组尺度金字塔
 - 每组尺度金字塔的各层尺度成等差数列
 - 不同组的尺度金字塔的公差等比增长
$$\text{Filter Size} = 3(2^{\text{Octave}} \times \text{Scale} + 1)$$
 - 为保持尺度空间的连续性，相邻组尺度金字塔有部分层重叠

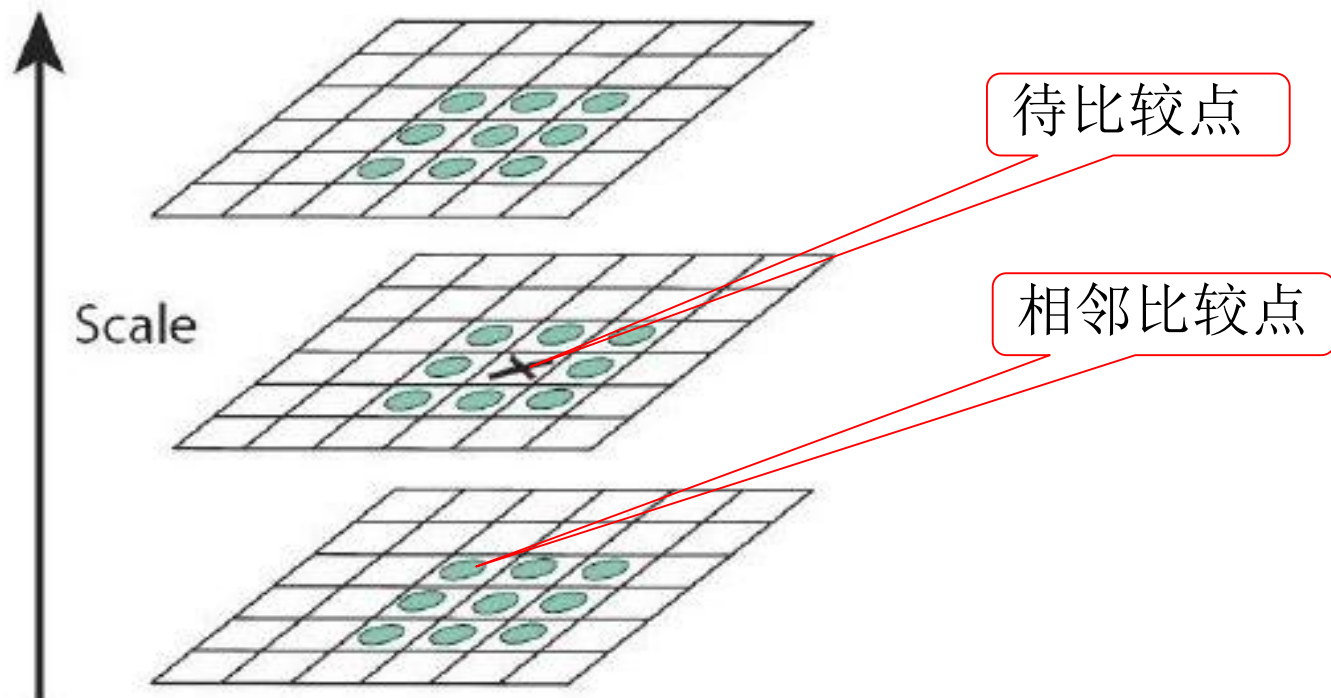
5.5 SURF算子——原理

- 尺度金字塔构建



5.5 SURF算子——原理

- 特征点定位
 - 非极大抑制



5.5 SURF算子——原理

- 特征点定位
 - 尺度空间插值
 - 得到亚像素级的特征点
 - 减小不同组第一层之间的差异

5.5 SURF算子——原理

- 特征点检测
- 特征点描述
 - 特征点主方向
 - Haar小波响应和描述符
- 快速索引匹配

5.5 SURF算子——原理

- 描述特征点邻域内各点的数值分布
- 利用x和y方向上的Haar小波响应提取信息
- 利用积分图加速
- 基于拉普拉斯算子符号的新索引步骤

5.5 SURF算子——原理

- 特征点主方向
 - 选择以特征点为中心、半径为 $6s$ （ s 为特征点所在的尺度值）的圆形邻域
 - 计算该圆形邻域内各点的 x 、 y 方向Haar小波响应（Haar小波边长取 $4s$ ）

5.5 SURF算子——原理

- 特征点主方向
 - Haar小波模板，白色权重为1，黑色为-1



X方向Haar小波滤波器



Y方向Haar小波滤波器

5.5 SURF算子——原理

- 特征点主方向
 - 用 $\sigma=2.5s$ 的高斯函数对响应加权
 - 以特征点为中心，张角为 $\pi/3$ 的扇形滑动窗口，计算窗口内的Harr小波响应值 dx 、 dy 的累加

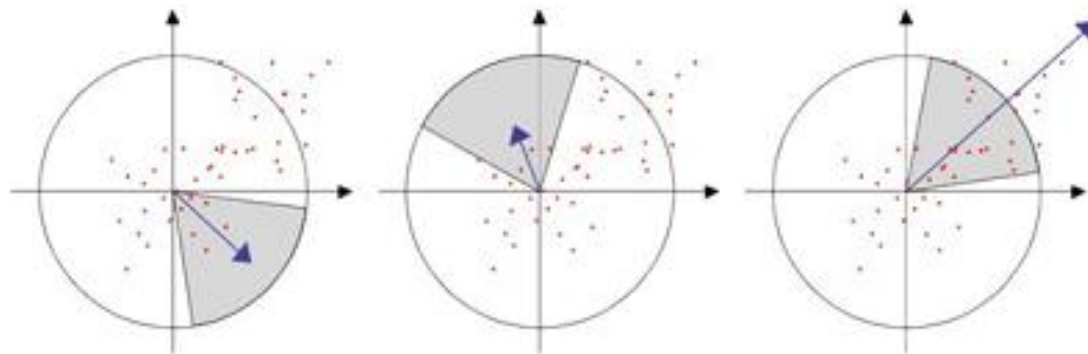
$$m_w = \sum_w dx + \sum_w dy$$

$$\theta_w = \arctan \left(\sum_w dx / \sum_w dy \right)$$

5.5 SURF算子——原理

- 特征点主方向
 - 特征点主方向为最大的Harr响应累加值所对应的方向

$$\theta = \theta_w \mid \max \{m_w\}$$



5.5 SURF算子——原理

- Haar小波响应和描述符
 - 构建以特征点为中心、方向为特征点主方向、边长为 $20s$ 的正方形区域

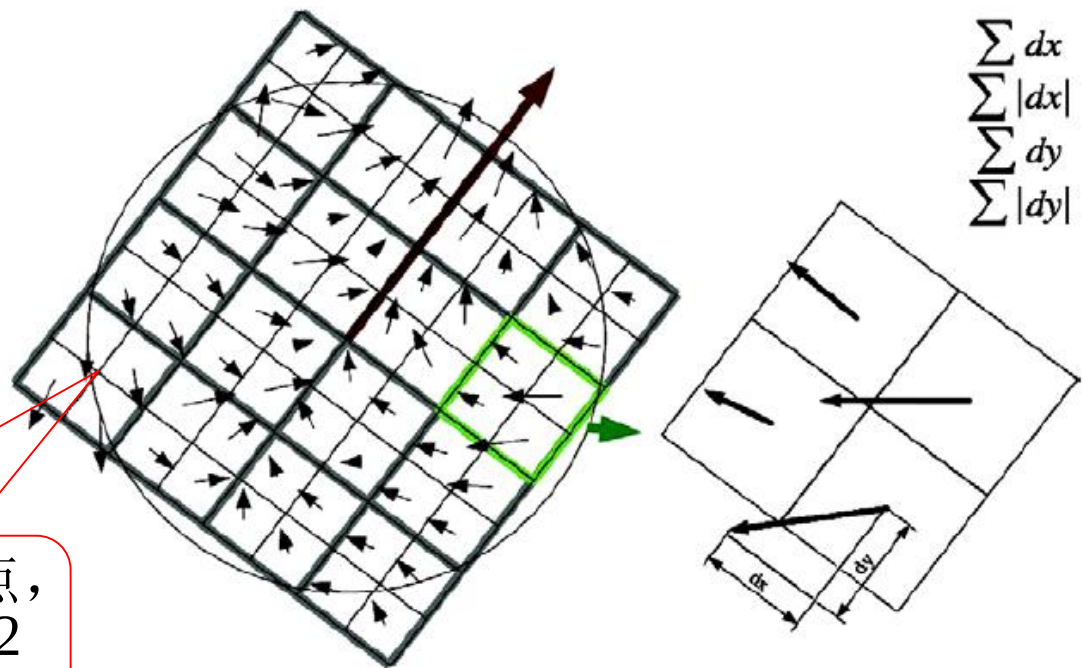


5.5 SURF算子——原理

- Haar小波响应和描述符
 - 将区域划分为 4×4 个子块
 - 每个子块内均匀采样 5×5 个样本点
 - 计算各采样点的Haar小波响应（Haar小波边长为 $2s$ ）
 - 用以特征点为中心点， $\sigma=3.3s$ 的高斯函数对响应加权

5.5 SURF算子——原理

- Haar小波响应和描述符
 - 在各子块内统计 $\sum dx$ 、 $\sum |dx|$ 、 $\sum dy$ 、 $\sum |dy|$ 形成子块特征向量



每个子块采样5x5个点，
这里为方便只展示2x2
个点

5.5 SURF算子——原理

- Haar小波响应和描述符

- 每个子块统计一个4维特征向量 v

$$v = (\sum dx, \sum dy, \sum |dx|, \sum |dy|)$$

- 将4x4个子块的特征向量 v 相连可得长度为64的特征向量，具有亮度不变性

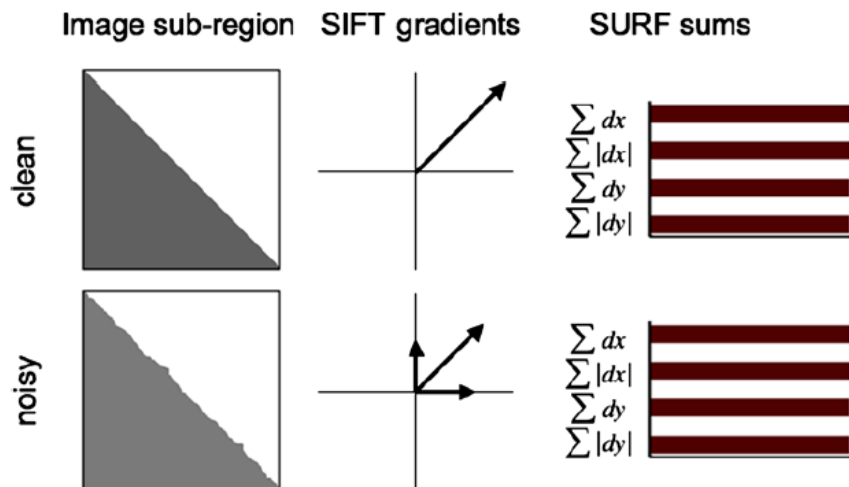
- 将特征向量归一化，具有对比度不变性

5.5 SURF算子——原理

- Haar小波响应和描述符



SURF描述子对不同纹理的区分性



SURF描述子较SIFT对噪声的鲁棒性

5.5 SURF算子——原理

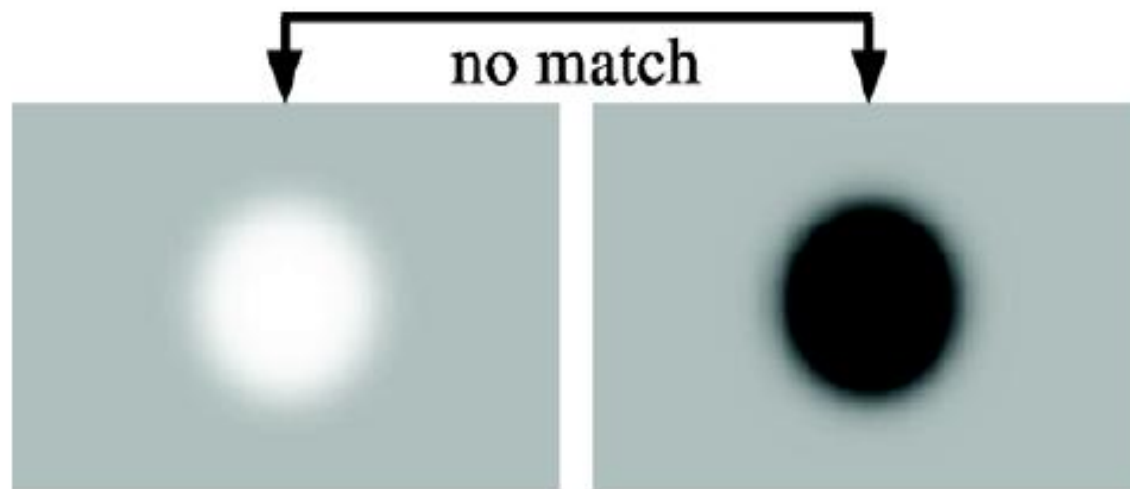
- 特征点检测
- 特征点描述
- 快速索引匹配

5.5 SURF算子——原理

- 快速索引匹配
 - 特征点一般表现为斑点（blob）结构
 - 根据拉普拉斯算子的符号（即Hessian矩阵的迹的符号）可区分亮斑点和暗斑点

5.5 SURF算子——原理

- 快速索引匹配
 - 在检测阶段，记录拉普拉斯算子的符号
 - 在匹配阶段中，匹配拉普拉斯算子符号相同的特征，从而降低运算量



5.5 SURF算子——实验

- 对比实验
- 3D应用
- 目标识别应用

5.5 SURF算子——实验

- 对比实验

- 检测器

- Fast-Hessian (FH, 盒子滤波器加积分图)
 - DoG
 - Harris-Laplace
 - Hessian-Laplace

5.5 SURF算子——实验

- 对比实验

- 检测器

- 计算效率

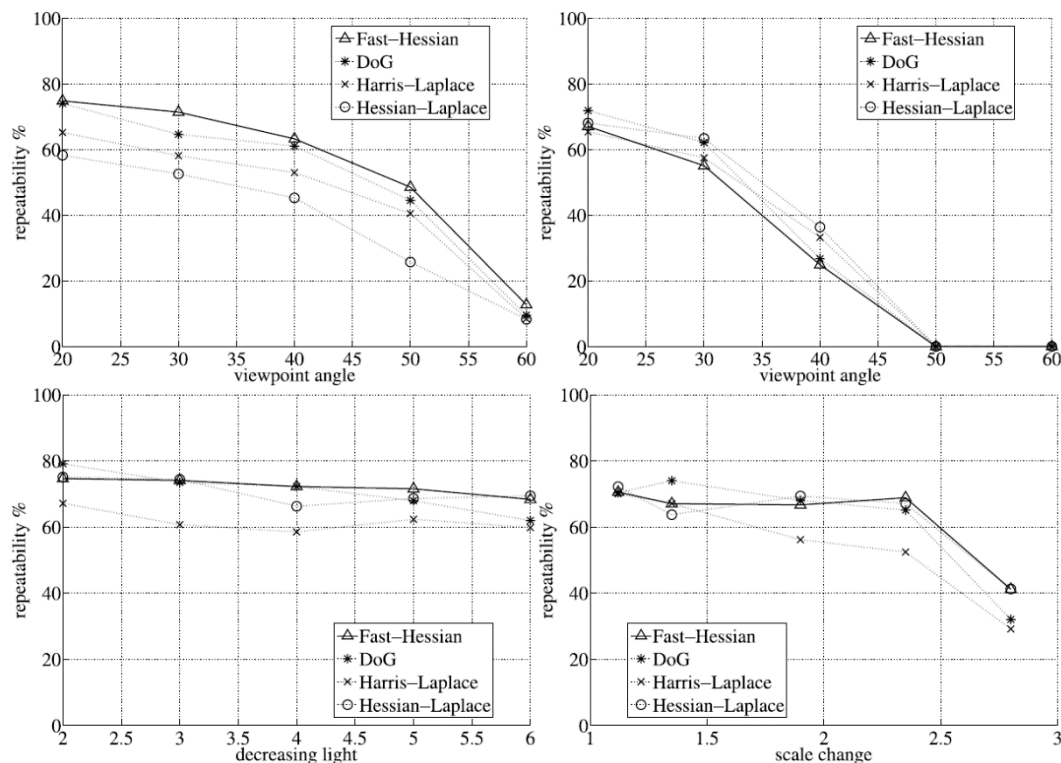
detector	threshold	nb of points	comp. time (msec)
Fast-Hessian	600	1418	120
Hessian-Laplace	1000	1979	650
Harris-Laplace	2500	1664	1800
DoG	default	1520	400

5.5 SURF算子——实验

- 对比实验

- 检测器

- 可重复性

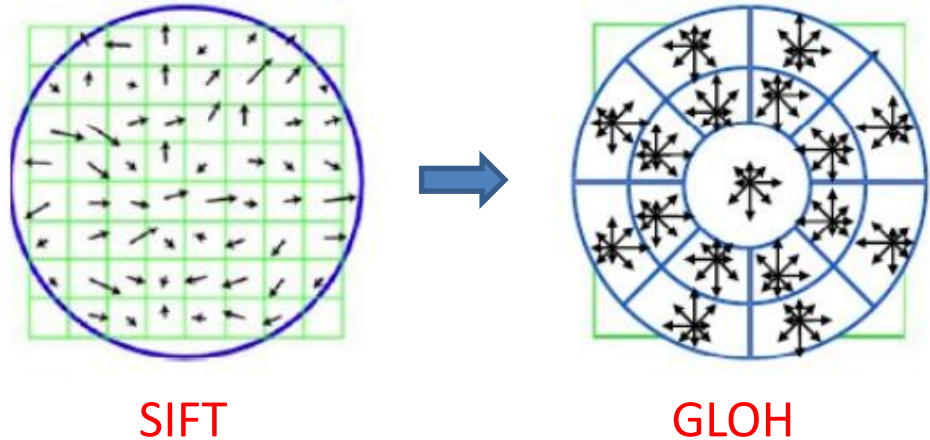


5.5 SURF算子——实验

- 对比实验

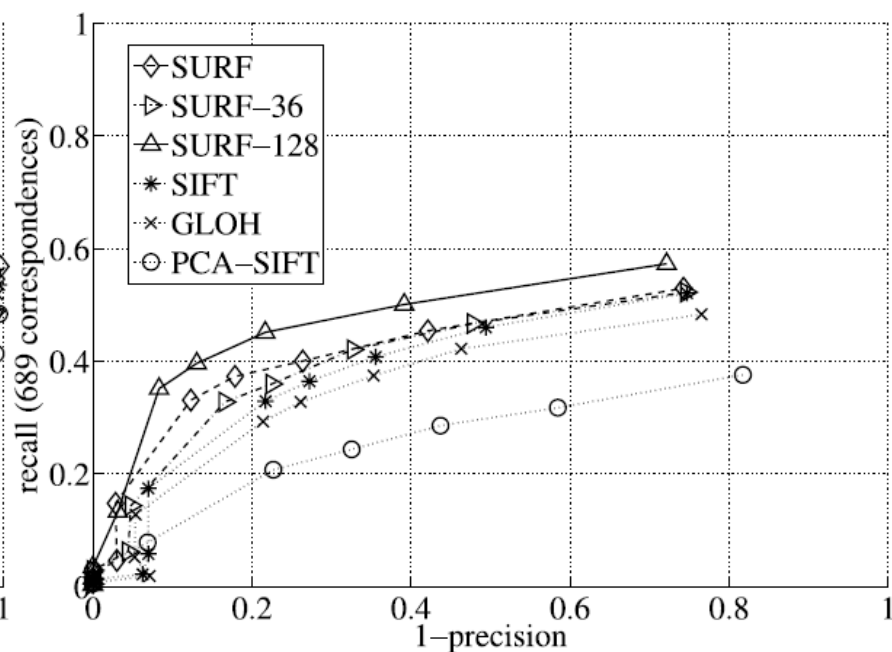
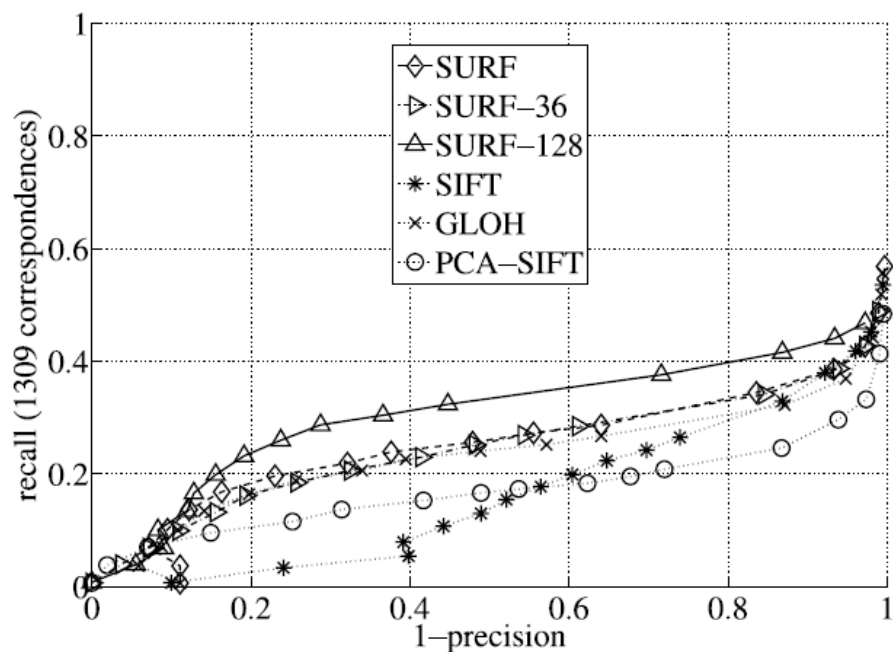
- 描述子

- SURF
 - SIFT
 - GLOH (Gradient location-orientation histogram)
 - PCA-SIFT



5.5 SURF算子——实验

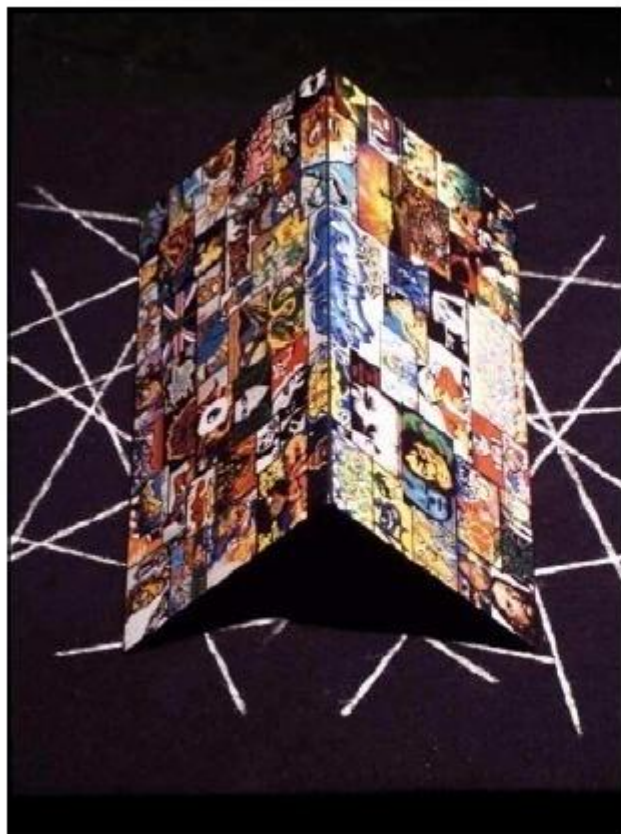
- 对比实验
 - SURF: 4x4x4
 - SURF-36: 3x3x4
 - 描述子
 - SURF-128: 4x4x8 (区分dx、dy大于0或小于0)



5.5 SURF算子——实验

- 3D应用

- 相机自校准和3D重建



5.5 SURF算子——实验

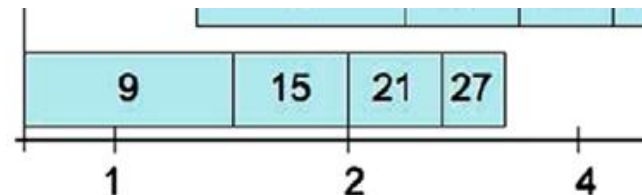
- 3D应用
 - 相机自校准和3D重建

Detector	Angle (°)	Mean dist (pixels)	SD (pixels)
FH-15	88.5	1.14	1.23
FH-9	88.4	1.64	1.78
DoG	88.9	1.95	2.14
Harris-Laplace	88.3	2.13	2.33
Hessian-Laplace	91.1	2.85	3.13

The true angle is 88.6°.

– FH-15为初始尺寸为15x15

– FH-9为初始尺寸为9x9

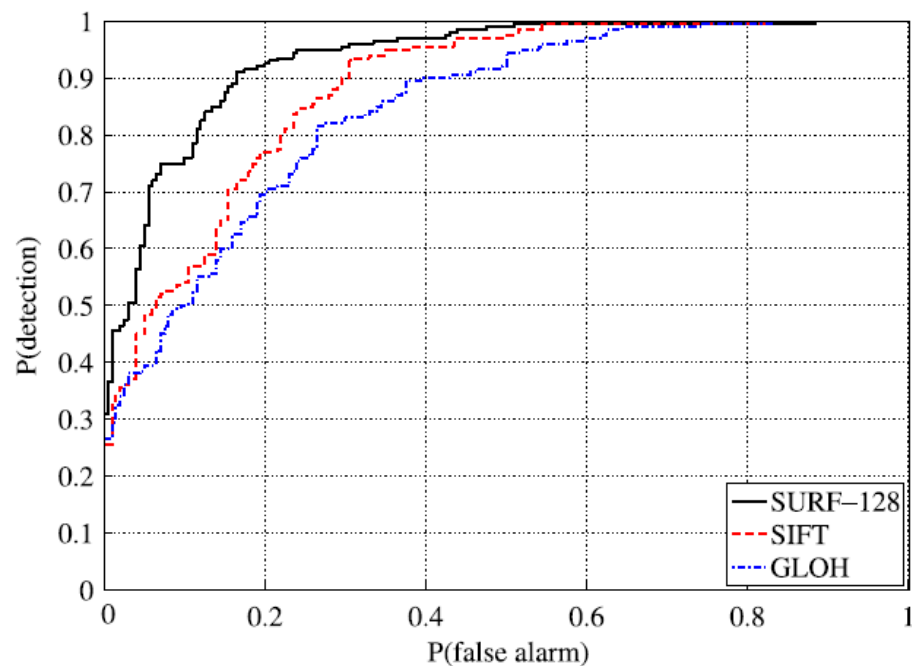
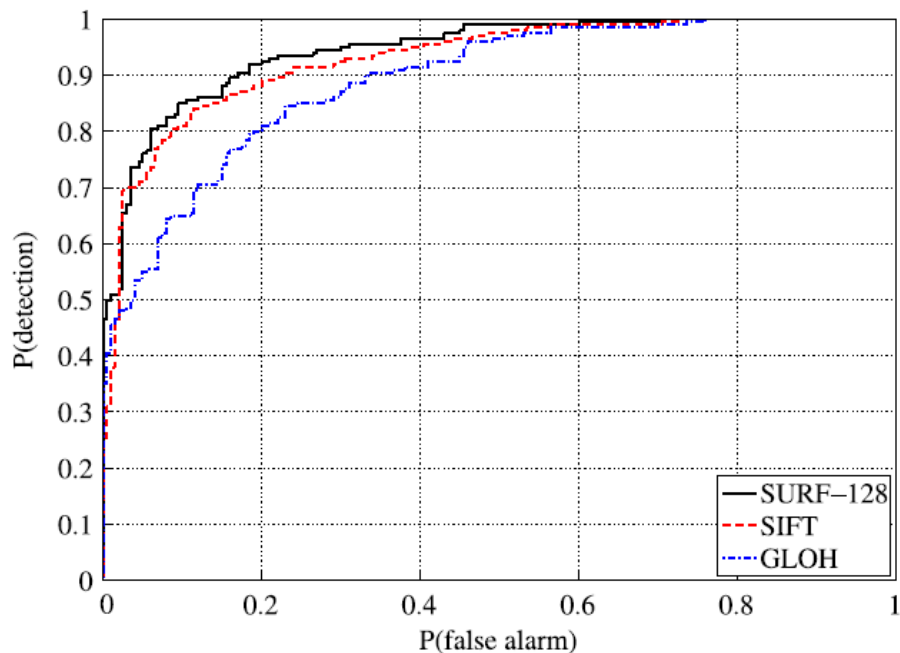


5.5 SURF算子——实验

- 目标识别应用
 - 使用Bayes分类器
 - 特征提取算子
 - SURF
 - SIFT
 - GLOH

5.5 SURF算子——实验

- 目标识别应用



作业

- SIFT算子在计算关键点描述时，使用了基于像素位置和梯度方向的三线性插值。请针对其中的梯度方向插值，解释其作用，并举例说明。假设梯度方向划分为0度、45度、90度、135度、180度、225度、270度、315度等8个方向。
- 请针对下图，给出积分图的每一步计算式及计算结果，并给出由积分图计算中间黑色阴影区域像素之和的计算式及结果。

3	2	4	1	7
2	8	7	0	1
4	2	8	4	2
5	6	5	5	1
2	3	7	6	9

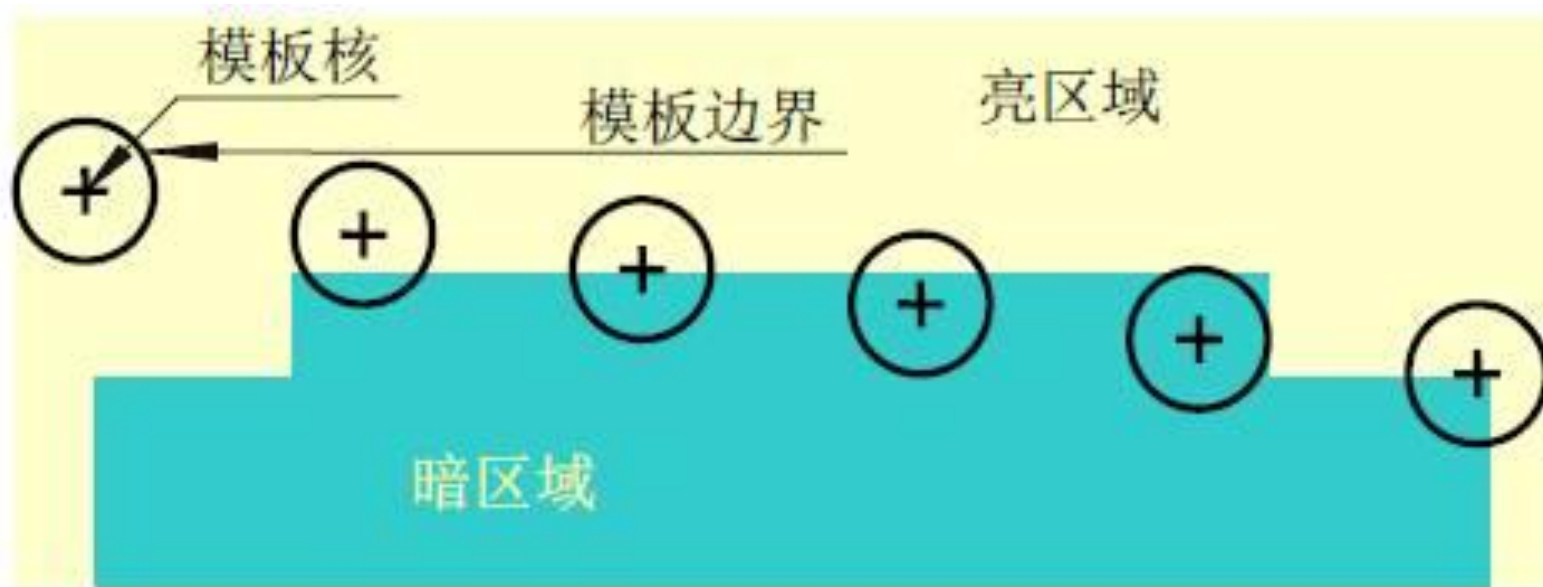
5.6 FAST算子

- FAST: **F**eatures from **A**ccelerated **S**egment **T**est
- E. Rosten, T. Drummond, **Fusing points and lines for high performance tracking**, IEEE International Conference on Computer Vision (ICCV), 2005.
- SUSAN算子的改进
- 仅涉及特征点检测，不涉及特征点描述

5.6 FAST算子

- E. Rosten, T. Drummond, Machine learning for high-speed corner detection, ECCV 2006.
- 基于机器学习的改进算法
- <http://www.edwardrosten.com/work/fast.html>

5.6 FAST算子—— SUSAN算子回顾



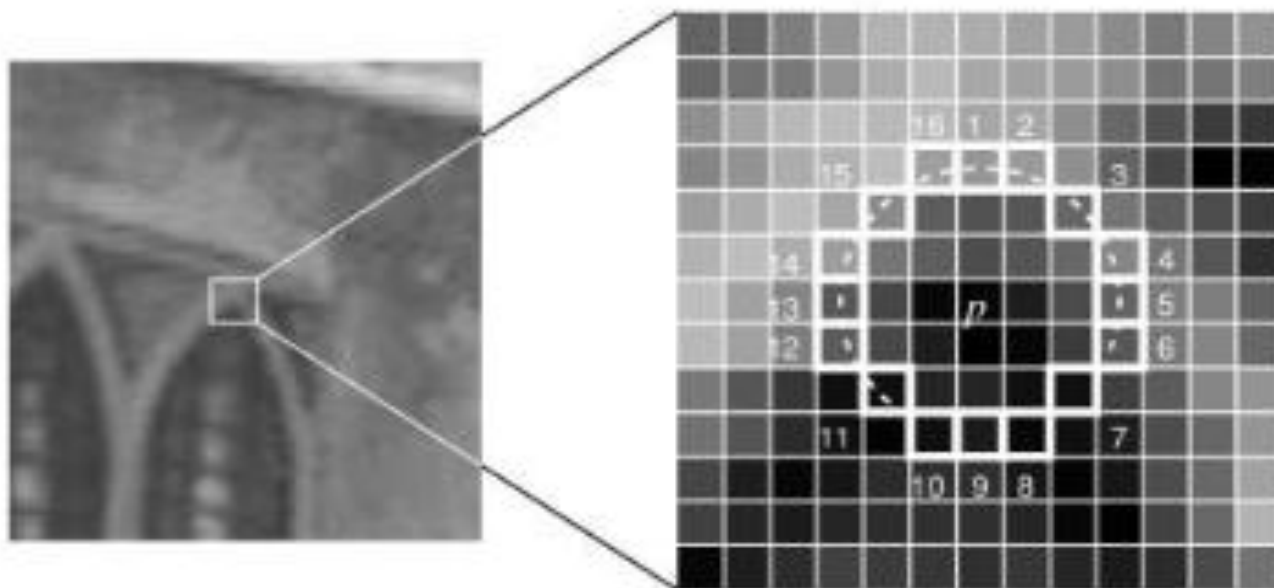
核同值的判断：灰度差阈值 T

角点响应：几何阈值 G 与核同值区大小之差

角点判断：角点响应大于1

5.6 FAST算子——原始算法

- 在像素 p 的Bresenham圆上选择 N 个像素。如果这 N 个像素中存在 n 个连续像素的灰度值都高于 $I_p + t$ ，或低于 $I_p - t$ ，则 p 是角点，其中 I_p 为 p 的灰度值， t 为阈值。



$N = 16$ 示意图（以后均以此为例）

5.6 FAST算子——原始算法

- 与SUSAN算子对比

SUSAN算子

T : 灰度差阈值
(不区分亮暗)

G : 几何阈值
(控制同值像素数)

考虑模板区域

无同值区域连通要求

FAST算子

t : 灰度差阈值
(区分亮暗)

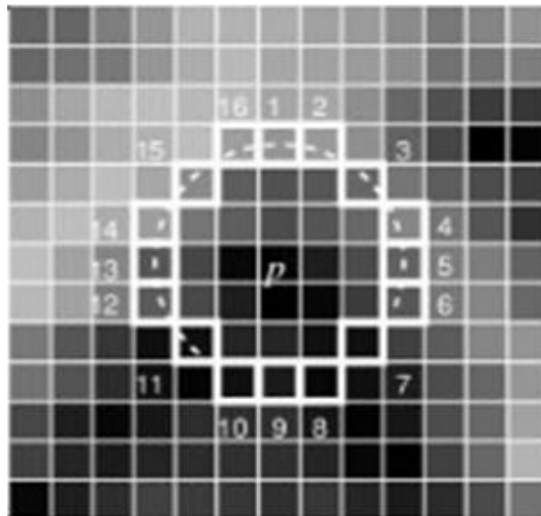
n : 几何阈值
(控制非同值像素数)

考虑模板边界

要求非同值像素连续

5.6 FAST算子——原始算法

- 加速（以 $n=12$ 为例）：
 - 检测像素点周围的四个点（1，5，9，13）中是否有三个点满足阈值要求
 - 如果不满足，则为非角点
 - 如果满足，则判断16个点中是否至少有12个连续点满足条件。若满足，则为角点，否则为非角点

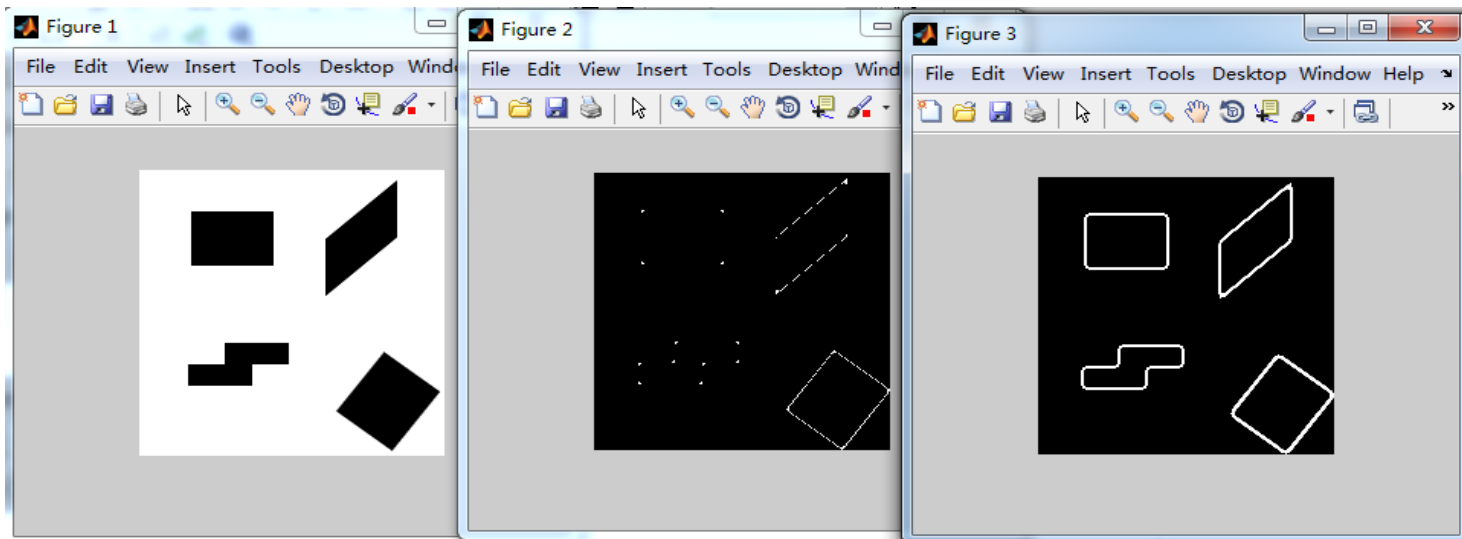


5.6 FAST算子——原始算法

- 不足：
 - 1、 $n < 12$ 时无法使用加速算法
 - 2、计算效率依赖于角点表观的分布和查询Bresenham圆上像素灰度值的查询顺序
 - 3、检测到的很多角点都是连在一起的
- 解决途径：
 - 1、机器学习
 - 2、非最大抑制（Non Maximum Suppression, NMS）

E. Rosten, T. Drummond, Machine learning for high-speed corner detection, ECCV 2006.

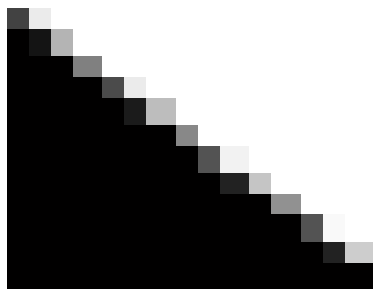
5.6 FAST算子—— SUSAN算子实验



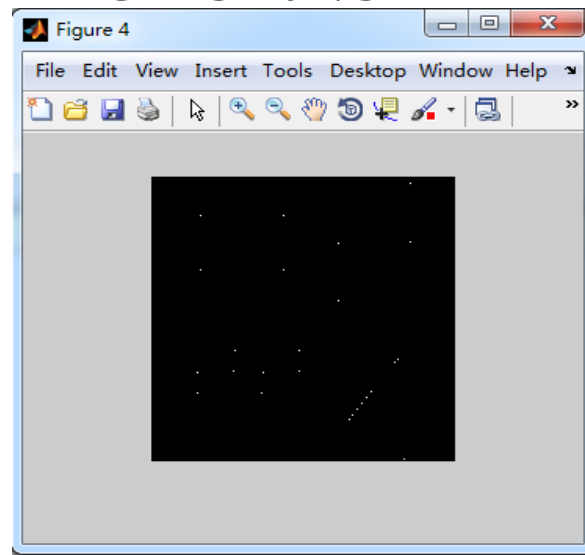
原图

$G=1S_{max}/2$

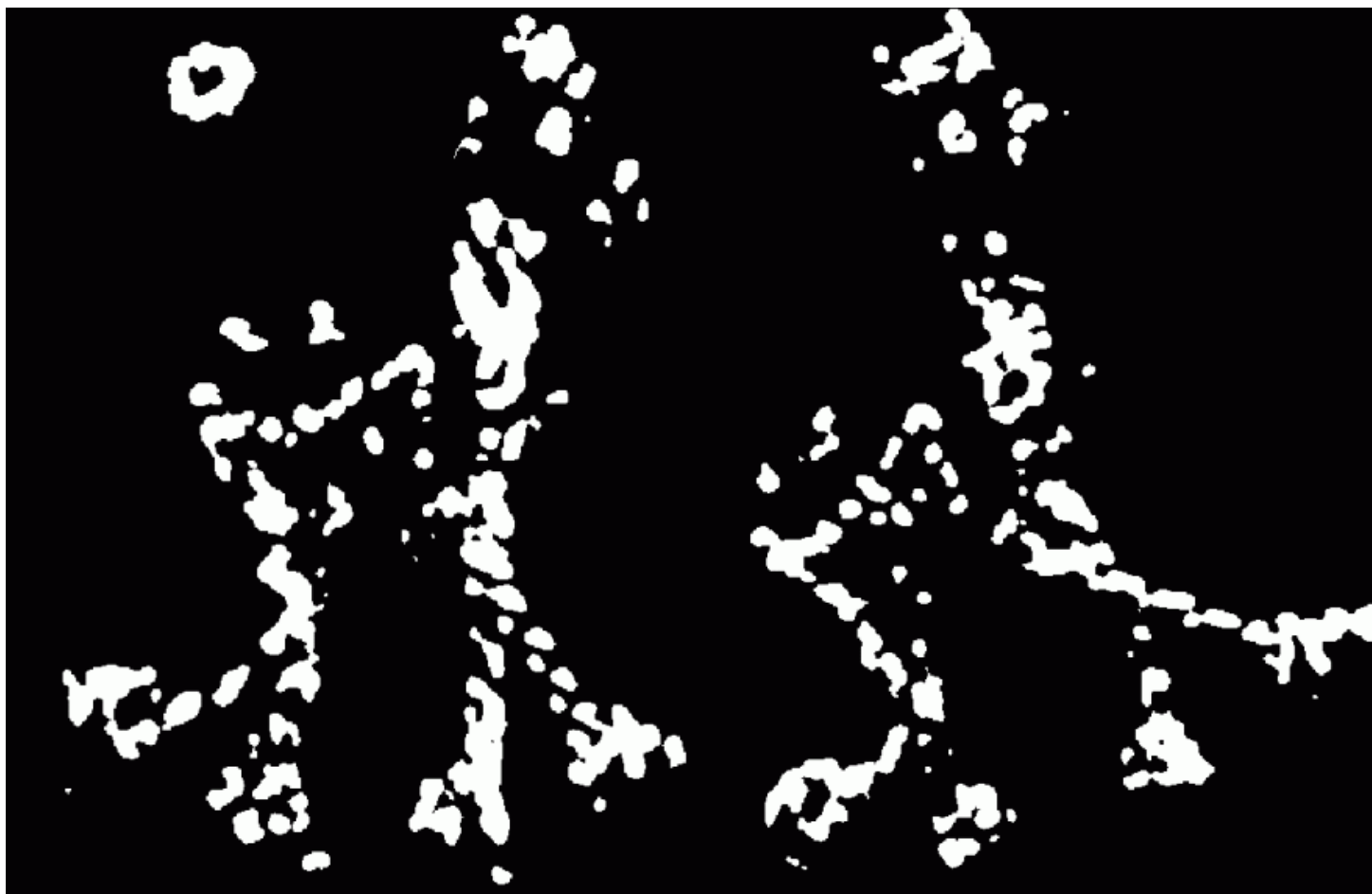
$G=2S_{max}/3$



非最大抑制
(non maximum
suppression)

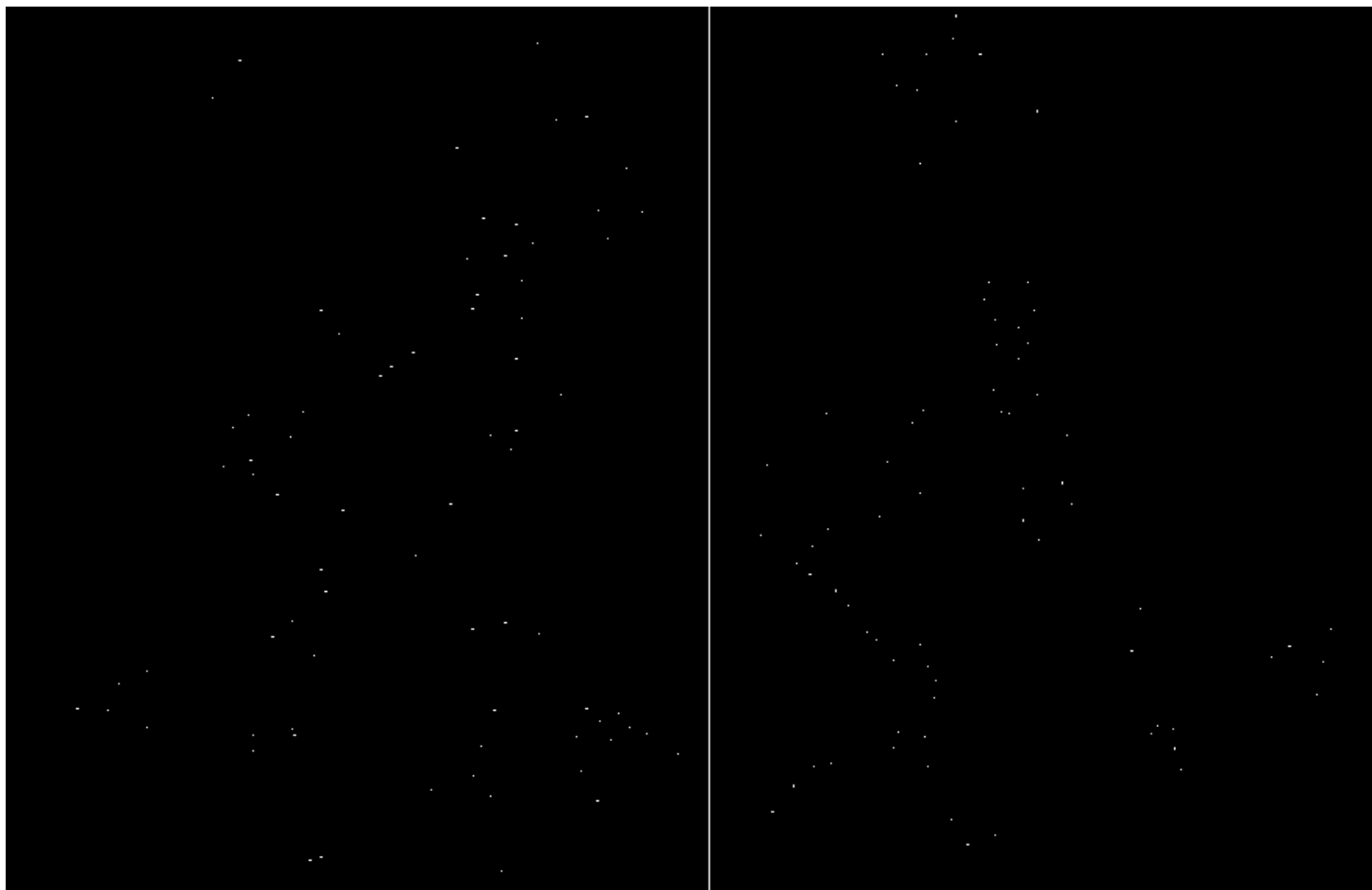


5.6 FAST算子—— Harris算子实验



响应图阈值化

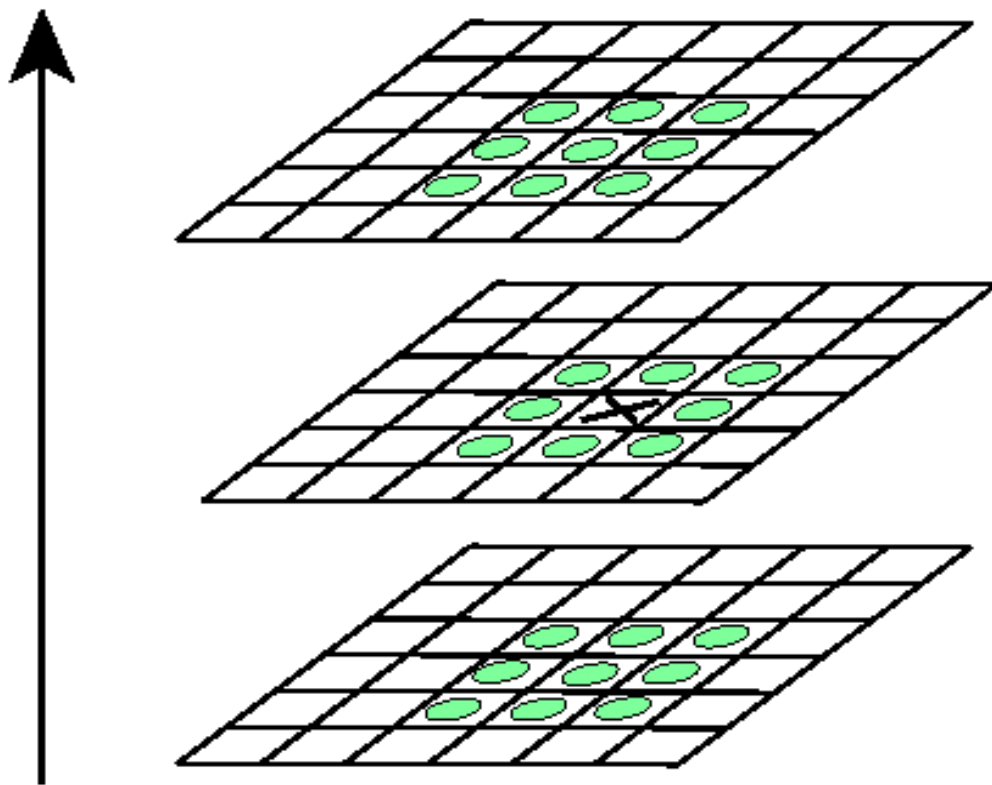
5.6 FAST算子——Harris算子实验



非最大抑制

5.6 FAST算子——SIFT算子原理

- 尺度空间极值检测
 - 空间极值点检测



对DoG金字塔中的每一像素，在三维邻域（两维图像、一维尺度）内判断是否极值点

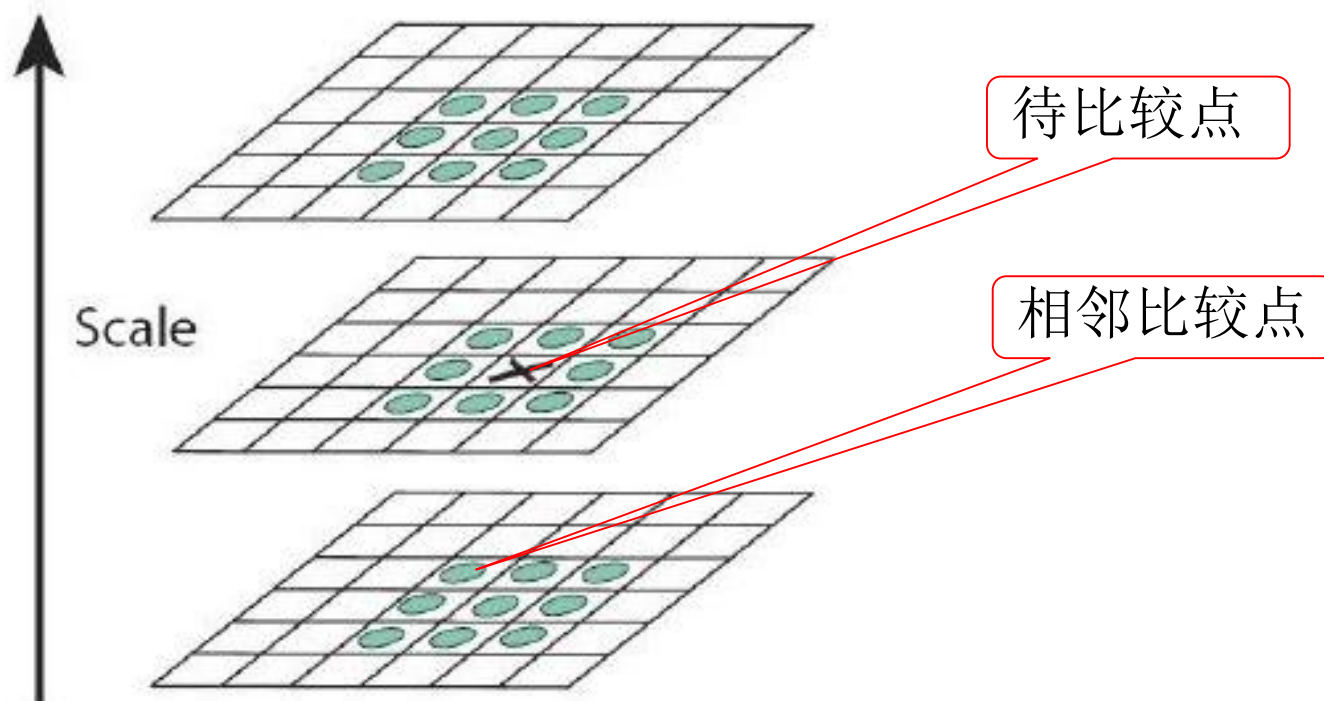
可以确定极值点的位置及尺度

极值的大小反映了极值处的对比度（反差）

为得到每一级的 s 个极值结果，高斯金字塔的每一级需有 $(s + 3)$ 幅图像

5.6 FAST算子——SURF算子原理

- 特征点定位
 - 非极大抑制



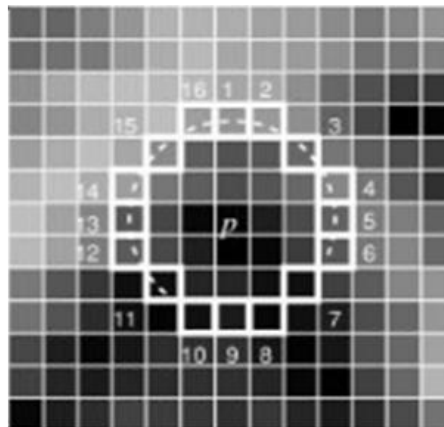
5.6 FAST算子——改进算法

- 机器学习：
 - 1、给定训练图像集合，设定 n 和阈值 t
 - 2、用前述角点检测算法检测出训练图像集中所有的角点
 - 3、对于像素 p ，其Bresenham圆上的像素 $p \rightarrow x$ ， $x \in \{1, 2, \dots, 16\}$ ，有如下三种状态：

$$S_{p \rightarrow x} = \begin{cases} d & I_{p \rightarrow x} \leq I_p - t & \text{暗} \\ s & I_p - t < I_{p \rightarrow x} < I_p + t & \text{相似} \\ b & I_{p \rightarrow x} \geq I_p + t & \text{亮} \end{cases}$$

5.6 FAST算子——改进算法

- 机器学习：
 - 4、给定一个 x ，对所有的像素 $p \in P$ （训练图像集中的**所有**像素），计算 $S_{p \rightarrow x}$ ，从而将 P 划分为 P_d 、 P_s 、 P_b 三个集合
 - 5、定义一个布尔变量 K_p ，当像素 p 为角点时， K_p 为真，否则为假
 - 6、若将 x 视为特征，则可采用ID3算法构建决策树



5.6 FAST算子——改进算法

- 机器学习：
 - 6.1、计算集合 P 的 K_p 熵

$$H(P) = (c + \bar{c}) \log_2 (c + \bar{c}) - c \log_2 c - \bar{c} \log_2 \bar{c}$$

其中

$$c = \left| \left\{ p \mid K_p \text{为真} \right\} \right|$$

$$\bar{c} = \left| \left\{ p \mid K_p \text{为假} \right\} \right|$$

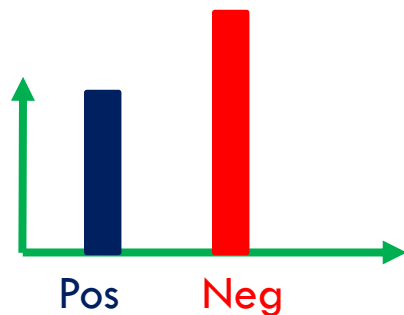
即 c 和 $\bar{c}$ 分别表示训练图像集中角点和非角点的像素数

角点和非角点各占一半时，熵最大；
全为角点或非角点时，熵最小，为0

5.6 FAST算子——改进算法

- 机器学习：
 - 6.2、选择使得信息增益最大的 x

$$IG = H(P) - H(P_d) - H(P_s) - H(P_b)$$



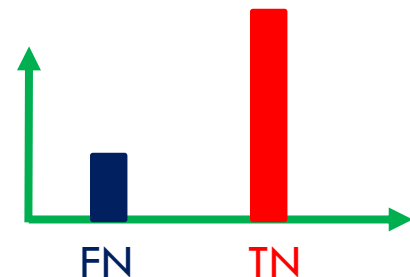
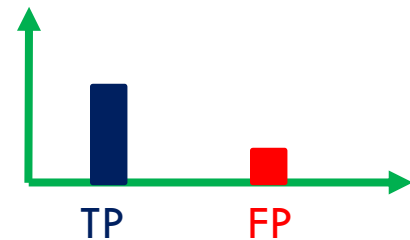
Data (A)



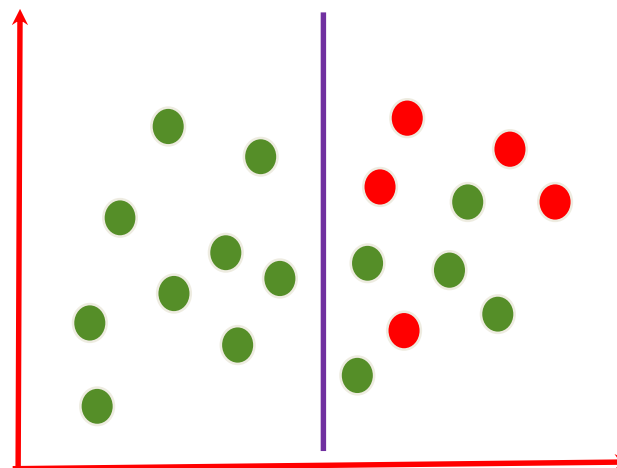
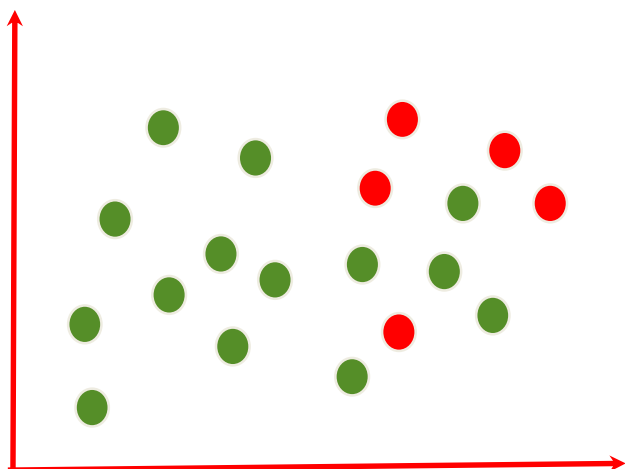
Data(A+)

Data(A-)

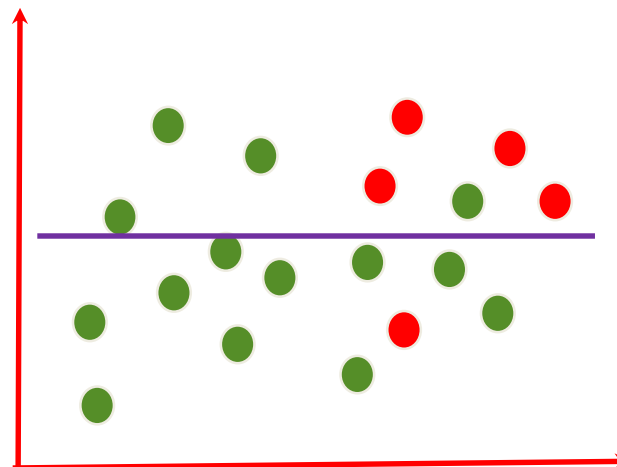
By computer



5.6 FAST算子——改进算法

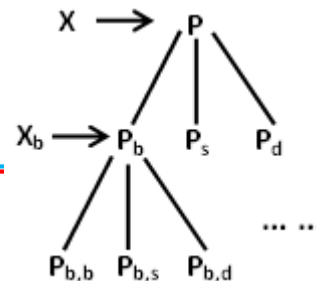


信息增益大



信息增益小

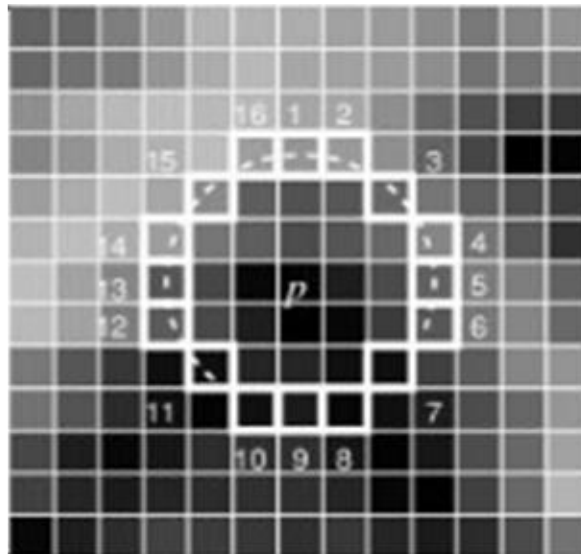
5.6 FAST算子——改进算法



- 机器学习：
 - 6.3、对 P_d 、 P_s 、 P_b 迭代重复上述过程，例如选择 x_b 将 P_b 划分为 $P_{b,d}$ 、 $P_{b,s}$ 、 $P_{b,b}$ ，选择 x_s 将 P_s 划分为 $P_{s,d}$ 、 $P_{s,s}$ 、 $P_{s,b}$ ，直至某一子集的熵为0（子集中的像素全为角点或全不为角点）
 - 6.4、为了进一步加速，限制 x_b 、 x_s 、 x_d 必须是相同的，因此第二次判断时的像素是相同的，于是第一次判断和第二次判断可以使用许多高性能微处理器都拥有的向量指令并行处理。由于大多数像素经过两次判断后就被排除，因此极大地提高了速度。

5.6 FAST算子——改进算法

- 机器学习：
 - 机器学习得到的角点检测器取决于训练样本。若训练样本未包含所有可能情况，该角点检测器与原始的采用分段测试（segment test）的检测器不会完全相同。这可以通过生成满足分段测试情况的所有可能样本来得到。



5.6 FAST算子——改进算法

- 非最大抑制：
 - 1、构建响应（得分）函数 V
 - 2、用 V 对每个检测到的角点打分
 - 3、若角点 a 有一个得分更高的邻居 b ，则将其删除

5.6 FAST算子——改进算法

- 非最大抑制：
 - 关键是构建响应（得分）函数V
 - 三种定义方式（1）使得p仍为角点的最大n值；（2）使得p仍为角点的最大t值；（3）p与其Bresenham圆上连续亮或暗像素灰度值绝对差之和
 - 定义（1）和（2）容易导致很多像素拥有相同的得分值
 - 使用定义（3）的简化版（不要求连续）：

$$V = \max \left(\sum_{x \in S_{\text{bright}}} |I_{p \rightarrow x} - I_p| - t, \sum_{x \in S_{\text{dark}}} |I_p - I_{p \rightarrow x}| - t \right)$$

$$S_{\text{bright}} = \left\{ x \mid I_{p \rightarrow x} \geq I_p + t \right\} \quad S_{\text{dark}} = \left\{ x \mid I_{p \rightarrow x} \leq I_p - t \right\}$$

作业

- 请针对如下的6个分属于两类的二维特征样本，分析采用哪一维特征进行分类，可以获得最大的信息增益。

类别1: $(2, 1)$, $(2, 2)$, $(4, 3)$

类别2: $(2, 3)$, $(2, 4)$, $(4, 2)$

5.6 FAST算子——扩展与改进

- FAST-ER
 - E. Rosten, R. Porter, T. Drummond, Faster and better: A machine learning approach to corner detection, IEEE TPAMI, 2010.
 - FAST-ER: 增强可重复性、轻微降低计算效率

5.6 FAST算子——扩展与改进

- AGAST (Adaptive and Generic Accelerated Segment Test)
 - E. Mair, G. D. Hager, Adaptive and generic corner detection based on the accelerated segment test, ECCV 2010.
 - 对FAST底层的the accelerated segment test进行改进，通过在扩展配置空间寻找最优决策树，使用特定树的组合获得自适应并且通用的accelerated segment test
 - 对FAST 9-16 detector提速约13%，对FAST 7-12提速最高30%，对FAST 5-8 提速最高50%
 - AGAST项目网址
<http://www6.in.tum.de/Main/ResearchAgast>.

5.6 FAST算子——扩展与改进

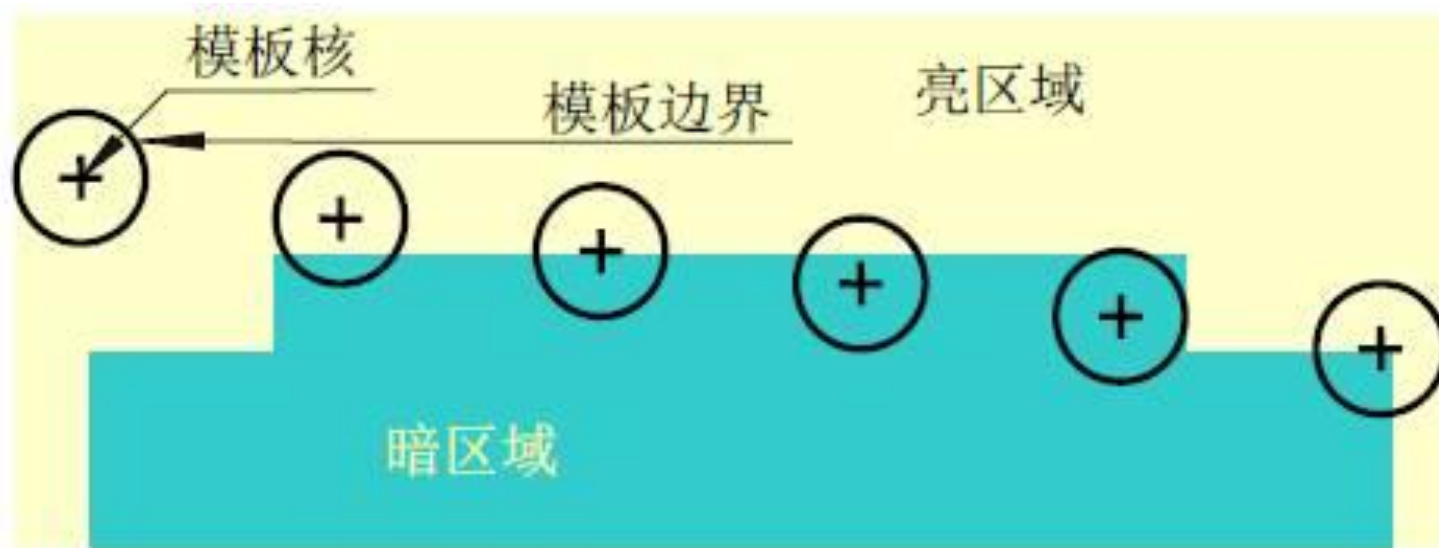
- Brisk (Binary robust invariant scalable keypoints)
 - S. Leutenegger, et.al. Brisk: Binary robust invariant scalable keypoints, ICCV 2011.
 - 在BRISK描述子中提出multi-scale AGAST检测器，并用实验证明与SURF有等效的可重复性 (equivalent repeatability)
 - 对Graffiti序列的第一幅图检测时间为17.20ms, 约为SURF的16%，SIFT的1%
 - BRISK项目地址
<http://www.asl.ethz.ch/people/lestefan/personal/BRISK>

5.7 SuperPoint深度网络

- 回顾传统特征点检测算子，均具有如下三个特点：
 - 观察现象：观察所关注特征点（如边缘、角点等）的特点
 - 总结规律：总结特征点所表现出的规律
 - 构建算子：构建特征点检测算子

5.7 SuperPoint深度网络

- SUSAN算子：
 - 特征点与非特征点的核同值区大小不同
 - 边缘点和角点的核同值区相对较小



5.7 SuperPoint深度网络

- SUSAN算子：
 - 构建关于核同值区的响应

$$R(x_0, y_0) = \begin{cases} G - S(x_0, y_0) & \text{如果 } S(x_0, y_0) < G \\ 0 & \text{其他} \end{cases}$$

$$S(x_0, y_0) = \sum_{(x, y) \in M(x, y)} C(x_0, y_0; x, y)$$

$$C(x_0, y_0; x, y) = \begin{cases} 1 & \text{当 } |f(x_0, y_0) - f(x, y)| \leq T \\ 0 & \text{当 } |f(x_0, y_0) - f(x, y)| > T \end{cases}$$

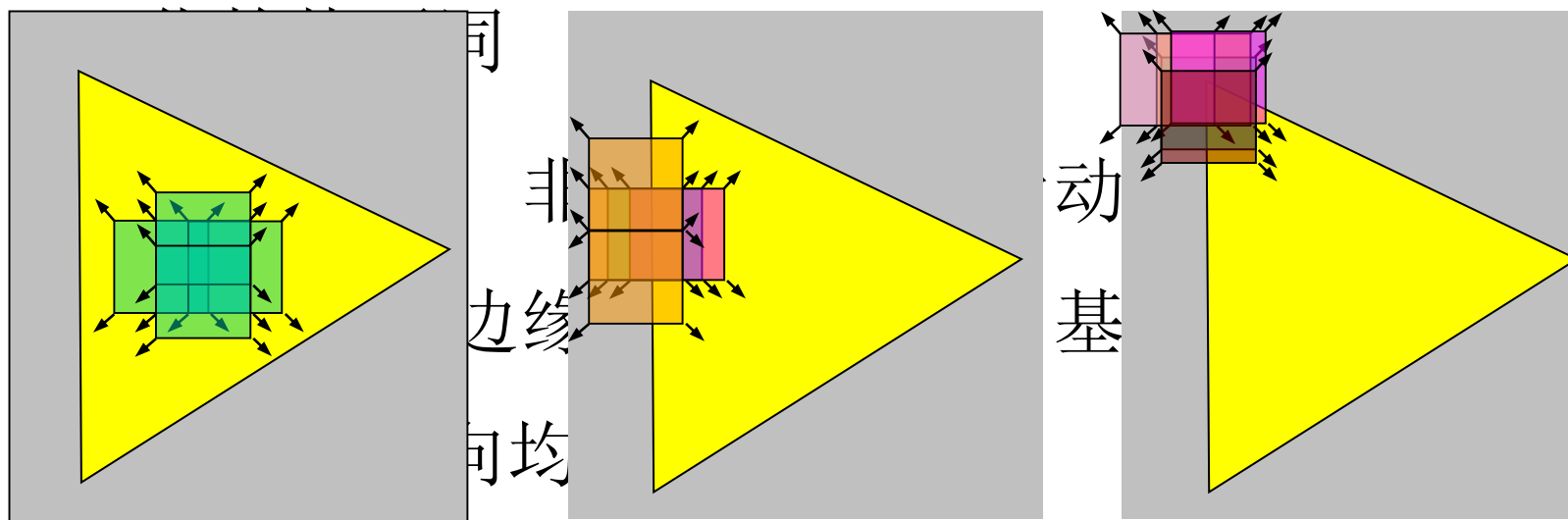
5.7 SuperPoint深度网络

- Harris算子：
 - 特征点与非特征点的周围滑动窗口内像素变化趋势不同
 - 非特征点周围滑动窗口内像素基本不变，边缘点沿边缘方向基本不变，角点沿各个方向均有剧烈变化

5.7 SuperPoint深度网络

- Harris算子:

— 特征点与非特征点的周围滑动窗口内像素变



5.7 SuperPoint深度网络

- Harris算子：
 - 构建关于像素点周围滑动窗口内像素变化趋势的响应

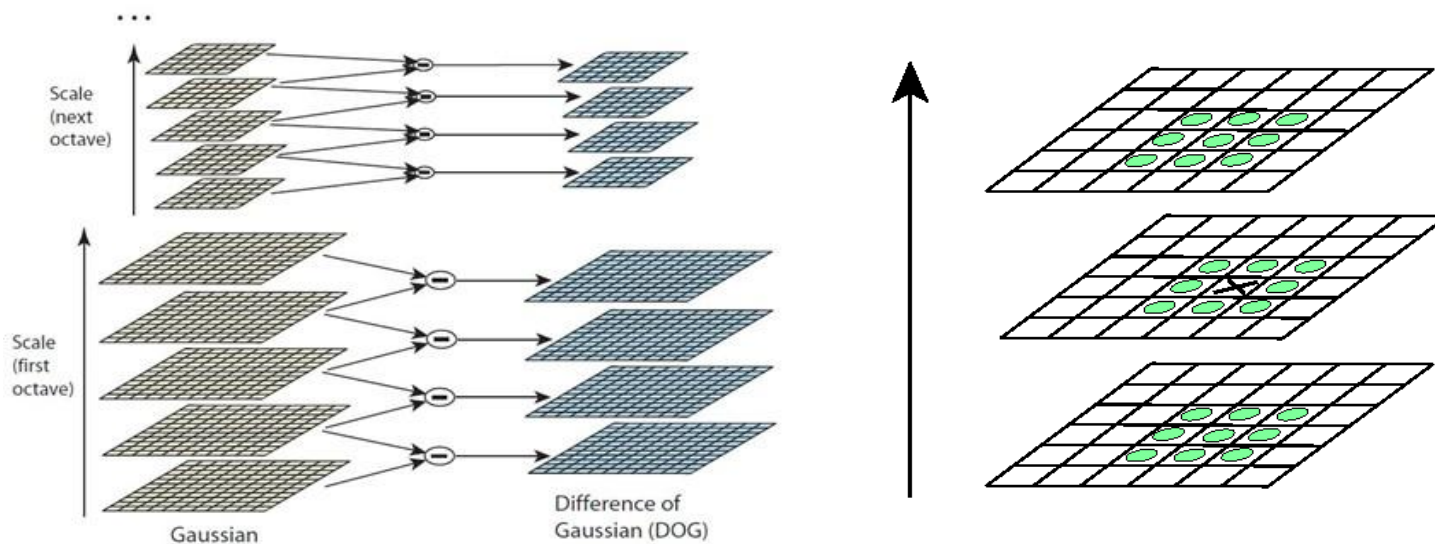
$$E(u, v) = \sum_{x, y} w(x, y) [I(x+u, y+v) - I(x, y)]^2$$

$$E(u, v) \cong [u \quad v] M \begin{bmatrix} u \\ v \end{bmatrix}$$

$$M = \sum_{x, y} w(x, y) \begin{bmatrix} I_x^2 & I_x I_y \\ I_x I_y & I_y^2 \end{bmatrix}$$

5.7 SuperPoint深度网络

- SIFT算子：
 - 特征点与非特征点的DOG值局部特性不同
 - 特征点的DOG值是局部像素空间和尺度空间内DOG值的极值



5.7 SuperPoint深度网络

- SIFT算子：
 - 构建关于像素点DOG值是否局部像素空间和尺度空间内DOG值的极值的响应

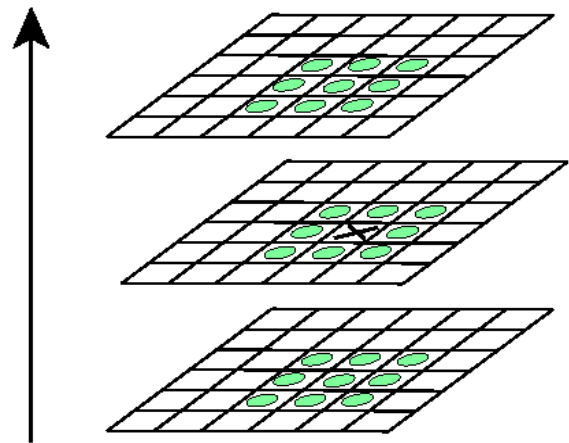
$$D(x, y, \sigma) = L(x, y, k\sigma) - L(x, y, \sigma)$$

$$L(x, y, \sigma) = G(x, y, \sigma) \otimes I(x, y)$$

5.7 SuperPoint深度网络

- SURF算子：
 - 特征点与非特征点的Hessian矩阵行列式值不同
 - 特征点的Hessian矩阵行列式值是局部像素空间和尺度空间内Hessian矩阵行列式值的极值

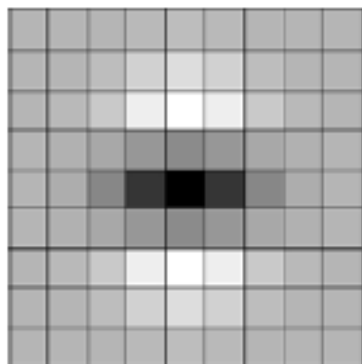
$$H(\mathbf{x}, \sigma) = \begin{bmatrix} L_{xx}(\mathbf{x}, \sigma) & L_{xy}(\mathbf{x}, \sigma) \\ L_{xy}(\mathbf{x}, \sigma) & L_{yy}(\mathbf{x}, \sigma) \end{bmatrix}$$



5.7 SuperPoint深度网络

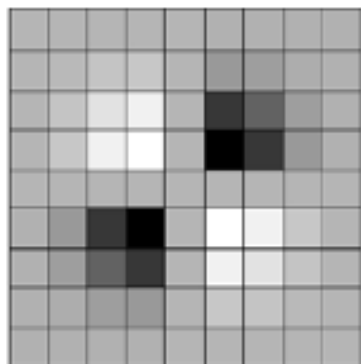
- SURF算子:

– 用盒子滤波器近似计算Hessian矩阵



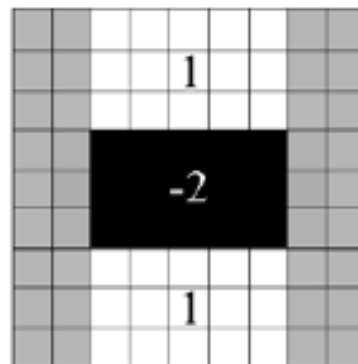
高斯二阶偏导数

L_{yy}



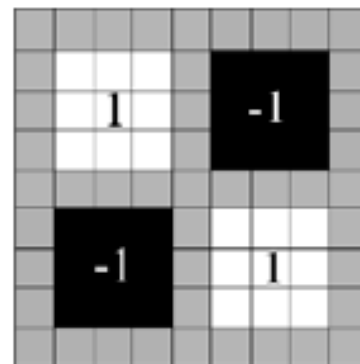
高斯二阶偏导数

L_{xy}



盒子滤波近似函数

D_{yy}



盒子滤波近似函数

D_{xy}

$$H(\mathbf{x}, N) = \begin{bmatrix} D_{xx}(\mathbf{x}, N) & D_{xy}(\mathbf{x}, N) \\ D_{xy}(\mathbf{x}, N) & D_{yy}(\mathbf{x}, N) \end{bmatrix}$$

5.7 SuperPoint深度网络

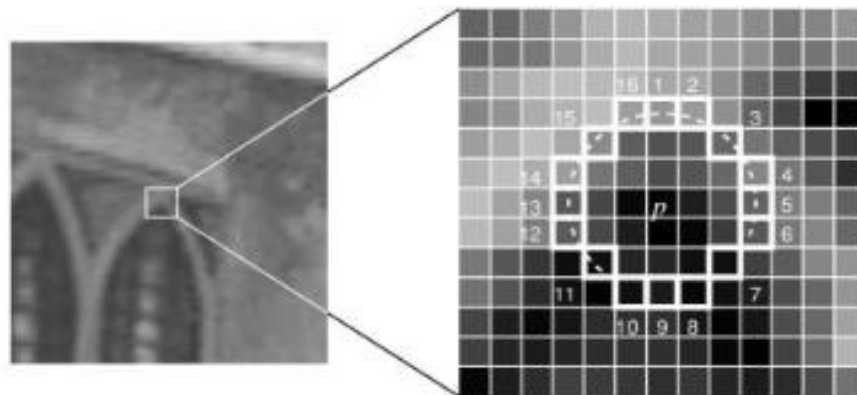
- SURF算子：
 - 构建关于像素点Hessian矩阵行列式值是否局部像素空间和尺度空间内Hessian矩阵行列式值的极值的响应

$$\det(H_{approx}) = D_{xx}D_{yy} - (wD_{xy})^2$$

$$w = \frac{|L_{xy}(1.2)|_F |D_{yy}(9)|_F}{|L_{yy}(1.2)|_F |D_{xy}(9)|_F} = 0.912... \approx 0.9$$

5.7 SuperPoint深度网络

- FAST算子：
 - 角点与非角点的Bresenham圆上像素与角点/非角点的灰度关系不同
 - 角点的Bresenham圆上存在连续的像素灰度值都高于或低于角点的灰度值一定大小



$N = 16$ 示意图

5.7 SuperPoint深度网络

- FAST算子：
 - 构建关于像素点的Bresenham圆上是否存在连续的像素灰度值都高于或低于该像素点的灰度值一定大小的响应

在像素 p 的Bresenham圆上选择 N 个像素。如果这个像素中存在 n 个连续像素的灰度值都高于 $I_p + t$ ，或低于 $I_p - t$ ，则 p 是角点，其中 I_p 为 p 的灰度值， t 为阈值。

5.7 SuperPoint深度网络

- 传统特征点检测算子的不足：
 - “现象-规律-算子”的研究思路依赖于人工经验（即所谓的手工特征）
 - 复杂条件（如光照变化、成像模糊）下特征点所表现出的独有现象难以观察、规律难以总结，导致构建的算子鲁棒性差

5.7 SuperPoint深度网络

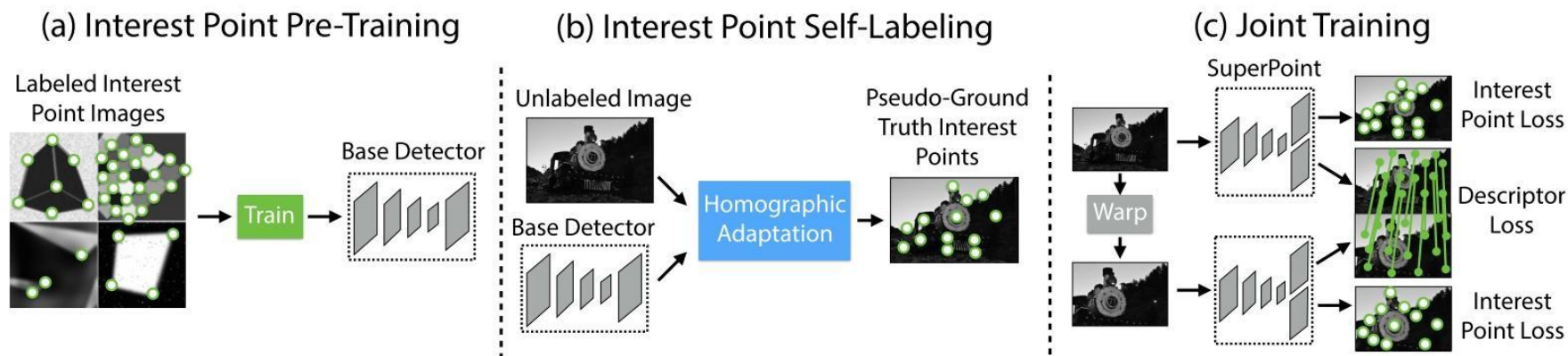
- 基于深度学习的特征点检测：
 - 利用大量训练样本图像
 - 构建关于特征点的损失函数
 - 采用“输入图像-输出特征点”的端到端框架
自动学习高重复、强判别特征
 - 突破传统瓶颈，提升复杂场景适应性

5.7 SuperPoint深度网络

- 动机：
 - 如何标注训练样本图像中的特征点：引入自监督学习，避免人工大量标注特征点
 - 如何在进行特征点检测的同时获得特征点的描述：联合优化特征点检测与描述

5.7 SuperPoint深度网络

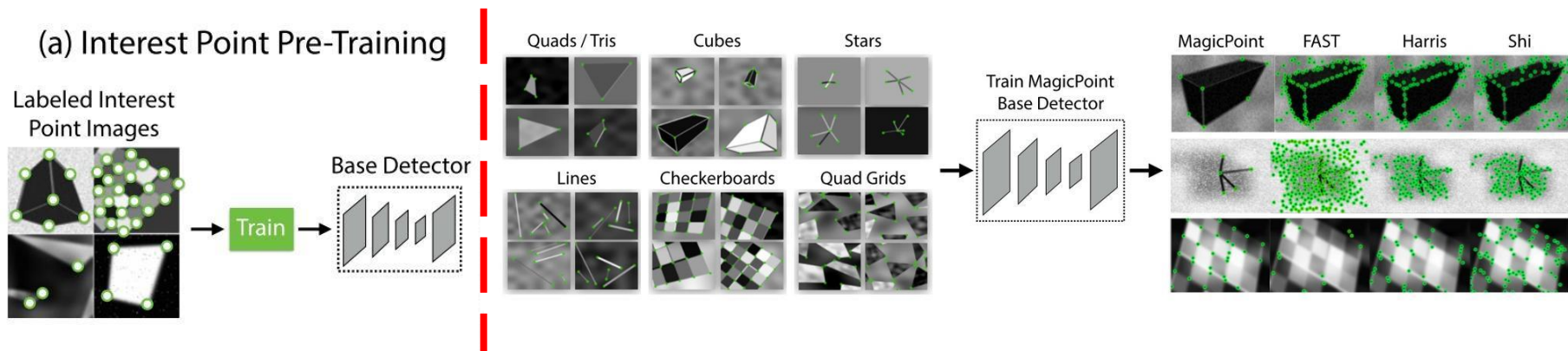
- 网络框架:



- (a): 特征点检测预训练
- (b): 特征点检测自标注
- (c): 特征点检测与描述联合训练

5.7 SuperPoint深度网络

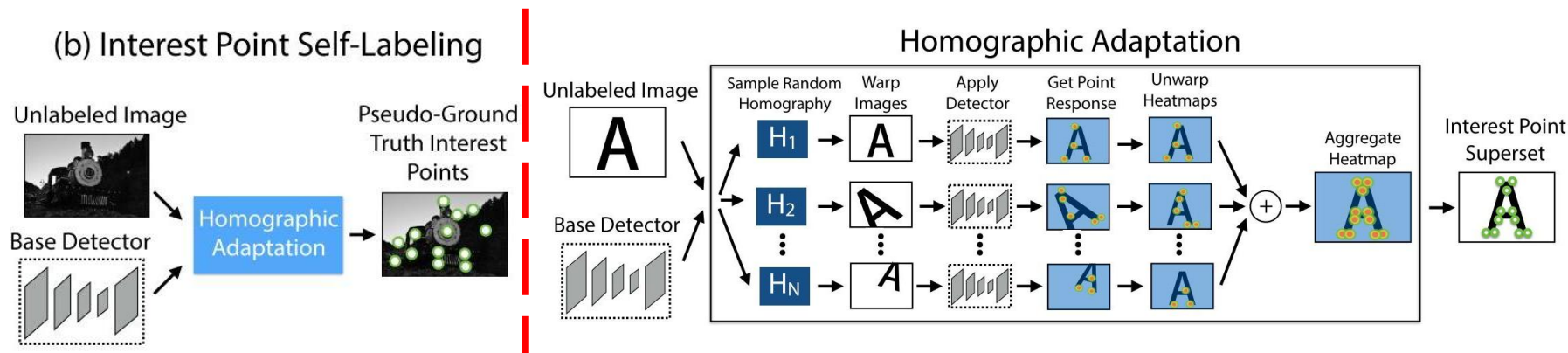
- 特征点检测预训练:



- 构建包含简单的几何形状（如三角形、矩形、线条等）的合成数据集**Synthetic Shapes**。
- 合成数据集图像中的特征点（角点、边缘）位置是明确且无二义性的可作为标签训练网络**Base Detector**，从而学习如何从简单几何形状中检测特征点。

5.7 SuperPoint深度网络

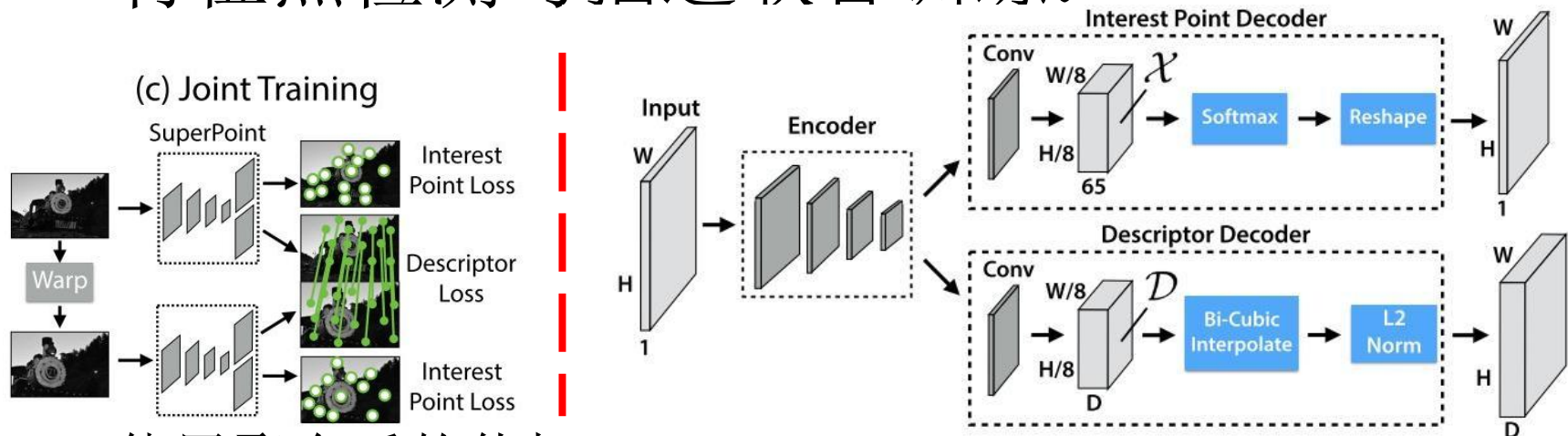
- 特征点检测自标注（自监督）：



- 对于每张训练图像，随机生成一组单应性变换（如平移、旋转、缩放等），并将这些变换应用到图像上，生成一组变换后的图像（多视角图像）。
- 使用预训练的Base Detector在变换后的图像上检测特征点。
- 聚合（平均）各变换后图像的特征点检测结果，生成特征点概率图（伪标签），从而提高检测结果的鲁棒性和一致性。

5.7 SuperPoint深度网络

- 特征点检测与描述联合训练:



- 使用聚合后的伪标签训练SuperPoint模型的特征点检测模块。
- 共享编码器：基于全卷积网络(FCN)，通过卷积与池化层提取多尺度共享特征。
- 特征点解码器：生成像素级得分图，通过非极大值抑制筛选出概率最高的特征点位置
- 描述子解码器：生成密集描述子图，为每个特征点提取高维局部特征向量，并进行归一化处理，确保特征独特性。

5.7 SuperPoint深度网络

- 损失函数:

$$L(X, X', D, D'; Y, Y', S) = L_p(X, Y) + L_p(X', Y') + \lambda L_d(D, D', S)$$

$$L_p(X, Y) = \frac{1}{H * W} \sum_h \sum_w \text{CrossEntropy}(x_{hw}, y_{hw})$$

$$L_p(X', Y') = \frac{1}{H * W} \sum_h \sum_w \text{CrossEntropy}(x'_{hw}, y'_{hw})$$

$$L_d(D, D', S) = \frac{1}{(H * W)^2} \sum_h \sum_w \sum_{h'} \sum_{w'} l_d(d_{hw}, d'_{h'w'}, s_{hwh'w'})$$

其中,

X 和 X' 是输入图像及其单应性变换图像的特征点检测结果

Y 和 Y' 是输入图像及其单应性变换图像的伪标签

D 和 D' 是输入图像及其单应性变换图像的特征点描述结果

S 是输入图像及其单应性变换图像的特征点对应关系

$$l_d(d, d', s) = \lambda_d * s * \max(0, m_p - d^T d') + (1 - s) * \max(0, d^T d' - m_n)$$

5.7 SuperPoint深度网络

- 实验结果:

运行时间: Titan X GPU, 480×640 图像, 13ms, 70FPS

重复性: Hpatches数据集

	57 Illumination Scenes		59 Viewpoint Scenes	
	NMS=4	NMS=8	NMS=4	NMS=8
<i>SuperPoint</i>	.652	.631	.503	.484
<i>MagicPoint</i>	.575	.507	.322	.260
<i>FAST</i>	.575	.472	.503	.404
<i>Harris</i>	.620	.533	.556	.461
<i>Shi</i>	.606	.511	.552	.453
<i>Random</i>	.101	.103	.100	.104

知乎 @陈知

5.7 SuperPoint深度网络

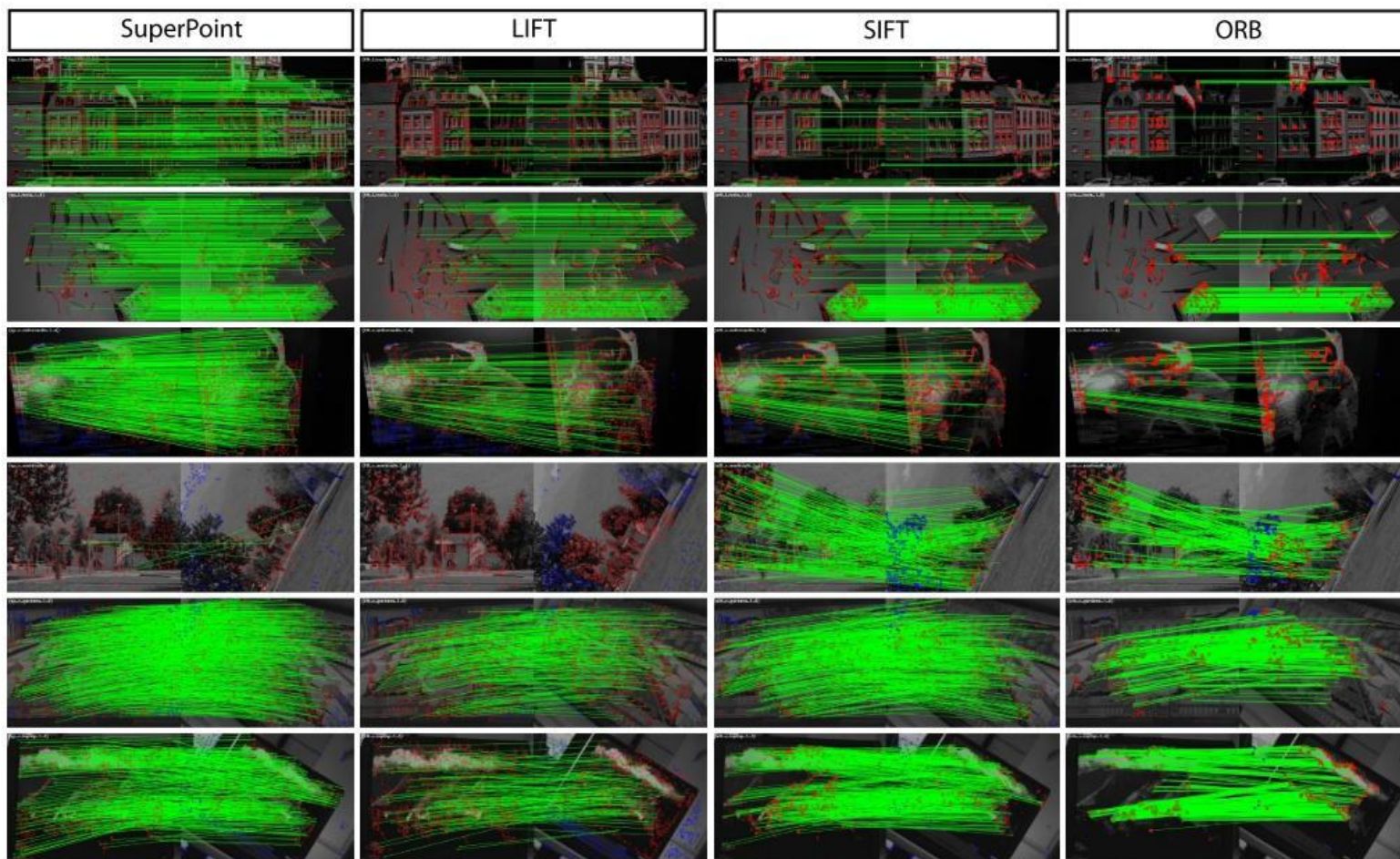
- 实验结果:

单应性估计: Hpatches数据集

	Homography Estimation			Detector Metrics		Descriptor Metrics	
	$\epsilon = 1$	$\epsilon = 3$	$\epsilon = 5$	Rep.	MLE	NN mAP	M. Score
<i>SuperPoint</i>	.310	.684	.829	.581	1.158	.821	.470
<i>LIFT</i>	.284	.598	.717	.449	1.102	.664	.315
<i>SIFT</i>	.424	.676	.759	.495	0.833	.694	.313
<i>ORB</i>	.150	.395	.538	.641	1.157	.735	.266

5.7 SuperPoint深度网络

- 实验结果：
可视化

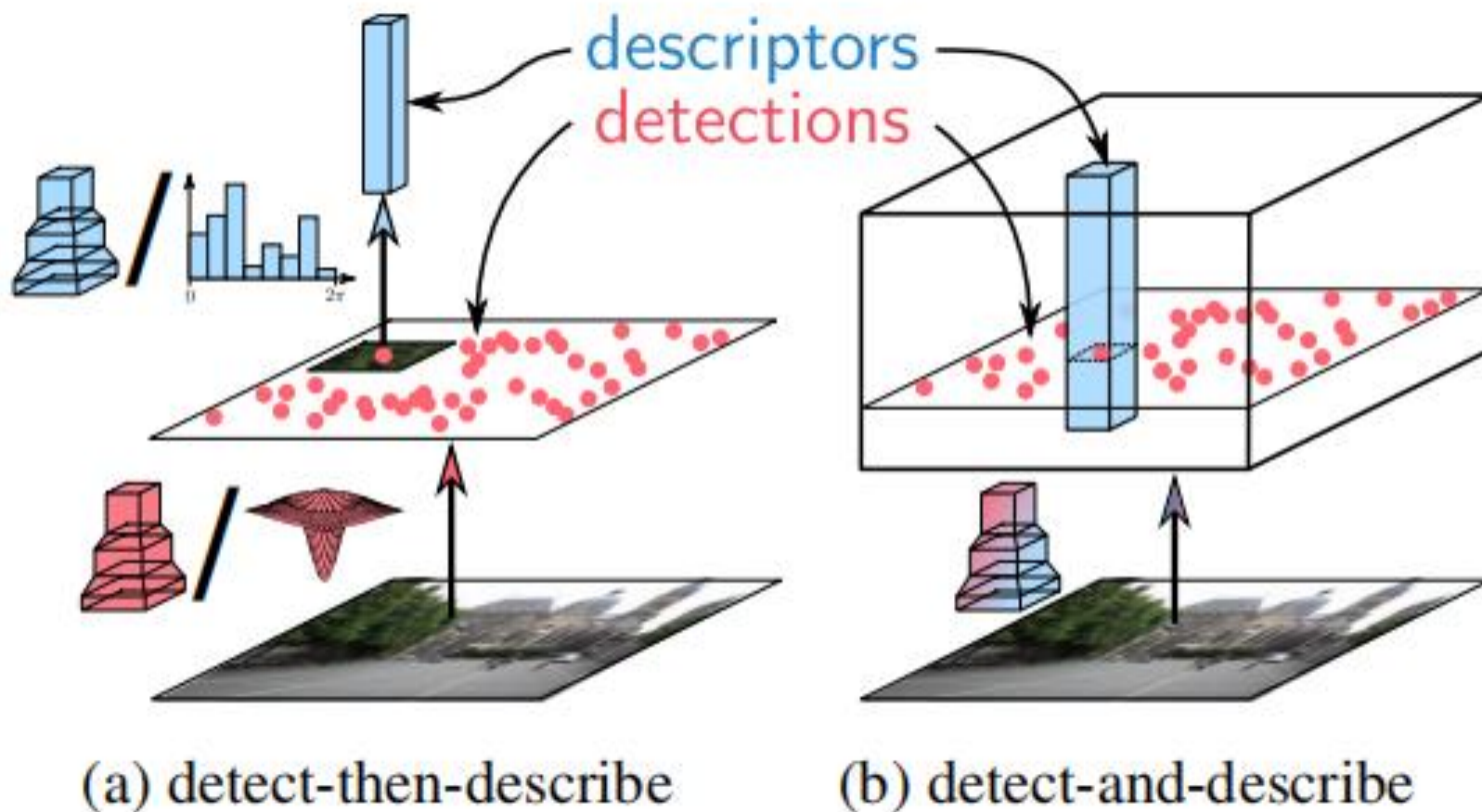


5.8 其他典型深度网络

- D2-Net:
 - 有监督方法：已知训练样本图像对间的对应
 - 不同于传统的“先检测后描述”，提出“密集描述和检测”新思路，将两个任务统一处理。
 - 直接使用在ImageNet上预训练的深度卷积网络（如VGG-16）作为骨干，利用其强大的通用视觉特征表示能力。
 - 特征图中每个空间位置的向量都是潜在的描述子，同时通过处理特征图可直接检测出关键点，实现一举两得。
 - 将检测和描述任务统一在一个框架下进行端到端训练，充分挖掘预训练模型的特征表达潜力。

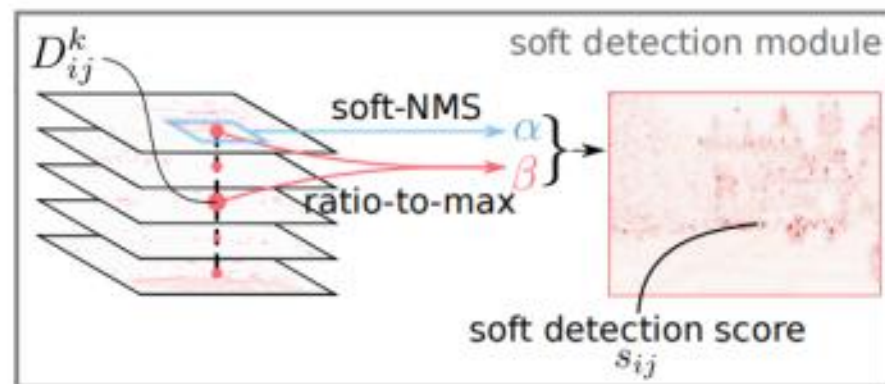
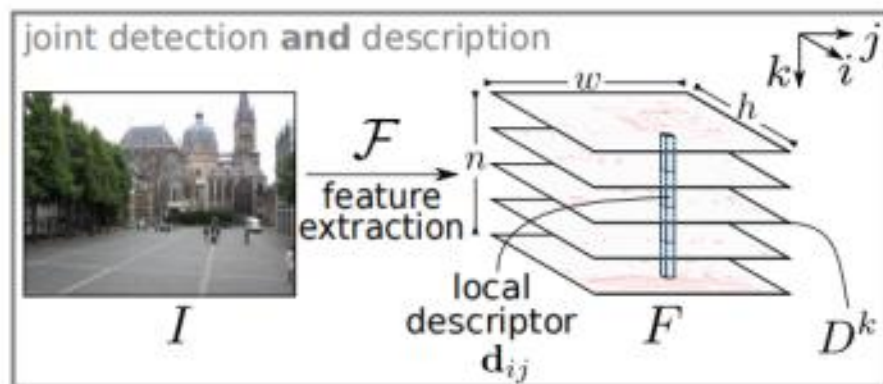
5.8 其他典型深度网络

- D2-Net:



5.8 其他典型深度网络

- D2-Net:



$$(i, j) \text{ is a detection} \iff D^k_{ij} \text{ is a local max. in } D^k$$

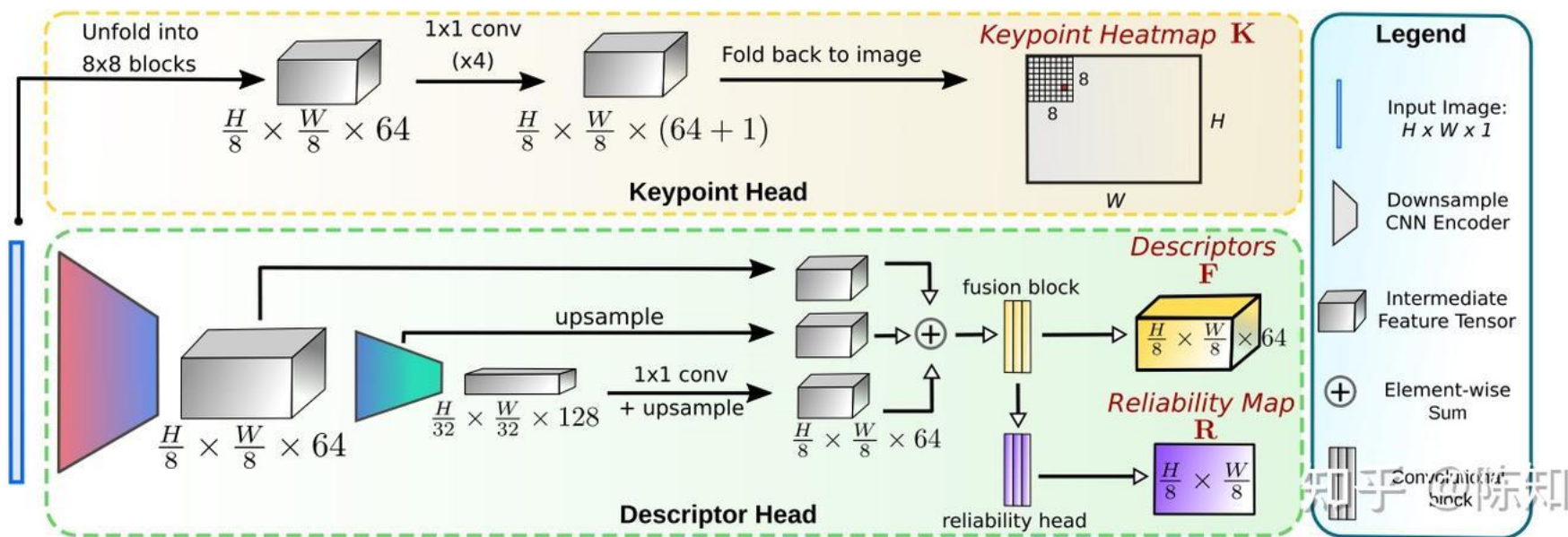
$$\text{with } k = \arg \max_t D^t_{ij} .$$

5.8 其他典型深度网络

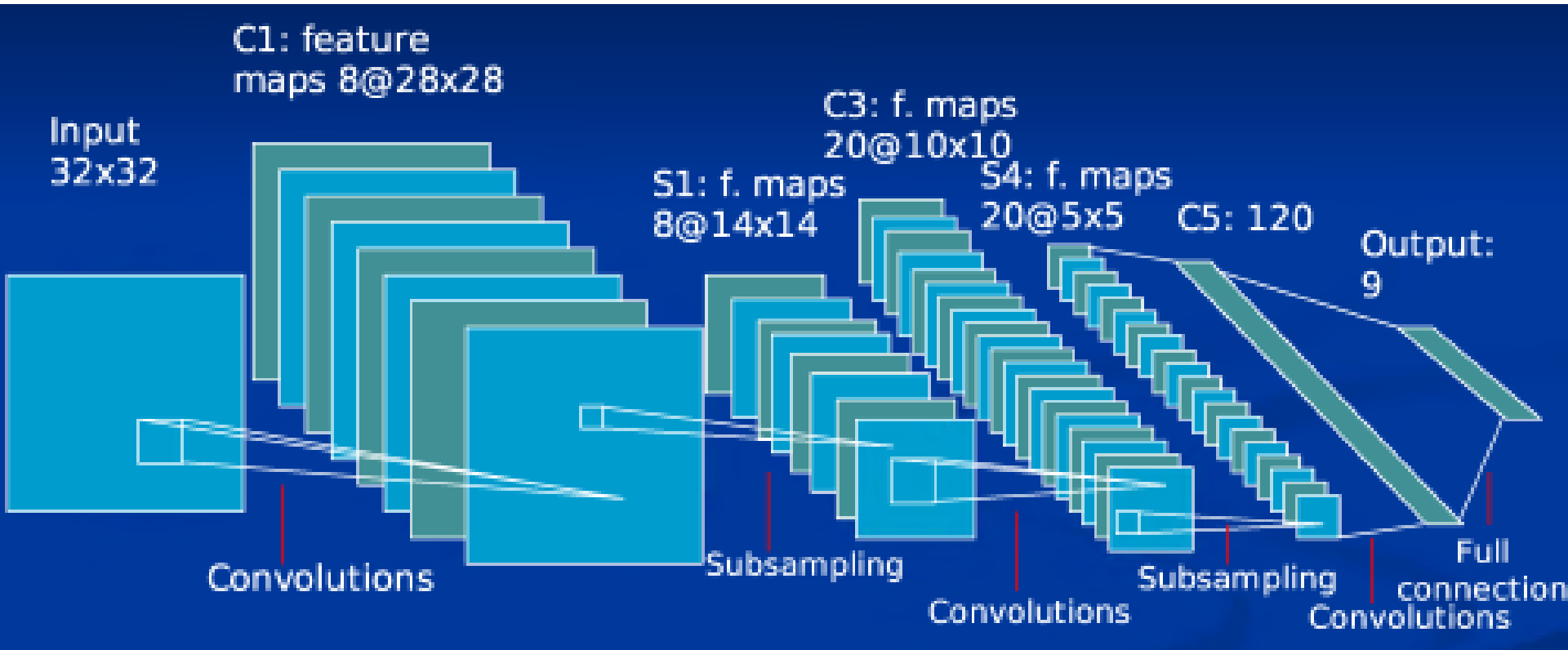
- XFEAT:
 - 有监督方法：已知训练样本图像对间的对应
 - 解决高精度算法（如D2-Net）计算复杂、难以在移动端实时运行的痛点，同时避免轻量级算法牺牲精度的问题。
 - 通过保持高分辨率特征图、减少通道数及采用深度可分离卷积等高效结构，优化网络以平衡速度与精度。
 - 为适应不同下游任务，同时支持稀疏匹配（用于SLAM）和半稠密匹配（用于立体视觉），提升算法的通用性。

5.8 其他典型深度网络

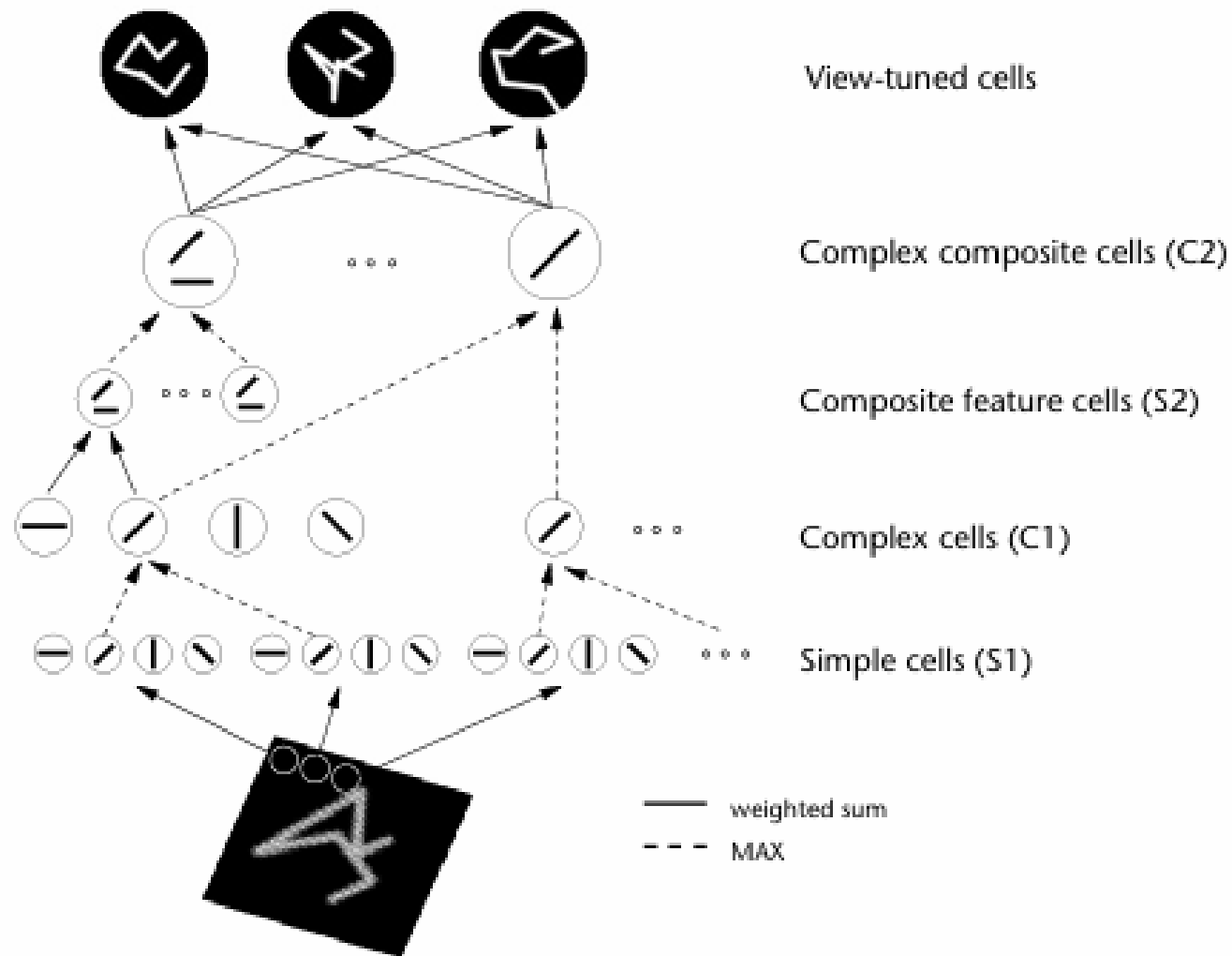
- XFEAT:



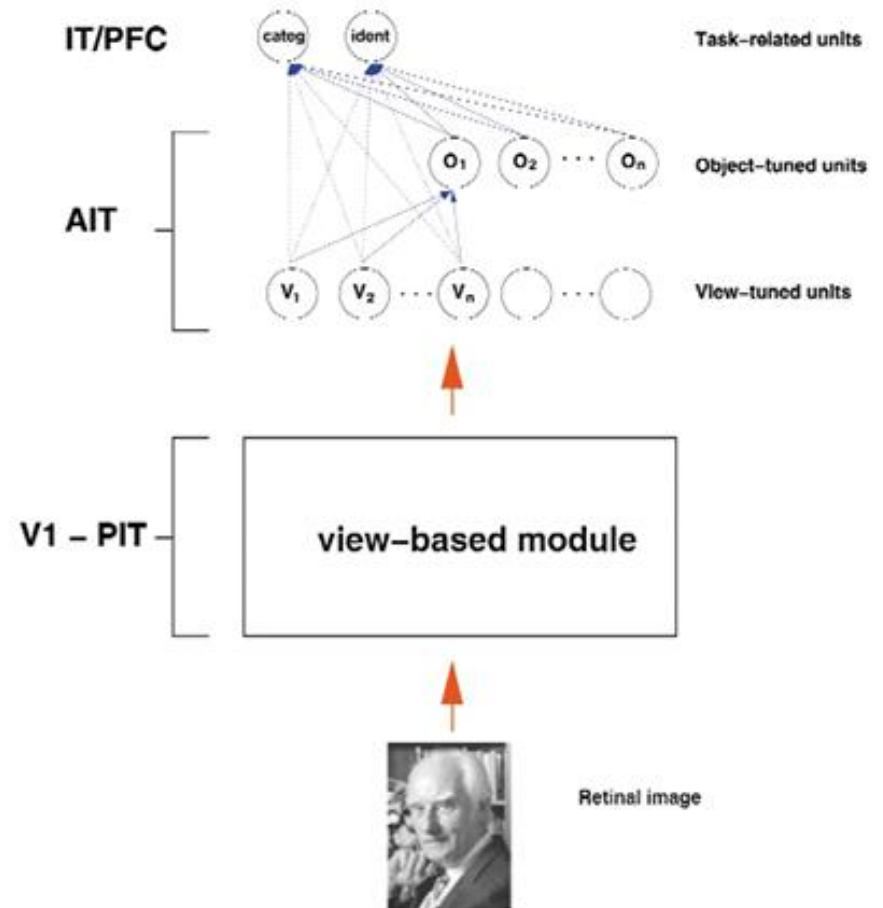
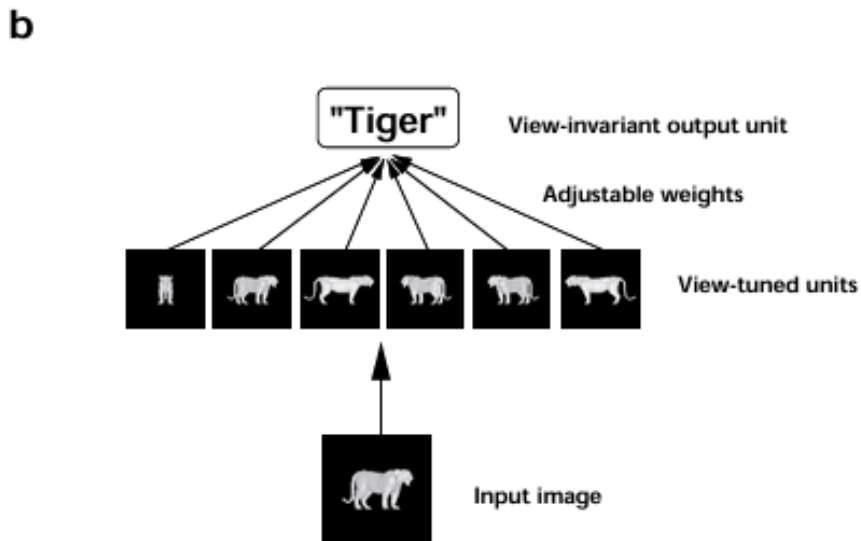
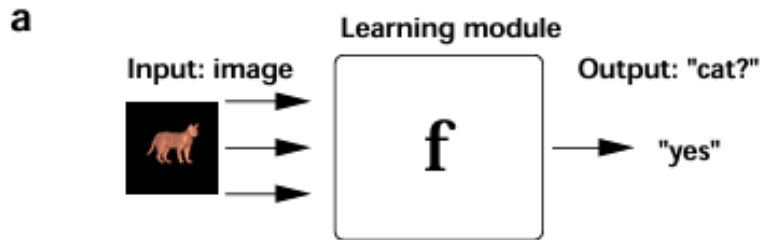
Yann Lecun, L'Éon Bottou, Yoshua Bengio, Patrick Haffner,
Gradient-Based Learning Applied to Document Recognition,
Proceedings of the IEEE, 1998 (**LeNet-5**)



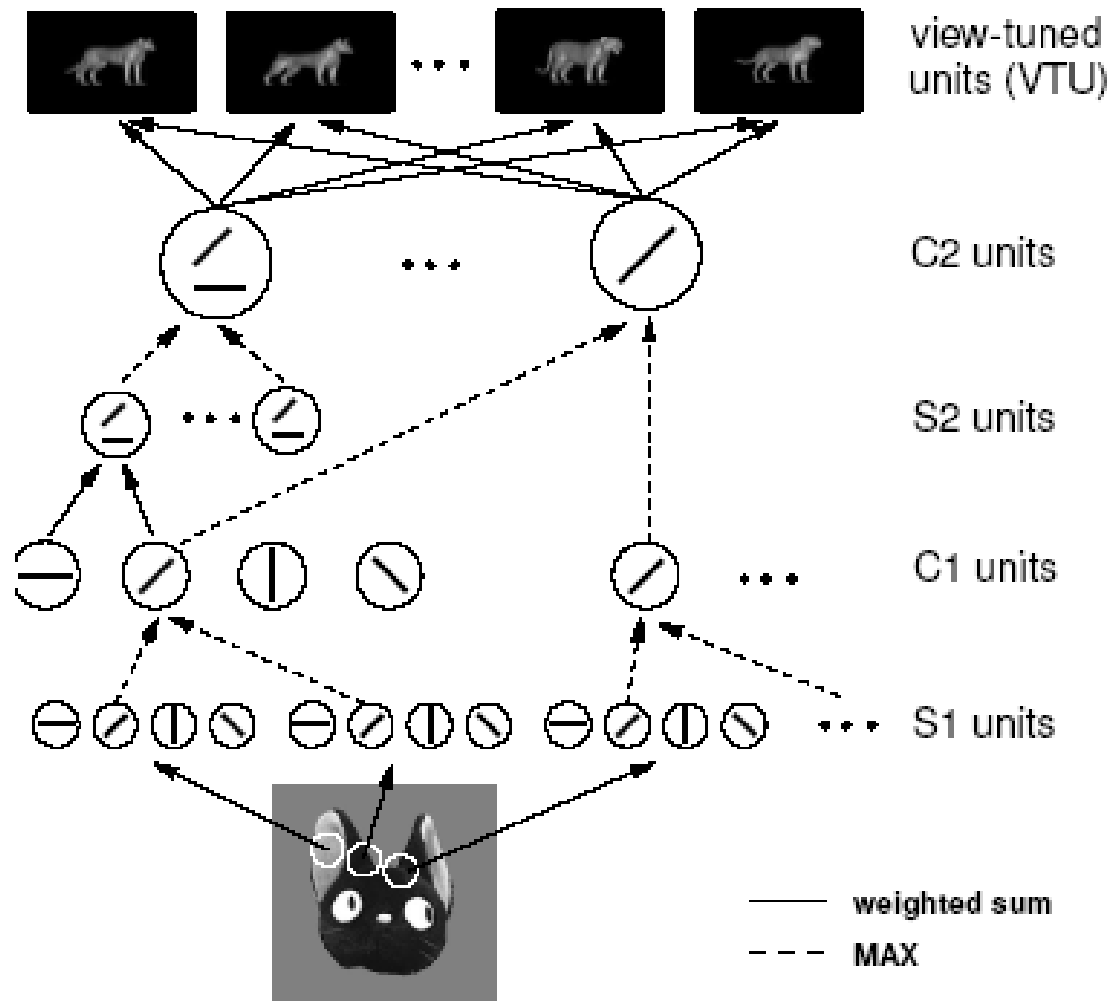
Maximilian Riesenhuber, Tomaso Poggio, Hierarchical models of object recognition in cortex, Nature 1999



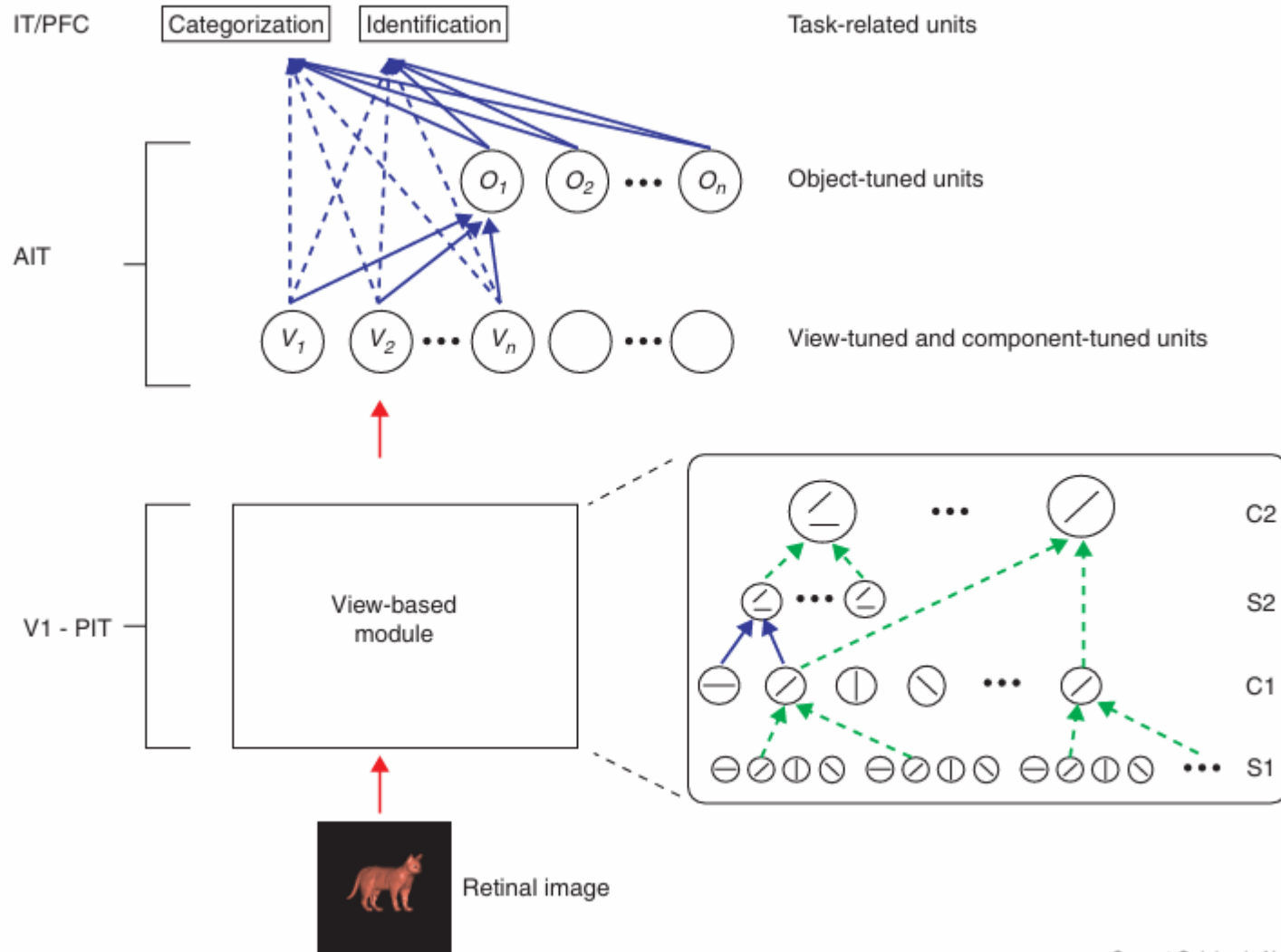
Maximilian Riesenhuber, Tomaso Poggio, Models of object recognition, Nature 2000



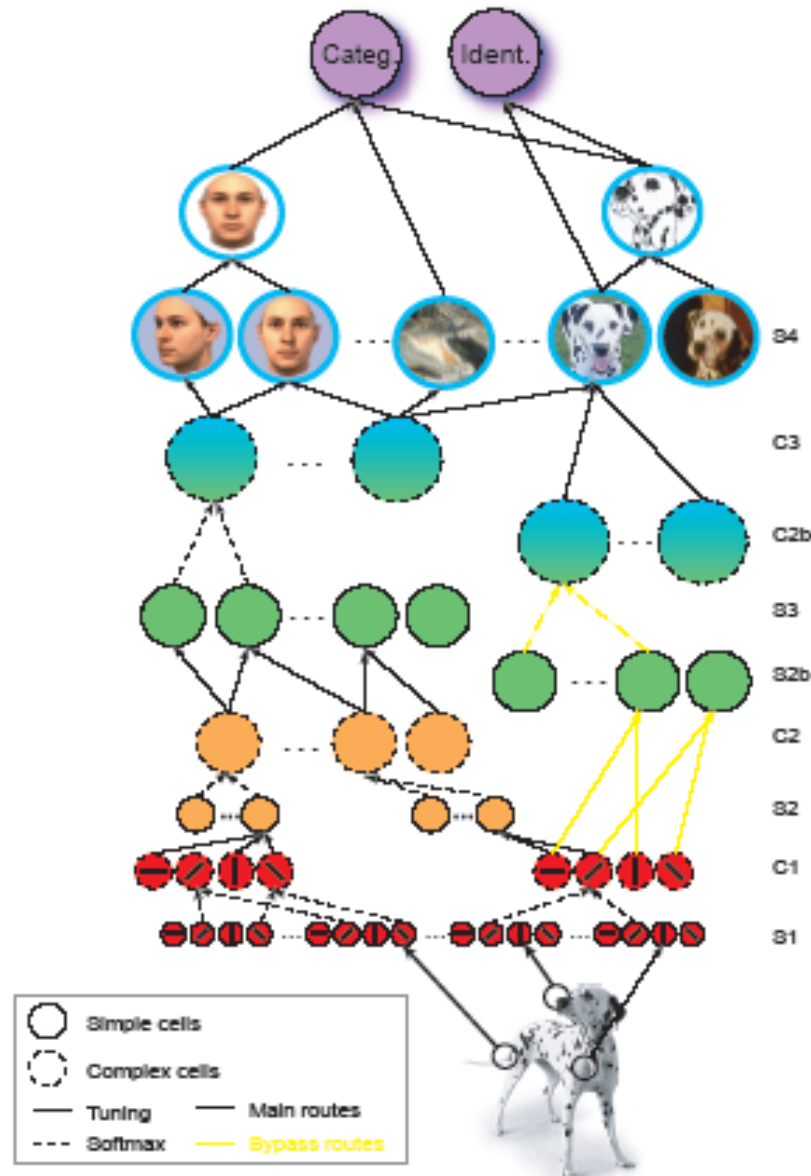
- view-based module:



Maximilian Riesenhuber, Tomaso Poggio, Neural mechanisms of object recognition, Current Opinion in Neurobiology, 2002

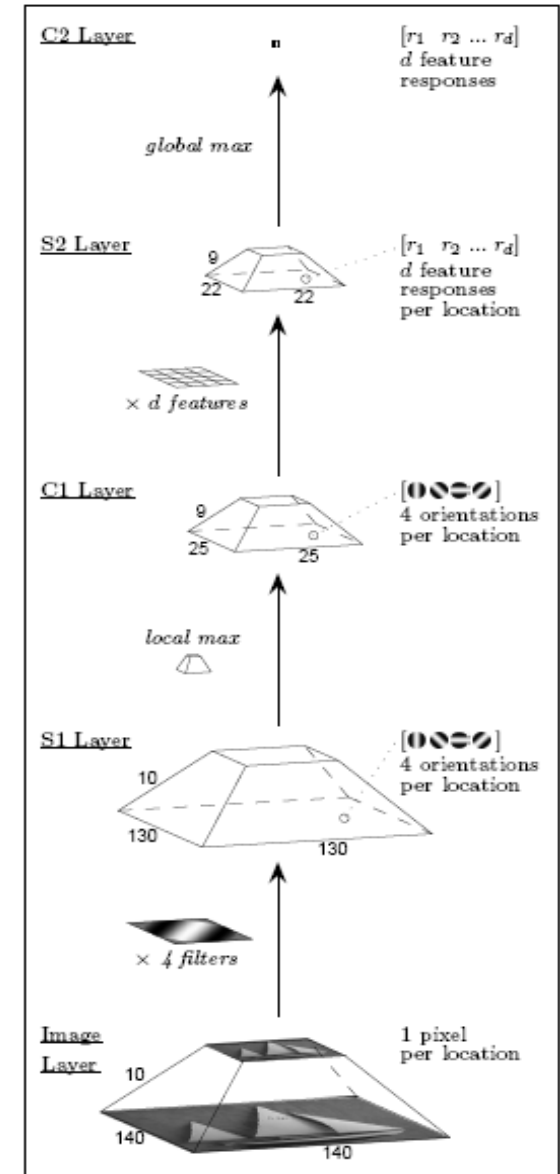


Maximilian Riesenhuber, Tomaso Poggio, Neural mechanisms of object recognition, Current Opinion in Neurobiology, 2002



Thomas Serre, Lior Wolf, Tomaso Poggio, Object Recognition with Features Inspired by Visual Cortex, CVPR 2005

Band Σ	1	2	3	4	5	6	7	8
filt. sizes s	7 & 9	11 & 13	15 & 17	19 & 21	23 & 25	27 & 29	31 & 33	35 & 37
σ	2.8 & 3.6	4.5 & 5.4	6.3 & 7.3	8.2 & 9.2	10.2 & 11.3	12.3 & 13.4	14.6 & 15.8	17.0 & 18.2
λ	3.5 & 4.6	5.6 & 6.8	7.9 & 9.1	10.3 & 11.5	12.7 & 14.1	15.4 & 16.8	18.2 & 19.7	21.2 & 22.8
grid size N^Σ	8	10	12	14	16	18	20	22
orient. θ	$0; \frac{\pi}{4}; \frac{\pi}{2}; \frac{3\pi}{4}$							
patch sizes n_i	$4 \times 4; 8 \times 8; 12 \times 12; 16 \times 16$ ($\times 4$ orientations)							





The end !